PRESERVE

– Code and Dataset

Checkpoint instead of kill under memory pressure

BITE303P Operating Systems Lab

VIT Vellore

Team

Arvind Gotluru (24BIT0176)

S G Subhiksha (24BIT0188)

Sai Bharath Chandra Nannur (24BIT0138)

Guide

Dr. G.N. Vivekananda

Generated 2026-10-25 from the project tree at ~/preserve

2. Table of contents

1. Cover page 1

2. Table of contents 2

3. Project folder tree

Project folder tree 3

4. Source code

src/governor.py (1771 lines) 6
src/admission.py (222 lines) 28
src/capture.py (376 lines) 31
src/cgroup.py (504 lines) 36
src/compare.py (72 lines) 42
src/config.py (242 lines) 43
src/costmodel.py (259 lines) 46
src/deps.py (225 lines) 50
src/matrix.py (351 lines) 53
src/runlock.py (226 lines) 58
src/sensor.py (387 lines) 61
workloads/compute_job.py (188 lines) 66
workloads/hog.py (129 lines) 69
workloads/pipeline_job.py (206 lines) 71
preserve.conf (159 lines) 74
run_demo.sh (212 lines) 76
run_failsafe.sh (296 lines) 79
run_all.sh (266 lines) 83
run_overhead.sh (302 lines) 87
scripts/video_demo.sh (363 lines) 91

5. Dataset / Evidence

evidence/matrix-20261022-043246.txt (62 lines shown) 96
evidence/failsafe-20261022-160303.txt (33 lines shown) 98
evidence/overhead-20261022-053739.txt (49 lines shown) 99
evidence/video-20261025-003739/summary.txt (43 lines shown) 100
run/throughput.json (7 lines shown) 101
evidence/telemetry.csv (50 lines shown) 102

3. Project folder tree

Project folder tree

Excluded: .git, __pycache__, venv, arena images (.img), files > 5 MiB. Evidence run folders are listed but not expanded.*

```
preserve/
|-- arena/   (CRIU checkpoint images: excluded, binary)
|-- evidence/
|   |-- demo-20261013-235249/   (5 files, run evidence; not expanded)
|   |-- demo-20261014-000118/   (5 files, run evidence; not expanded)
|   |-- demo-20261014-002028-baseline/   (6 files, run evidence; not expanded)
|   |-- demo-20261014-002235-baseline/   (6 files, run evidence; not expanded)
|   |-- demo-20261014-002847-baseline-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261014-003439-live-B/   (5 files, run evidence; not expanded)
|   |-- demo-20261015-174127-baseline-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261015-175308-live-B/   (5 files, run evidence; not expanded)
|   |-- demo-20261021-221641-baseline-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261021-221753-live-B/   (5 files, run evidence; not expanded)
|   |-- demo-20261021-221938-baseline-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261021-222034-live-A/   (5 files, run evidence; not expanded)
|   |-- demo-20261021-223601-live-B/   (5 files, run evidence; not expanded)
|   |-- demo-20261021-223612-live-B/   (5 files, run evidence; not expanded)
|   |-- demo-20261021-223717-live-A/   (5 files, run evidence; not expanded)
|   |-- demo-20261021-223752-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261021-224133-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-032553-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-032703-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-032813-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-032920-baseline-B/   (5 files, run evidence; not expanded)
|   |-- demo-20261022-033029-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033030-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033031-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033031-live-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033032-live-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033033-live-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033034-baseline-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033034-live-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033035-baseline-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033036-baseline-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033037-baseline-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033037-live-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033038-live-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033039-live-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-033040-live-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-040713-live-B/   (2 files, run evidence; not expanded)
|   |-- demo-20261022-040845-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-041256-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-041406-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-041516-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-041626-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-041737-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-041747-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-041750-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-041752-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-041755-baseline-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-041758-baseline-B/   (2 files, run evidence; not expanded)
|   |-- demo-20261022-043246-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-043357-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-043508-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-043619-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-043730-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-043839-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-043949-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-044059-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-044208-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-044318-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-044428-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-044612-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-044759-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-044943-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-045129-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-045315-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-045500-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-045644-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-045828-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-050014-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-050159-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050256-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050352-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050447-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050544-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050641-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050736-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-050833-baseline-A/   (7 files, run evidence; not expanded)
```

Project folder tree (continued)

```
|   |-- demo-20261022-050928-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-051024-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-051120-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-051250-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-051420-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-051549-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-051719-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-051850-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-052021-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-052151-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-052320-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-052450-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-151834-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-152158-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-152623-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-152909-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-153018-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-153201-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-153309-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-153851-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-154301-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-154410-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-154553-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-154701-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-154844-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-154953-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-155136-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-155245-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-155428-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-155537-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-155720-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-155828-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-160012-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-160120-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261022-183053-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261022-183301-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261022-183841-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261022-183852-live-B/   (3 files, run evidence; not expanded)
|   |-- demo-20261022-184018-live-C/   (8 files, run evidence; not expanded)
|   |-- demo-20261024-235855-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261024-235955-live-A/   (3 files, run evidence; not expanded)
|   |-- demo-20261025-003743-baseline-A/   (7 files, run evidence; not expanded)
|   |-- demo-20261025-003840-live-A/   (6 files, run evidence; not expanded)
|   |-- demo-20261025-004010-baseline-B/   (7 files, run evidence; not expanded)
|   |-- demo-20261025-004120-live-B/   (6 files, run evidence; not expanded)
|   |-- demo-20261025-004306-baseline-C/   (9 files, run evidence; not expanded)
|   |-- demo-20261025-004418-live-C/   (8 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-arena-too-small/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-baseline/   (7 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-clean/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-dump-fail/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-dump-timeout/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-governor-dies/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-restore-blocked/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261021-232655-stale-pid/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-arena-too-small/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-baseline/   (7 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-clean/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-dump-fail/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-dump-timeout/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-governor-dies/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-restore-blocked/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-001107-stale-pid/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-arena-too-small/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-baseline/   (7 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-clean/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-dump-fail/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-dump-timeout/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-governor-dies/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-restore-blocked/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-003815-stale-pid/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-arena-too-small/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-baseline/   (7 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-clean/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-dump-fail/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-dump-timeout/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-governor-dies/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-restore-blocked/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-052621-stale-pid/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-arena-too-small/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-baseline/   (7 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-clean/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-dump-fail/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-dump-timeout/   (6 files, run evidence; not expanded)
```

Project folder tree (continued)

```
|   |-- failsafe-20261022-160303-governor-dies/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-restore-blocked/   (6 files, run evidence; not expanded)
|   |-- failsafe-20261022-160303-stale-pid/   (6 files, run evidence; not expanded)
|   |-- GOLD-dry-run/   (6 files, run evidence; not expanded)
|   |-- GOLD-live-run/   (5 files, run evidence; not expanded)
|   |-- GOLD-scenarioA-baseline/   (6 files, run evidence; not expanded)
|   |-- GOLD-scenarioB-baseline/   (6 files, run evidence; not expanded)
|   |-- GOLD-scenarioB-live/   (5 files, run evidence; not expanded)
|   |-- overhead-20261022-032513/   (11 files, run evidence; not expanded)
|   |-- overhead-20261022-032822/   (9 files, run evidence; not expanded)
|   |-- overhead-20261022-053739/   (23 files, run evidence; not expanded)
|   |-- throughput-persistence/   (3 files, run evidence; not expanded)
|   |-- video-20261024-235847/   (6 files, run evidence; not expanded)
|   |-- video-20261025-003739/   (28 files, run evidence; not expanded)
|   |-- .failsafe-20261021-231234.rows
|   |-- .failsafe-20261021-232655.rows
|   |-- .failsafe-20261021-235738.rows
|   |-- .failsafe-20261022-001107.rows
|   |-- .failsafe-20261022-003144.rows
|   |-- .failsafe-20261022-003555.rows
|   |-- .failsafe-20261022-003815.rows
|   |-- .failsafe-20261022-052621.rows
|   |-- .failsafe-20261022-160303.rows
|   |-- 00_criu_feasibility.log
|   |-- ARENA-COST-NOTE.md
|   |-- C-extra7.log
|   |-- dumpfail-recheck.log
|   |-- failsafe-20261021-232655.txt
|   |-- failsafe-20261022-001107.txt
|   |-- failsafe-20261022-003815.txt
|   |-- failsafe-20261022-052621.txt
|   |-- failsafe-20261022-160303.txt
|   |-- final-failsafe.log
|   |-- GOLD-dumpfail-ENOSPC.log
|   |-- GOLD-failsafe.rows
|   |-- GOLD-failsafe.txt
|   |-- GOLD-live-telemetry.csv
|   |-- GOLD-matrix.txt
|   |-- governor.log
|   |-- matrix-20261021-221549.jsonl
|   |-- matrix-20261021-221549.txt
|   |-- matrix-20261022-032552.jsonl
|   |-- matrix-20261022-032552.txt
|   |-- matrix-20261022-041252.jsonl
|   |-- matrix-20261022-043246.arena
|   |-- matrix-20261022-043246.jsonl
|   |-- matrix-20261022-043246.txt
|   |-- night-failsafe.log
|   |-- night-matrix.log
|   |-- night-overhead.log
|   |-- overhead-20261022-032513.txt
|   |-- overhead-20261022-053739.txt
|   |-- repeat10-console.log
|   `-- telemetry.csv
|-- run/
|   |-- compute_job.progress.log
|   |-- compute_job.state.json
|   |-- governor.lock
|   |-- hog.state.json
|   `-- throughput.json
|-- scripts/
|   `-- video_demo.sh
|-- src/
|   |-- admission.py
|   |-- capture.py
|   |-- cgroup.py
|   |-- compare.py
|   |-- config.py
|   |-- costmodel.py
|   |-- deps.py
|   |-- governor.py
|   |-- matrix.py
|   |-- runlock.py
|   `-- sensor.py
|-- workloads/
|   |-- compute_job.py
|   |-- hog.py
|   `-- pipeline_job.py
|-- preserve.conf
|-- README.md
|-- run_all.sh
|-- run_demo.sh
|-- run_failsafe.sh
`-- run_overhead.sh
```

4. Source code

src/governor.py (1771 lines)

```
"""
governor.py -- PRESERVE main loop: sense -> classify -> score -> admit -> act.

The problem
-----------
When a memory cgroup runs out of room, Linux has one remedy left: the OOM
killer sends SIGKILL to a victim, and everything that process had computed is
lost. PRESERVE is a user-space governor that offers a gentler remedy: under
pressure it **checkpoints** a valuable process to disk with CRIU (which frees
its RAM), and when pressure has passed it **restores** the process, which
continues from the exact instruction where it stopped.

Control loop
------------
Every ``sample_interval_s`` the governor runs one step::

    M1 sensor     read PSI (memory.pressure) + memory.current/max/events
        |
    classifier    NORMAL / ADVISORY / ACTION, with hysteresis for recovery
        |
    M2 costmodel  (ACTION only) score each managed process: restart cost R
        |         vs capture cost C
    M3 admission  is the best candidate worth it, will it fit in the arena,
        |         and will the dump finish in time?
    M4 capture    criu dump -> process killed, memory released
        ...
    (later, once pressure stayed below the recovery thresholds)
    M3 admission  does the cgroup have headroom for the process again?
    M4 capture    criu restore -> process resumes, confined to the cgroup

**Pressure levels.** The classifier combines two signals:

* PSI ``some`` avg10 of the *test cgroup* (percent of time tasks stalled
  on memory), compared with ``advisory_pct`` and ``action_pct``.
* Usage ratio ``memory.current / memory.max``, compared with
  ``usage_advisory_frac`` and ``usage_action_frac``. Usage alone is not
  pressure (page cache can fill a cgroup harmlessly), but anonymous memory
  close to the limit with little swap may reach OOM before stall time
  accumulates. The usage trigger covers that case.

**Hysteresis.** Restoring as soon as pressure dips would recreate it and lead
to dump/restore thrashing, the same pathology as page thrashing. Restore is
therefore only considered after ``recovery_hold_samples`` *consecutive*
samples below the separate, lower ``recovery_pct`` / ``usage_recovery_frac``
thresholds. The counter is reset after every restore.

**Cooldown.** After a capture the governor waits ``capture_cooldown_s``
before evicting another process. Freed memory needs a moment to show up in
the PSI averages, and acting again too early would evict more than needed.

Safety
------
The governor only acts through :class:`cgroup.PidWhitelist`. It contains
exactly the processes this governor launched into the test cgroup, and its
``verify`` method hard-refuses everything else. See ``cgroup.py`` for the
full rule list.

Demo scenario (``run_demo.sh``)
-------------------------------
1. Create the test cgroup with the limits from ``preserve.conf``.
2. Launch ``compute_job`` (valuable: accumulates CPU work and holds a
   resident working set).
3. After ``hog_start_delay_s``, launch ``hog``, which allocates until the
   cgroup is under pressure.
4. Expected: the governor ranks compute_job as worth preserving (high R, low
   C) and the hog as not worth it (no work to lose). It checkpoints
   compute_job, the hog finishes, pressure recovers, and compute_job is
   restored. The compute_job progress log must then show the iteration
   counter *continuing*, not restarting at 1.
5. Shut down: SIGTERM to remaining managed processes, remove the cgroup, and
   print a summary including the continuity verdict.

Telemetry for plotting is appended to ``evidence/telemetry.csv``, one row per
sample.

Modes
-----
* ``live``: the full mechanism (requires root for CRIU).
* ``dry-run``: everything except CRIU. Decisions are logged only.
* ``baseline``: the *control experiment* PRESERVE is measured against.
```

```
  Same workloads, cgroup and thresholds, but the governor only observes:
  it logs PSI, memory, the scoring table and what it *would* have
  checkpointed, and leaves the kernel's cgroup-local OOM killer to act. Swap
  is set to ``[cgroup] baseline_memory_swap_max`` (normally 0) so that the
  OOM killer is not just deferred by swapping. At the end the governor
  reports ``memory.events:oom_kill``, which process the kernel killed (from
  the kernel log), how far compute_job had got, how much work was destroyed,
  and a side-by-side comparison with a saved live run
  (``[baseline] reference_live_run``). Because no CRIU is involved, baseline
  mode does not need root when the governor itself is started inside the
  user's delegated cgroup subtree (``systemd-run --user --scope``).

Usage::

    sudo python3 src/governor.py               # live (CRIU requires root)
    sudo python3 src/governor.py --dry-run     # decide and log, never run CRIU
    systemd-run --user --scope python3 src/governor.py --baseline
    sudo python3 src/governor.py --no-hog      # control run without the antagonist
"""
from __future__ import annotations

import argparse
import configparser
import csv
import enum
import json
import logging
import math
import os
import re
import resource
import shutil
import signal
import subprocess
import sys
import time
from pathlib import Path

from admission import AdmissionController, Arena, tree_bytes
from capture import CriuCapture
from cgroup import PRESERVED, RUNNING, CgroupError, ManagedProcess, PidWhitelist, SafetyRefusal, TestCgroup
from config import Config, give_to_invoking_user, invoking_ids, setup_logging
from runlock import EXIT_LOCKED, LockHeld, RunLock, holder_info
from costmodel import CostModel, Score
import deps
from sensor import MIB, Sample, Sensor, format_sample, read_cgroup_memory, read_kernel_oom_kills, read_uptime

log = logging.getLogger("preserve.governor")

MODES = ("live", "dry-run", "baseline", "overhead")
OVERHEAD_RESULT = "overhead_result.json"
# Exit codes that mean "no experiment ran" (distinct from the 0-4 outcome codes):
#   75 (runlock.EXIT_LOCKED)  another governor holds the lock
#   76 (EXIT_SETUP_FAILED)    setup aborted, e.g. could not create an exclusive cgroup
EXIT_SETUP_FAILED = 76
BASELINE_RESULT = "baseline_result.json"

# Fault injection (live mode only), used to demonstrate the fail-safe property:
# every path that does not reach ADMIT must end in STAND DOWN, and in STAND DOWN
# the system behaves no worse than the unmodified kernel would. Each fault forces
# one real precondition to fail; none of them weakens the scoring, admission or
# capture logic -- they only make a genuine check fire so its handling is observed.
FAULTS = ("arena-too-small", "dump-fail", "dump-timeout", "restore-blocked", "stale-pid", "governor-dies")
FAULT_ARENA_CAPACITY_BYTES = 1 << 20  # arena-too-small: budget far below any real image (~307 MiB)

COMPUTE_STATE = "compute_job.state.json"
COMPUTE_PROGRESS = "compute_job.progress.log"
HOG_STATE = "hog.state.json"
# Objective 6: pipeline_job = parent + forked child joined by a pipe (one dependent group).
PIPELINE_ROLES = {"parent": "pipeline", "child": "pipeline-child"}
PIPELINE_STATE = "pipeline_job.{}.state.json"
PIPELINE_PROGRESS = "pipeline_job.{}.progress.log"
RUN_FILES = (COMPUTE_STATE, COMPUTE_STATE + ".tmp", COMPUTE_PROGRESS, HOG_STATE, HOG_STATE + ".tmp") + tuple(
    f.format(r) + sfx for r in PIPELINE_ROLES for f, sfx in
    ((PIPELINE_STATE, ""), (PIPELINE_STATE, ".tmp"), (PIPELINE_PROGRESS, "")))

TELEMETRY_FIELDS = [
    "wall_time", "elapsed_s", "level", "sys_some_avg10", "cg_some_avg10", "cg_full_avg10",
    "mem_current_mib", "mem_max_mib", "swap_current_mib", "usage_frac", "oom_kill",
    "running", "preserved", "event",
]

PROGRESS_RE = re.compile(r"^(\d+\.\d+) iteration=(\d+) work_s=([\d.]+) .*pid=(\d+)")

```

src/governor.py (1771 lines) (continued)

```
165
166 class Level(enum.Enum):
167     NORMAL = "normal"
168     ADVISORY = "advisory"
169     ACTION = "action"
170
171
172 class PressureClassifier:
173     """Maps a sample to a pressure level and tracks recovery with hysteresis."""
174
175     def __init__(self, cfg: Config) -> None:
176         p, g = "psi", "governor"
177         self.source = cfg.get_str(p, "source")
178         self.metric = cfg.get_str(p, "metric")
179         self.window = cfg.get_str(p, "window")
180         self.advisory_pct = cfg.get_float(p, "advisory_pct")
181         self.action_pct = cfg.get_float(p, "action_pct")
182         self.recovery_pct = cfg.get_float(p, "recovery_pct")
183         self.hold = cfg.get_int(p, "recovery_hold_samples")
184         self.usage_advisory = cfg.get_float(g, "usage_advisory_frac")
185         self.usage_action = cfg.get_float(g, "usage_action_frac")
186         self.usage_recovery = cfg.get_float(g, "usage_recovery_frac")
187         if self.source not in ("cgroup", "system"):
188             raise ValueError("[psi] source must be cgroup or system")
189         if self.metric not in ("some", "full") or self.window not in ("avg10", "avg60", "avg300"):
190             raise ValueError("[psi] metric must be some|full and window avg10|avg60|avg300")
191         if not (self.recovery_pct < self.advisory_pct <= self.action_pct):
192             raise ValueError("[psi] need recovery_pct < advisory_pct <= action_pct")
193         if not (self.usage_recovery < self.usage_advisory <= self.usage_action):
194             raise ValueError("[governor] need usage_recovery < usage_advisory <= usage_action")
195         self.calm_samples = 0
196
197     def signals(self, s: Sample) -> tuple[float, float]:
198         """(PSI percentage, usage fraction) used for classification."""
199         psi = s.cgroup.pressure if (self.source == "cgroup" and s.cgroup is not None) else s.system_psi
200         return psi.get(self.metric, self.window), (s.cgroup.usage_frac if s.cgroup else 0.0)
201
202     def classify(self, s: Sample) -> Level:
203         psi, usage = self.signals(s)
204         if psi >= self.action_pct or usage >= self.usage_action:
205             level = Level.ACTION
206         elif psi >= self.advisory_pct or usage >= self.usage_advisory:
207             level = Level.ADVISORY
208         else:
209             level = Level.NORMAL
210         calm = psi <= self.recovery_pct and usage <= self.usage_recovery
211         self.calm_samples = self.calm_samples + 1 if calm else 0
212         return level
213
214     @property
215     def recovered(self) -> bool:
216         return self.calm_samples >= self.hold
217
218     def reset_recovery(self) -> None:
219         self.calm_samples = 0
220
221
222 def progress_rows(path: Path) -> list[tuple[float, int, float, int]]:
223     """(wall_time, iteration, work_s, pid) for every line of a compute_job progress log."""
224     try:
225         lines = path.read_text().splitlines()
226     except OSError:
227         return []
228     return [(float(m[1]), int(m[2]), float(m[3]), int(m[4])) for m in map(PROGRESS_RE.match, lines) if m]
229
230
231 def largest_gap(rows: list[tuple[float, int, float, int]]) -> tuple[float, int]:
232     """(gap_seconds, index i) of the longest pause between rows[i] and rows[i+1]."""
233     return max((rows[k + 1][0] - rows[k][0], k) for k in range(len(rows) - 1))
234
235
236 def progress_continuity(path: Path) -> tuple[str, str]:
237     """Judge from compute_job's progress log whether it RESUMED or RESTARTED.
238
239     Finds the largest wall-clock gap between consecutive progress lines (the
240     time spent checkpointed) and checks that the iteration counter never went
241     backwards and that the PID stayed the same across the gap.
242     """
243     rows = progress_rows(path)
244     if len(rows) < 2:
245         return "NO-DATA", f"only {len(rows)} progress line(s) in {path}"
246     gap, i = largest_gap(rows)
247     (_, it0, _, pid0), (_, it1, _, pid1) = rows[i], rows[i + 1]
248     backwards = [(a[1], b[1]) for a, b in zip(rows, rows[1:]) if b[1] <= a[1]]
```

src/governor.py (1771 lines) (continued)

```
    if backwards:
        return "RESTARTED", (f"iteration counter went backwards {backwards[:3]}: "
                             "the job started again from scratch")
    return "RESUMED", (f"largest gap {gap:.1f}s between iteration {it0} (pid {pid0}) and "
                       f"iteration {it1} (pid {pid1}); counter never went backwards "
                       f"over {len(rows)} lines (last iteration {rows[-1][1]})")


def load_reference_run(path: Path) -> dict:
    """Headline numbers from a saved live-run evidence directory (read-only).

    Used by baseline mode to print PRESERVE and the kernel OOM killer side by
    side. Reads the run's preserve.conf, compute_job progress log and final
    state, and the console log (for captures/restores and oom_kill).
    """
    ref: dict = {"path": str(path), "available": path.is_dir()}
    if not ref["available"]:
        return ref
    try:
        console = (path / "console.log").read_text(errors="replace")
    except OSError:
        console = ""
    # Prefer what the kernel actually reported for the run (scenario overrides
    # may differ from the base value in the saved preserve.conf).
    m = re.search(r"memory\.swap\.max = (\S+)", console)
    if m:
        ref["swap_max"] = m[1]
    else:
        cp = configparser.ConfigParser(inline_comment_prefixes=("#",), interpolation=None)
        cp.read(path / "preserve.conf")
        ref["swap_max"] = cp.get("cgroup", "memory_swap_max", fallback="?")
    m = re.search(r"scenario=(\S+)", console)
    ref["scenario"] = m[1] if m else "A"  # runs before scenarios existed used the base values
    rows = progress_rows(path / COMPUTE_PROGRESS)
    if len(rows) >= 2:
        gap, i = largest_gap(rows)
        ref.update(gap_s=gap, before_iteration=rows[i][1], before_work_s=rows[i][2],
                   after_iteration=rows[i + 1][1], resumed=all(b[1] > a[1] for a, b in zip(rows, rows[1:])))
    try:
        state = json.loads((path / COMPUTE_STATE).read_text())
        ref.update(final_iteration=state.get("iteration"), final_work_s=state.get("work_seconds"))
    except (OSError, ValueError):
        pass
    m = re.search(r"captures=(\d+)\s+restores=(\d+)", console)
    if m:
        ref.update(captures=int(m[1]), restores=int(m[2]))
    kills = [int(x) for x in re.findall(r"oom_kill=(\d+)", console)]
    ref["oom_kill"] = max(kills) if kills else None
    return ref


def side_by_side_table(ref: dict, base: dict) -> list[tuple[str, object, object]]:
    """Rows comparing a live PRESERVE run (``ref``) with a baseline result (``base``).

    ``ref`` comes from :func:`load_reference_run`, ``base`` is the
    ``baseline_result.json`` document written by baseline mode.
    """
    victims = base.get("victims", [])
    if base["compute_killed"]:
        b_fate = f"killed by kernel at iteration {base['compute_last_iteration']}"
        b_kept, b_lost = "0.00s", f"{base['compute_work_lost_s']:.2f}s"
    else:
        b_fate = "not selected by the OOM killer; ran on"
        b_kept, b_lost = f"{base['compute_final_work_s'] or 0:.2f}s (never interrupted)", "0.00s"
    killed = ", ".join(f"{v['role']} pid={v['pid']}" for v in victims) or "none"
    killed_work = sum(v.get("work_lost_s", 0.0) for v in victims)
    if ref.get("resumed") and ref.get("captures"):
        r_fate = f"checkpointed at iteration {ref.get('before_iteration')}, resumed at {ref.get('after_iteration')}"
        r_kept = f"{ref.get('before_work_s', 0):.2f}s at checkpoint"
        r_pause = f"{ref.get('gap_s', 0):.1f}s (checkpointed)"
    else:
        r_fate, r_kept, r_pause = "see reference log", "?", "?"
    swap = base["memory_swap_max"]
    return [
        ("", f"PRESERVE (live, scenario {ref.get('scenario')})",
         f"BASELINE (kernel OOM, scenario {base.get('scenario')})"),
        ("memory.swap.max", ref.get("swap_max"), "max" if swap is None else swap),
        ("governor checkpoints/restores", f"{ref.get('captures')}/{ref.get('restores')}", "0/0 (observe only)"),
        ("memory.events oom_kill", ref.get("oom_kill"), base["oom_kill"]),
        ("process(es) killed", "none" if not ref.get("oom_kill") else "see reference log", killed),
        ("compute_job fate", r_fate, b_fate),
        ("compute work preserved", r_kept, b_kept),
        ("compute work destroyed", "0.00s" if ref.get("resumed") else "?", b_lost),
        ("compute paused for", r_pause, "0s" if not base["compute_killed"] else "forever (dead)"),
```

```
        ("compute final iteration", ref.get("final_iteration"), base["compute_last_iteration"]),
        ("total work destroyed (all)", "0.00s" if not ref.get("oom_kill") else "?", f"{killed_work:.2f}s"),
    ]


# ---------------------------------------------------------------------------
# Arena housekeeping: --arena-status and --reap-arena
# ---------------------------------------------------------------------------
# Each image set carries a small state record so a reaper can tell an image that is
# the ONLY copy of a live process (checkpointed, never restored) from leftovers.
IMAGE_STATE = "preserve-state.json"
CRIU_DUMP_OK = "Dumping finished successfully"  # last line CRIU logs after a complete dump


def write_image_state(image_dir: Path | None, state: str, **fields: object) -> None:
    """Record inside an image set whether it still holds a not-yet-restored process."""
    if image_dir is None:
        return
    try:
        (Path(image_dir) / IMAGE_STATE).write_text(
            json.dumps({"state": state, "updated": time.time(), **fields}, indent=2))
    except OSError as exc:
        log.warning("could not record image state in %s: %s", image_dir, exc)


def classify_image(image_dir: Path) -> tuple[bool, str]:
    """(keep, why) for one image set. Keeping is the default whenever in doubt."""
    try:
        record = json.loads((image_dir / IMAGE_STATE).read_text())
    except FileNotFoundError:
        record = None
    except (OSError, ValueError) as exc:
        return True, f"state record unreadable ({exc}); kept"
    if record is not None:
        if record.get("state") == "checkpointed":
            return True, (f"KEPT: pid {record.get('pid')} ({record.get('role')}) is still checkpointed "
                          f"and was never restored -- this image is its only copy")
        if record.get("state") == "restored":
            return False, f"restored (pid {record.get('pid')}); image no longer needed"
        return True, f"unknown state {record.get('state')!r}; kept"
    # No record: an image made before records existed, or a dump that never finished.
    try:
        complete = CRIU_DUMP_OK in (image_dir / "dump.log").read_text(errors="replace")
    except OSError:
        complete = False
    if complete:
        return True, ("KEPT: complete dump with no state record (made before records existed); it may "
                      "be the only copy of a process -- delete by hand only if you are sure")
    return False, "partial or failed dump (CRIU never reported success)"


def _age(seconds: float) -> str:
    s = int(max(0, seconds))
    if s >= 86400:
        return f"{s // 86400}d{(s % 86400) // 3600}h"
    if s >= 3600:
        return f"{s // 3600}h{(s % 3600) // 60}m"
    if s >= 60:
        return f"{s // 60}m"
    return f"{s}s"


def _scan_arena(arena: Arena) -> list[Path]:
    if not arena.path.exists():
        return []
    return sorted(p for p in arena.path.iterdir() if p.is_dir())


def _arena_readable(arena: Arena) -> bool:
    return not arena.path.exists() or os.access(arena.path, os.R_OK | os.X_OK)


def arena_status(cfg: Config) -> int:
    """Print occupancy, capacity and every image with size, age and fate-if-reaped."""
    arena = Arena.from_config(cfg)
    if not _arena_readable(arena):
        # Never report 0: os.walk on an unreadable directory silently yields nothing.
        print(f"ARENA NOT READABLE by uid {os.geteuid()}: {arena.path} -- run with sudo")
        return 1
    lock = cfg.get_path("paths", "lock_file")
    from runlock import describe_held, is_held, read_holder
    images = _scan_arena(arena)
    used = arena.used_bytes()
    print(f"arena     : {arena.path}")
```

```
417     print(f"occupancy : {used / MIB:.1f} MiB in {len(images)} image set(s) "
418           f"of {arena.capacity_bytes / MIB:.0f} MiB budget ({100 * used / arena.capacity_bytes:.1f}%)")
419     print(f"fs free   : {arena.fs_free_bytes() / MIB:.0f} MiB (admission keeps {arena.fs_reserve_bytes / MIB:.0f} Mi
  + B spare)")
420     print(f"lock      : " + (f"HELD -- {describe_held(read_holder(lock))}; images may be in use"
421                              if is_held(lock) else "free"))
422     now = time.time()
423     for d in images:
424         keep, why = classify_image(d)
425         print(f"  {d.name:44s} {tree_bytes(d) / MIB:8.1f} MiB  age {_age(now - d.stat().st_mtime):>6s}  "
426               f"{'keep  ' if keep else 'reap  '} {why}")
427     return 0
428
429
430 def reap_arena(cfg: Config) -> int:
431     """Delete every image set except those that are the only copy of a live process."""
432     arena = Arena.from_config(cfg)
433     lock = RunLock(cfg.get_path("paths", "lock_file"))
434     try:
435         # Holding the governor lock: no governor can start (and own fresh images) mid-reap,
436         # and none is running now, so no in-flight reservation exists anywhere.
437         lock.acquire(holder_info("reap-arena", None, sys.argv))
438     except LockHeld as exc:
439         print(f"ARENA: NOT REAPED -- {exc}")
440         return EXIT_LOCKED
441     try:
442         if not _arena_readable(arena):
443             print(f"ARENA: NOT REAPED -- {arena.path} is not readable by uid {os.geteuid()} (run with sudo)")
444             return 1
445         before, images = arena.used_bytes(), _scan_arena(arena)
446         deleted, freed, kept_live, kept_other = 0, 0, 0, 0
447         for d in images:
448             keep, why = classify_image(d)
449             size = tree_bytes(d)
450             if keep:
451                 log.warning("reap-arena: %s (%.1f MiB) -- %s", d.name, size / MIB, why)
452                 if why.startswith("KEPT: pid"):
453                     kept_live += 1
454                 else:
455                     kept_other += 1
456                 continue
457             if not arena.contains(d):
458                 log.error("reap-arena: refusing to delete %s: outside the arena", d)
459                 kept_other += 1
460                 continue
461             shutil.rmtree(d)
462             deleted, freed = deleted + 1, freed + size
463             log.info("reap-arena: deleted %s (%.1f MiB): %s", d.name, size / MIB, why)
464         after = arena.used_bytes()
465         # There is no persisted arena accounting to reset: occupancy is measured from
466         # disk each time, and reservations live only inside a running governor.
467         line = (f"ARENA: before {before / MIB:.1f} MiB in {len(images)} image(s); deleted {deleted} "
468                 f"({freed / MIB:.1f} MiB); kept {kept_live} still-checkpointed"
469                 + (f" + {kept_other} unrecorded/unknown" if kept_other else "")
470                 + f"; now {after / MIB:.1f} of {arena.capacity_bytes / MIB:.0f} MiB")
471         log.info(line)
472         print(line)
473         return 0
474     finally:
475         lock.release()
476
477
478 class Governor:
479     """Owns the test cgroup, the managed processes and the control loop."""
480
481     def __init__(self, cfg: Config, mode: str, reset_throughput: bool = False,
482                  inject: str | None = None) -> None:
483         if mode not in MODES:
484             raise ValueError(f"mode must be one of {MODES}")
485         if inject is not None and inject not in FAULTS:
486             raise ValueError(f"--inject must be one of {FAULTS}")
487         if inject is not None and mode != "live":
488             raise ValueError("--inject is only meaningful in live mode")
489         self.cfg = cfg
490         self.mode = mode
491         self.reset_throughput = reset_throughput
492         self.inject = inject
493         self.fault_stood_down = False  # set once the injected fault has forced a stand-down
494         # Self-verification bookkeeping: did the injected fault's precondition actually occur?
495         self.injection_failed = False     # the fault did NOT fire (e.g. a dump that should fail succeeded)
496         self.injection_reason = ""        # why it is considered failed/unverified
497         self.fault_dump_attempted = False
498         self.fault_dump_failed = False
499         self.fault_dump_enospc = False
```

```
        self.fault_setup_failed = False
        self.scenario = cfg.scenario
        self.observe_only = mode != "live"  # dry-run and baseline never invoke CRIU
        self.uid, self.gid = invoking_ids()
        self.cgroup = TestCgroup.for_run(cfg)  # unique per run: never shared with another run
        self.whitelist = PidWhitelist(self.cgroup)
        self.sensor = Sensor(self.cgroup.path)
        self.cost = CostModel(cfg)
        self.arena = Arena.from_config(cfg)
        self.admission = AdmissionController(cfg, self.arena)
        self.capture = CriuCapture(cfg, self.whitelist, self.cgroup, self.arena)
        self.classifier = PressureClassifier(cfg)
        self.run_dir = cfg.get_path("paths", "run_dir")
        self.throughput_file = cfg.get_path("paths", "throughput_state")

        g = "governor"
        self.interval = cfg.get_float("sensor", "sample_interval_s")
        self.log_every = cfg.get_int("sensor", "log_every_n_samples")
        self.cooldown_s = cfg.get_float(g, "capture_cooldown_s")
        self.decision_log_interval_s = cfg.get_float(g, "decision_log_interval_s")
        self.restore_max_attempts = cfg.get_int(g, "restore_max_attempts")
        self.grace_s = cfg.get_float(g, "terminate_grace_s")

        # --measure-overhead bookkeeping (mode "overhead")
        self.overhead_duration_s = cfg.get_float("overhead", "duration_s")
        self.step_times: list[float] = []
        self.level_counts: dict[str, int] = {}
        self.compute_args: list[str] = []
        self.overhead: dict = {}

        self.start = time.monotonic()
        self.level = Level.NORMAL
        self.samples = 0
        self.captures = 0
        self.restores = 0
        self.last_capture = -math.inf
        self.last_decision_log = -math.inf
        self.last_oom_kill = 0
        self.restore_attempts: dict[int, int] = {}
        self.preserved_rss: dict[int, int] = {}
        self.restored_pids: set[int] = set()  # successfully restored at least once (continuity verdict)
        # Last known facts about each managed process, kept so that a process
        # killed by the kernel can still be described after it is gone.
        self.start_uptime = read_uptime()
        self.swap_max_used: int | None = None
        self.last_state: dict[int, dict] = {}
        self.last_cpu: dict[int, float] = {}
        self.last_rss: dict[int, int] = {}
        self.exits: dict[int, tuple[float, int | None]] = {}  # pid -> (elapsed, returncode)
        self.first_oom_at: float | None = None
        # Objective 6: dependency groups
        self.workload = cfg.get_str("workloads", "workload")  # compute | pipeline
        self.logged_edges: set[tuple] = set()
        self.logged_groups: set[tuple[int, ...]] = set()
        self.group_standdowns: set[str] = set()
        self.group_of: dict[int, list[int]] = {}         # root pid -> members, for the current decision
        self.preserved_groups: dict[int, list[int]] = {}  # root pid -> members checkpointed together
        self.final_events: dict[str, int] = {}
        self.events: list[str] = []
        self._pending_event: list[str] = []
        self._stop = False
        self._telemetry_file = None
        self._telemetry: csv.writer | None = None

    # -- bookkeeping ------------------------------------------------------
    def elapsed(self) -> float:
        return time.monotonic() - self.start

    def event(self, msg: str, level: int = logging.INFO) -> None:
        """Log a significant event and remember it for the summary and telemetry."""
        log.log(level, msg)
        self.events.append(f"{self.elapsed():7.1f}s  {msg}")
        self._pending_event.append(msg)

    def request_stop(self, signum: int, _frame) -> None:
        log.warning("received %s; stopping after this step", signal.Signals(signum).name)
        self._stop = True

    # -- setup ------------------------------------------------------------
    def setup(self) -> None:
        if not self.observe_only:
            ok, msg = self.capture.available()
            if not ok:
                raise SystemExit(f"cannot use CRIU: {msg}. Use --dry-run to observe without capturing.")
```

```
            log.info("criu: %s", msg)
        self._init_throughput()
        c = "cgroup"
        swap_key = "baseline_memory_swap_max" if self.mode == "baseline" else "memory_swap_max"
        self.swap_max_used = self.cfg.get_size(c, swap_key)
        log.info("mode=%s: memory.swap.max = %s (from [cgroup] %s = %s)", self.mode,
                 "max" if self.swap_max_used is None else self.swap_max_used, swap_key,
                 self.cfg.get_str(c, swap_key))
        self.cgroup.create(self.cfg.get_size(c, "memory_max"), self.swap_max_used,
                           self.cfg.get_size(c, "memory_high"))
        self.arena.ensure()
        if self.inject == "arena-too-small":
            # Shrink the arena budget so the real M3 *capacity* test ("arena full")
            # is the check that fails. NB: do not do this with arena.reserve() --
            # a reservation also counts as an in-flight capture, so the DENY would
            # come from max_inflight_captures instead, mislabelling the fault.
            was = self.arena.capacity_bytes
            self.arena.capacity_bytes = FAULT_ARENA_CAPACITY_BYTES
            log.warning("FAULT INJECTION [arena-too-small]: arena budget shrunk from %.0f MiB to "
                        "%.2f MiB so the capture admission capacity test sees no room",
                        was / MIB, self.arena.capacity_bytes / MIB)
        log.info(self.arena.summary())
        self.run_dir.mkdir(parents=True, exist_ok=True)
        for name in RUN_FILES:
            (self.run_dir / name).unlink(missing_ok=True)
        (self.run_dir / BASELINE_RESULT).unlink(missing_ok=True)
        (self.run_dir / OVERHEAD_RESULT).unlink(missing_ok=True)
        if self._drop_to() is not None:
            os.chown(self.run_dir, self.uid, self.gid)
        self._open_telemetry()
        self.event(f"run start: scenario={self.scenario} mode={self.mode} "
                   f"memory.swap.max={self.cgroup.read('memory.swap.max')}"
                   + (f" inject={self.inject}" if self.inject else ""))

    def _stand_down(self, precondition: str) -> None:
        """Log a fault-induced stand-down, naming the fault and the failed precondition.

        STAND DOWN means: do not capture; leave the kernel in charge, exactly as
        if PRESERVE were not running. This is the fail-safe path the report claims.
        """
        self.fault_stood_down = True
        self.event(f"STAND DOWN [fault={self.inject}]: {precondition}; "
                   "not capturing -- kernel left in charge (fail-safe)", logging.WARNING)

    def _shown(self, path: Path) -> str:
        try:
            return str(path.relative_to(self.cfg.root))
        except ValueError:
            return str(path)

    def _init_throughput(self) -> None:
        """Load (or, with --reset-throughput, discard) the persisted dump-throughput estimate."""
        shown = self._shown(self.throughput_file)
        if self.reset_throughput:
            discarded = ""
            if self.throughput_file.exists():
                try:
                    old = json.loads(self.throughput_file.read_text()).get("throughput_mib_s")
                    discarded = f"saved estimate {float(old):.1f} MiB/s in {shown} " if old is not None else ""
                except (OSError, ValueError, TypeError):
                    discarded = f"unreadable {shown} "
                self.throughput_file.unlink()
            provenance = f"default, {discarded or 'no saved estimate '}discarded by --reset-throughput"
        else:
            provenance = self.cost.load_throughput(self.throughput_file, shown)
        log.info("dump throughput estimate %.1f MiB/s (%s)", self.cost.throughput_mib_s, provenance)

    def _save_throughput(self) -> None:
        try:
            self.cost.save_throughput(self.throughput_file)
            give_to_invoking_user(self.throughput_file)
            log.info("dump throughput estimate %.1f MiB/s saved to %s (%d observation(s))",
                     self.cost.throughput_mib_s, self._shown(self.throughput_file), self.cost.observations)
        except OSError as exc:
            log.warning("could not save throughput estimate to %s: %s", self.throughput_file, exc)

    def _drop_to(self) -> tuple[int, int] | None:
        """Credentials for workloads: the sudo caller, not root (if configured)."""
        if os.geteuid() != 0 or self.uid == 0 or not self.cfg.get_bool("workloads", "run_as_invoking_user"):
            return None
        return self.uid, self.gid

    def _open_telemetry(self) -> None:
        path = self.cfg.get_path("paths", "telemetry_csv")
```

```
        path.parent.mkdir(parents=True, exist_ok=True)
        new = not path.exists()
        self._telemetry_file = open(path, "a", newline="")
        self._telemetry = csv.writer(self._telemetry_file)
        if new:
            self._telemetry.writerow(TELEMETRY_FIELDS)
        give_to_invoking_user(path)

    # -- launching workloads ------------------------------------------------------
    def _launch(self, role: str, script: str, args: list[str], state_file: Path) -> ManagedProcess:
        python = self.cfg.get_str("workloads", "python_bin") or sys.executable
        argv = [python, str(self.cfg.root / "workloads" / script)] + args
        popen = subprocess.Popen(
            argv,
            stdin=subprocess.DEVNULL, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
            close_fds=True,
            start_new_session=True,  # setsid(): own session, no controlling tty
            cwd=self.run_dir,
            preexec_fn=self.cgroup.preexec_enter(self._drop_to()),  # join cgroup before exec
        )
        entry = self.whitelist.admit_launched(popen, role, state_file)
        self.event(f"launched {role} pid={popen.pid} into {self.cgroup.rel}")
        return entry

    def launch_compute(self) -> ManagedProcess:
        w = "workloads"
        state = self.run_dir / COMPUTE_STATE
        args = [
            "--state-file", str(state),
            "--progress-log", str(self.run_dir / COMPUTE_PROGRESS),
            "--working-set-mb", self.cfg.get_str(w, "compute_working_set_mb"),
            "--hashes-per-iteration", self.cfg.get_str(w, "compute_hashes_per_iteration"),
            "--step-sleep-s", self.cfg.get_str(w, "compute_step_sleep_s"),
            "--report-every", self.cfg.get_str(w, "compute_report_every"),
            "--touch-every", self.cfg.get_str(w, "compute_touch_every"),
            "--max-iteration-credit-s", self.cfg.get_str(w, "compute_max_iteration_credit_s"),
        ]
        # Recorded so an A/B run can start the *identical* workload without the governor.
        self.compute_args = list(args)
        return self._launch("compute", "compute_job.py", args, state)

    def launch_hog(self) -> ManagedProcess:
        w = "workloads"
        state = self.run_dir / HOG_STATE
        return self._launch("hog", "hog.py", [
            "--state-file", str(state),
            "--chunk-mb", self.cfg.get_str(w, "hog_chunk_mb"),
            "--interval-s", self.cfg.get_str(w, "hog_interval_s"),
            "--target-mb", self.cfg.get_str(w, "hog_target_mb"),
            "--hold-s", self.cfg.get_str(w, "hog_hold_s"),
            "--retouch-interval-s", self.cfg.get_str(w, "hog_retouch_interval_s"),
        ], state)

    def launch_pipeline(self) -> ManagedProcess:
        """pipeline_job: the parent is launched; its child is admitted when discovered."""
        w = "workloads"
        return self._launch(PIPELINE_ROLES["parent"], "pipeline_job.py", [
            "--run-dir", str(self.run_dir),
            "--parent-mb", self.cfg.get_str(w, "pipeline_parent_mb"),
            "--child-mb", self.cfg.get_str(w, "pipeline_child_mb"),
            "--hashes-per-iteration", self.cfg.get_str(w, "compute_hashes_per_iteration"),
            "--step-sleep-s", self.cfg.get_str(w, "compute_step_sleep_s"),
            "--report-every", self.cfg.get_str(w, "compute_report_every"),
            "--touch-every", self.cfg.get_str(w, "compute_touch_every"),
            "--max-iteration-credit-s", self.cfg.get_str(w, "compute_max_iteration_credit_s"),
        ], self.run_dir / PIPELINE_STATE.format("parent"))

    # -- dependency groups (Objective 6) ------------------------------------------
    def _discover_children(self) -> None:
        """Admit processes in the test cgroup forked by a whitelisted parent (cgroup.py rules)."""
        known = {e.pid for e in self.whitelist.with_state(RUNNING)}
        for pid in sorted(self.cgroup.procs() - known):
            snap = self.sensor.proc(pid)
            parent = self.whitelist.get(snap.ppid) if snap is not None else None
            if parent is None:
                continue
            state_file = None
            if parent.role == PIPELINE_ROLES["parent"]:
                role = PIPELINE_ROLES["child"]
                state_file = self.run_dir / PIPELINE_STATE.format("child")
            else:
                role = f"{parent.role}-child"
            if self.whitelist.admit_child(pid, role, state_file) is not None:
                self.event(f"admitted {role} pid={pid}: child of whitelisted {parent.role} pid={parent.pid}, "
```

```
                        f"inside {self.cgroup.rel}")

    def _groups(self) -> list[deps.Group]:
        """Dependency groups among running managed processes; logs every new edge with its proof."""
        groups = deps.build_groups([e.pid for e in self.whitelist.with_state(RUNNING)])
        for g in groups:
            for e in g.edges:
                if e.key not in self.logged_edges:
                    self.logged_edges.add(e.key)
                    self.event(f"DEPENDENCY {e.describe()}")
            key = tuple(g.members)
            if g.size > 1 and key not in self.logged_groups:
                self.logged_groups.add(key)
                roles = [self.whitelist.get(p).role for p in g.members]
                self.event(f"GROUP {list(key)} ({'+'.join(roles)}): {len(g.edges)} edge(s); "
                           "scored, checkpointed and restored as one unit")
        return groups

    def _member_fates(self) -> list[tuple[ManagedProcess, str]]:
        out = []
        for role in PIPELINE_ROLES.values():
            for e in self.whitelist.by_role(role):
                if e.pid in self.exits:
                    t, rc = self.exits[e.pid]
                    st = self._read_state(e)
                    how = (f"exited rc={rc}" if rc is not None and rc >= 0 else
                           f"killed by {signal.Signals(-rc).name}" if rc is not None else
                           "died (not the governor's child: exit status unknown)")
                    if st.get("partner_lost"):
                        how += f"; its own final state says partner lost ({st['partner_lost']})"
                    out.append((e, f"KILLED at {t:.1f}s, {how}"))
                elif e.state == PRESERVED:
                    out.append((e, f"CHECKPOINTED, NOT RESTORED (image {e.image_dir})"))
                elif e.pid in self.restored_pids:
                    out.append((e, "RESUMED"))
                else:
                    out.append((e, "RAN CONTINUOUSLY"))
        return out

    def _verify_chains(self) -> dict[str, tuple[bool, str]]:
        """Recompute every pipeline member's hash chain from its progress log (in parallel)."""
        python = self.cfg.get_str("workloads", "python_bin") or sys.executable
        procs = {}
        for short in PIPELINE_ROLES:
            path = self.run_dir / PIPELINE_PROGRESS.format(short)
            if not path.exists():
                continue
            procs[short] = subprocess.Popen(
                [python, str(self.cfg.root / "workloads" / "pipeline_job.py"), "--verify-log", str(path),
                 "--role", short, "--hashes-per-iteration",
                 self.cfg.get_str("workloads", "compute_hashes_per_iteration")],
                stdin=subprocess.DEVNULL, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
        out = {}
        for short, p in procs.items():
            text, _ = p.communicate()
            out[short] = (p.returncode == 0, text.strip().splitlines()[-1] if text.strip() else "(no output)")
        return out

    def group_fate(self) -> tuple[str, str]:
        """Group verdict from per-member events and hash chains. RESUMED only if EVERY member resumed."""
        fates = self._member_fates()
        if not fates:
            return "NO-DATA", "no pipeline group was launched"
        chains = self._verify_chains()
        for e, fate in fates:
            short = "parent" if e.role == PIPELINE_ROLES["parent"] else "child"
            ok, line = chains.get(short, (False, "no progress log"))
            log.info("  member %s pid=%d: %s; hash chain %s: %s", e.role, e.pid, fate,
                     "OK" if ok else "FAILED", line)
        n = len(fates)
        if n < len(PIPELINE_ROLES):
            return "INCOMPLETE", f"only {n} of {len(PIPELINE_ROLES)} members were ever whitelisted"
        resumed = [e for e, f in fates if f == "RESUMED"]
        killed = [e for e, f in fates if f.startswith("KILLED")]
        chains_ok = all(chains.get(s, (False, ""))[0] for s in PIPELINE_ROLES)
        if killed:
            return "KILLED", f"{len(killed)} of {n} members died"
        if len(resumed) == n and chains_ok:
            return "RESUMED", f"all {n} members checkpointed together, restored, alive, hash chains verified"
        if len(resumed) == n:
            return "NOT RESUMED", "all members restored but a hash chain failed verification"
        if any(f.startswith("CHECKPOINTED") for _, f in fates):
            return "CHECKPOINTED, NOT RESTORED", f"{n - len(resumed)} member(s) still only an image"
        return "RAN CONTINUOUSLY", f"no member was checkpointed; chains {'OK' if chains_ok else 'FAILED'}"
```

```
836
837     @staticmethod
838     def _read_state(entry: ManagedProcess) -> dict:
839         """Workload-declared progress, only if it was written by this very PID."""
840         if entry.state_file is None:
841             return {}
842         try:
843             data = json.loads(entry.state_file.read_text())
844         except (OSError, ValueError):
845             return {}
846         return data if data.get("pid") == entry.pid else {}
847
848     # -- one control step ---------------------------------------------------------
849     def step(self) -> None:
850         sample = self.sensor.sample()
851         self.samples += 1
852
853         for e in self.whitelist.refresh():
854             rc = e.returncode
855             self.exits[e.pid] = (self.elapsed(), rc)
856             if rc is not None and rc < 0:
857                 how = f"killed by {signal.Signals(-rc).name}"
858             elif rc is None:
859                 how = "exited (not our child; status unknown)"
860             else:
861                 how = f"exited rc={rc}"
862             st = self.last_state.get(e.pid, {})
863             if "iteration" in st:
864                 how += f" (last reported iteration {st['iteration']}, work {st.get('work_seconds', 0):.2f}s)"
865             self.event(f"{e.role} pid={e.pid} {how}", logging.WARNING if rc not in (0, None) else logging.INFO)
866
867         self._discover_children()
868         self._groups()  # logs new dependency edges with their proof
869
870         for e in self.whitelist.with_state(RUNNING):
871             snap = self.sensor.proc(e.pid)
872             if snap is not None:
873                 self.last_cpu[e.pid] = snap.cpu_seconds
874                 self.last_rss[e.pid] = snap.rss_bytes
875             st = self._read_state(e)
876             if st:
877                 self.last_state[e.pid] = st
878
879         if sample.cgroup is not None:
880             ook = sample.cgroup.events.get("oom_kill", 0)
881             if ook > self.last_oom_kill:
882                 self.event(f"kernel OOM killer fired in test cgroup (oom_kill {self.last_oom_kill} -> {ook})",
883                            logging.WARNING)
884                 self.last_oom_kill = ook
885                 if self.first_oom_at is None:
886                     self.first_oom_at = self.elapsed()
887
888         level = self.classifier.classify(sample)
889         if level is not self.level:
890             self.event(f"pressure {self.level.value} -> {level.value}: {format_sample(sample)}")
891             self.level = level
892         elif self.samples % self.log_every == 0:
893             log.info("[%s] %s", level.value, format_sample(sample))
894
895         if level is Level.ACTION:
896             self._maybe_capture()
897         self._maybe_restore(sample)
898         self._write_telemetry(sample)
899
900     def _maybe_capture(self) -> None:
901         now = time.monotonic()
902         if now - self.last_capture < self.cooldown_s:
903             return
904         verbose = now - self.last_decision_log >= self.decision_log_interval_s
905         if verbose:
906             self.last_decision_log = now
907
908         scores: list[Score] = []
909         self.group_of = {}
910         tree_check: dict[int, tuple[bool, str]] = {}
911         for g in self._groups():
912             if g.size == 1:  # a single process is a group of one: scored exactly as before
913                 e = self.whitelist.get(g.members[0])
914                 snap = self.sensor.proc(e.pid)
915                 if snap is None or not snap.alive:
916                     continue
917                 scores.append(self.cost.score(snap, e.role, self._read_state(e).get("work_seconds")))
918                 continue
919             members = [(self.sensor.proc(p), self.whitelist.get(p)) for p in g.members]
```

```
            if any(snap is None or not snap.alive for snap, _ in members):
                continue
            ok, root, why = deps.capturable(g)
            root = root if root is not None else g.members[0]
            self.group_of[root] = list(g.members)
            tree_check[root] = (ok, why)
            label = "group[" + "+".join(e.role for _, e in members) + "]"
            scores.append(self.cost.score_group(
                root, label, [(snap, self._read_state(e).get("work_seconds")) for snap, e in members]))
        ranked = self.cost.rank(scores)
        if verbose:
            for s in ranked:
                log.info("  M2 %s", s.describe())
                if s.pid in self.group_of:
                    log.info("     members %s; R, RSS and C summed over all members; CRIU tree check: %s",
                             self.group_of[s.pid], tree_check[s.pid][1])

        for s in ranked:
            decision = self.admission.admit_capture(s)
            refused_tree = s.pid in self.group_of and not tree_check[s.pid][0]
            if verbose or (decision.admitted and not refused_tree):
                log.info("  M3 %s pid=%d: %s", s.role, s.pid, decision)
            if not decision.admitted:
                if self.inject == "arena-too-small" and s.eligible and not self.fault_stood_down:
                    # The real M3 test denied an otherwise-admissible process.
                    self._stand_down(f"M3 capture admission DENY for {s.role} pid={s.pid}: {decision.reason}")
                continue
            if s.pid in self.group_of:
                ok, why = tree_check[s.pid]
                if not ok:
                    # CRIU would take a different set of processes than the group, or
                    # break a link to an outside process: refuse to capture any of it.
                    if why not in self.group_standdowns:
                        self.group_standdowns.add(why)
                        self.event(f"STAND DOWN [group]: {s.role} members {self.group_of[s.pid]} not "
                                   f"capturable as one CRIU tree: {why}; not capturing -- kernel left "
                                   "in charge", logging.WARNING)
                    continue
                log.info("  group %s members %s: ADMIT as one CRIU tree: %s", s.role, self.group_of[s.pid], why)
            self.last_capture = now
            if self.observe_only:
                self.event(f"{self.mode.upper()}: WOULD checkpoint {s.role} pid={s.pid} "
                           f"(R={s.restart_cost_s:.1f}s, C~{s.capture_cost_s:.2f}s, frees ~{s.mem_mib:.0f}MiB); "
                           "CRIU not invoked" + ("; kernel OOM killer left to act" if self.mode == "baseline" else "
"))
                return
            if self.inject == "governor-dies":
                # Abrupt crash after the ACTION threshold is crossed and a victim
                # is chosen, but before any capture. No cleanup runs: the cgroup
                # and workloads survive under the kernel, exactly as if PRESERVE
                # had never been here. run_failsafe.sh cleans up the remains.
                self.event(f"FAULT INJECTION [governor-dies]: ACTION crossed, {s.role} pid={s.pid} "
                           "chosen; simulating an abrupt governor crash before any capture "
                           "(os._exit, no shutdown)", logging.CRITICAL)
                sys.stdout.flush()
                sys.stderr.flush()
                os._exit(137)
            self.event(f"ACTION: checkpoint {s.role} pid={s.pid} (R={s.restart_cost_s:.1f}s, "
                       f"C~{s.capture_cost_s:.2f}s, frees ~{s.mem_mib:.0f}MiB)")
            self._capture(s)
            return
        if verbose:
            log.warning("ACTION level but no admissible candidate: kernel reclaim / OOM killer decides")

    def _capture(self, s: Score) -> None:
        entry = self.whitelist.get(s.pid)
        before = self._read_state(entry) if entry else {}

        if self.inject == "stale-pid" and entry is not None:
            # Corrupt the recorded (pid, starttime) so the real whitelist reuse
            # guard mismatches, as if the PID had been recycled. Restore it after
            # so shutdown can still manage the process.
            saved = entry.start_ticks
            entry.start_ticks = -1
            self.event("FAULT INJECTION [stale-pid]: recorded starttime corrupted so the "
                       "(pid, starttime) reuse check will mismatch", logging.WARNING)
            try:
                self.capture.dump(s.pid)  # verify() must refuse before CRIU runs
                self.event("stale-pid: dump was NOT refused -- this is a bug", logging.ERROR)
            except SafetyRefusal as exc:
                self._stand_down(f"whitelist PID-reuse guard refused the dump ({exc})")
            finally:
                entry.start_ticks = saved
            return
```

```
1003
1004        fault = self.inject if self.inject in ("dump-fail", "dump-timeout") else None
1005        if fault:
1006            self.fault_dump_attempted = True
1007        members = self.group_of.get(s.pid)
1008        if members:
1009            # Re-check right before CRIU runs: the tree must still be exactly the group.
1010            ok, _root, why = deps.capturable(deps.Group(members))
1011            if not ok:
1012                self.event(f"STAND DOWN [group]: tree changed before dump: {why}; not capturing",
1013                           logging.WARNING)
1014                return
1015        key = f"dump-{s.pid}"
1016        self.arena.reserve(key, s.est_image_bytes)
1017        try:
1018            res = self.capture.dump(s.pid, fault=fault, members=members)
1019        except SafetyRefusal as exc:
1020            self.event(f"capture refused: {exc}", logging.ERROR)
1021            return
1022        finally:
1023            self.arena.release(key)  # release accounting whatever happened
1024
1025        if fault and res.setup_failed:
1026            # The fault could not be established (e.g. tmpfs did not mount). Do NOT
1027            # run or report an unfaulted result -- declare the injection failed.
1028            self.fault_setup_failed = True
1029            self.injection_failed = True
1030            self.injection_reason = res.error
1031            self.event(f"INJECTION SETUP FAILED - fault did not occur: {res.error}", logging.CRITICAL)
1032            return
1033        if fault and res.ok:
1034            # A dump that was supposed to fail completed. This must never be
1035            # reported as a fault row; flag the injection as failed.
1036            self.injection_failed = True
1037            self.injection_reason = (f"{fault}: CRIU dump SUCCEEDED when it was required to fail "
1038                                     "(fault mechanism ineffective)")
1039            self.event(f"INJECTION FAILED - fault did not occur: {self.injection_reason}", logging.CRITICAL)
1040            # The process is now checkpointed; restore it so we leave nothing dead.
1041            self.captures += 1
1042            self.preserved_rss[s.pid] = int(s.mem_mib * MIB)
1043            self.event("attempting restore so the wrongly-dumped process is not left checkpointed",
1044                       logging.WARNING)
1045            try:
1046                self.capture.restore(s.pid)
1047            except (SafetyRefusal, CgroupError) as exc:
1048                self.event(f"could not restore wrongly-dumped process: {exc}", logging.ERROR)
1049            return
1050
1051        if res.ok:
1052            self.captures += 1
1053            self.preserved_rss[s.pid] = int(s.mem_mib * MIB)
1054            self.cost.observe_capture(res.image_bytes, res.seconds)
1055            self._save_throughput()
1056            it = before.get("iteration")
1057            self.event(f"PRESERVED {res.describe()}" + (f"; last reported iteration {it}" if it else ""))
1058            if members:
1059                self.preserved_groups[s.pid] = list(members)
1060                for m in members:
1061                    st = self._read_state(self.whitelist.get(m))
1062                    self.event(f"  group member {self.whitelist.get(m).role} pid={m} checkpointed in the same "
1063                               f"image; last reported iteration {st.get('iteration')}, "
1064                               f"work {st.get('work_seconds')}s")
1065            # Record in the image itself that a live process now exists only as this
1066            # image, so --reap-arena can never delete it while it is not restored.
1067            write_image_state(res.image_dir, "checkpointed", pid=s.pid, role=s.role,
1068                              scenario=self.scenario, governor_pid=os.getpid(),
1069                              last_iteration=it, work_seconds=before.get("work_seconds"),
1070                              **({"members": members} if members else {}))
1071            if self.inject == "restore-blocked":
1072                # Arm a permanent restore DENY through the real admission headroom
1073                # test: claim the process needs more memory than can ever fit.
1074                self.preserved_rss[s.pid] = 1 << 62
1075                self.event("FAULT INJECTION [restore-blocked]: restore admission will DENY for "
1076                           "the whole run; process stays checkpointed on disk", logging.WARNING)
1077        else:
1078            self.event(res.describe(), logging.ERROR)
1079            if res.log_tail:
1080                log.error("dump.log tail:\n%s", res.log_tail)
1081            if fault:
1082                self.fault_dump_failed = True
1083                self.fault_dump_enospc = res.enospc
1084            if self.inject == "dump-fail":
1085                if res.enospc:
1086                    self.event("dump-fail: confirmed CRIU failed with ENOSPC (tmpfs full) as intended")
```

```
                else:
                    # It failed, but not by the mechanism we injected: do not count it.
                    self.injection_failed = True
                    self.injection_reason = (f"dump-fail: dump failed but NOT with ENOSPC "
                                             f"(error: {res.error})")
                    self.event(f"INJECTION FAILED - fault did not occur: {self.injection_reason}",
                               logging.CRITICAL)
            self._after_failed_dump(s.pid, res)

    def _after_failed_dump(self, pid: int, res) -> None:
        """Clean up a failed/timed-out dump and report, honestly, if the target survived."""
        if res.image_dir is not None and self.arena.contains(res.image_dir) and res.image_dir.exists():
            try:
                os.chmod(res.image_dir, 0o700)  # dump-fail injection left it read-only
            except OSError:
                pass
            shutil.rmtree(res.image_dir, ignore_errors=True)
            self.event(f"cleaned up partial image set {res.image_dir.name}; arena accounting released")
        # Did the target survive a failed dump, or was it lost? (honest, no recovery logic)
        snap = self.sensor.proc(pid)
        entry = self.whitelist.get(pid)
        alive = (snap is not None and snap.alive and entry is not None
                 and snap.start_ticks == entry.start_ticks)
        if alive:
            self.event(f"after failed dump: target {entry.role} pid={pid} SURVIVED, still running "
                       "and unfrozen; no work lost")
        else:
            self.event(f"after failed dump: target pid={pid} is GONE -- a failed dump LOST the "
                       "process (real limitation, reported as-is; no recovery logic added)", logging.ERROR)
        if self.inject in ("dump-fail", "dump-timeout"):
            self.fault_stood_down = True
            self.event(f"STAND DOWN [fault={self.inject}]: dump did not complete; no process left "
                       "frozen or half-dumped, kernel left in charge (fail-safe)", logging.WARNING)

    def _maybe_restore(self, sample: Sample) -> None:
        # Non-root group members come back with their root's tree; never restore them alone.
        followers = {m for root, ms in self.preserved_groups.items() for m in ms if m != root}
        pending = [e for e in self.whitelist.with_state(PRESERVED)
                   if self.restore_attempts.get(e.pid, 0) < self.restore_max_attempts
                   and e.pid not in followers]
        if not pending:
            return
        entry = pending[0]  # FIFO: oldest checkpoint first
        decision = self.admission.admit_restore(self.preserved_rss.get(entry.pid, 0), sample.cgroup,
                                                self.classifier.recovered)
        if not decision.admitted:
            now = time.monotonic()
            if self.classifier.recovered and now - self.last_decision_log >= self.decision_log_interval_s:
                self.last_decision_log = now
                log.info("  M3 restore %s pid=%d deferred: %s", entry.role, entry.pid, decision)
                if self.inject == "restore-blocked" and not self.fault_stood_down:
                    self._stand_down(f"M3 restore admission DENY for {entry.role} pid={entry.pid}: "
                                     f"{decision.reason}; process remains safely checkpointed")
            return

        attempt = self.restore_attempts.get(entry.pid, 0) + 1
        self.restore_attempts[entry.pid] = attempt
        self.event(f"RECOVERY: restore {entry.role} pid={entry.pid} attempt {attempt} ({decision.reason})")
        members = self.preserved_groups.get(entry.pid)
        try:
            res = self.capture.restore(entry.pid, members=members)
        except (SafetyRefusal, CgroupError) as exc:
            self.event(f"restore aborted: {exc}", logging.ERROR)
            return
        if res.ok:
            self.restores += 1
            self.restored_pids.add(entry.pid)
            self.classifier.reset_recovery()
            self.event(f"RESTORED {res.describe()}")
            for m in members or []:
                snap = self.sensor.proc(m)
                me = self.whitelist.get(m)
                alive = snap is not None and snap.alive and me.state == RUNNING \
                    and snap.start_ticks == me.start_ticks and self.cgroup.contains(m)
                if alive:
                    self.restored_pids.add(m)
                self.event(f"  group member {me.role} pid={m} after restore: "
                           + (f"ALIVE, ppid={snap.ppid}, in {self.cgroup.rel}" if alive else "NOT ALIVE"),
                           logging.INFO if alive else logging.ERROR)
            # If the image is kept after restore, it no longer holds the only copy.
            if res.image_dir is not None and res.image_dir.is_dir():
                write_image_state(res.image_dir, "restored", pid=entry.pid, role=entry.role,
                                  scenario=self.scenario, governor_pid=os.getpid())
        else:
```

```
            self.event(res.describe(), logging.ERROR)
            if res.log_tail:
                log.error("restore.log tail:\n%s", res.log_tail)
            if attempt >= self.restore_max_attempts:
                self.event(f"giving up on {entry.role} pid={entry.pid}; image kept at {entry.image_dir}",
                           logging.ERROR)

    def _write_telemetry(self, s: Sample) -> None:
        if self._telemetry is None:
            return
        cg = s.cgroup
        self._telemetry.writerow([
            f"{s.wall:.3f}", f"{self.elapsed():.2f}", self.level.value,
            s.system_psi.get("some", "avg10"),
            cg.pressure.get("some", "avg10") if cg else "",
            cg.pressure.get("full", "avg10") if cg else "",
            f"{cg.current / MIB:.1f}" if cg else "",
            f"{cg.max / MIB:.1f}" if cg and cg.max else "",
            f"{cg.swap_current / MIB:.1f}" if cg else "",
            f"{cg.usage_frac:.3f}" if cg else "",
            cg.events.get("oom_kill", 0) if cg else "",
            len(self.whitelist.with_state(RUNNING)),
            len(self.whitelist.with_state(PRESERVED)),
            " | ".join(self._pending_event),
        ])
        self._telemetry_file.flush()
        self._pending_event.clear()

    # -- scenario ------------------------------------------------------------
    def run_demo(self, with_hog: bool) -> None:
        w, g = "workloads", "governor"
        hog_delay = self.cfg.get_float(w, "hog_start_delay_s")
        duration = self.cfg.get_float(g, "demo_duration_s")
        post_restore = self.cfg.get_float(g, "demo_post_restore_s")
        post_oom = self.cfg.get_float("baseline", "post_oom_observe_s")
        if self.workload == "pipeline":
            self.launch_pipeline()
        else:
            self.launch_compute()
        hog_started = not with_hog
        restored_at: float | None = None
        while not self._stop:
            t0 = time.monotonic()
            if not hog_started and self.elapsed() >= hog_delay:
                self.launch_hog()
                hog_started = True
            self.step()
            if restored_at is None and self.restores:
                restored_at = self.elapsed()
            if restored_at is not None and self.elapsed() - restored_at >= post_restore:
                self.event(f"demo complete: watched restored job for {post_restore:.0f}s")
                break
            if self.mode == "baseline" and self.first_oom_at is not None \
                    and self.elapsed() - self.first_oom_at >= post_oom:
                self.event(f"baseline complete: watched {post_oom:.0f}s past the first OOM kill")
                break
            if self.elapsed() >= duration:
                self.event(f"demo_duration_s ({duration:.0f}s) reached", logging.WARNING)
                break
            if hog_started and not self.whitelist.with_state(RUNNING) and not self.whitelist.with_state(PRESERVED):
                self.event("no managed processes left", logging.WARNING)
                break
            time.sleep(max(0.0, self.interval - (time.monotonic() - t0)))

    # -- overhead measurement (mode "overhead") ------------------------------
    def run_overhead(self) -> None:
        """Measure what the governor costs while doing nothing, which is the normal case.

        The workload runs with **no antagonist**, so usage stays well under
        ``usage_advisory_frac`` and PSI stays at zero: the loop samples and
        evaluates but never captures. Needs no CRIU and therefore no root.
        """
        dur = self.overhead_duration_s
        self.launch_compute()
        me = os.getpid()
        snap0 = self.sensor.proc(me)
        ru0 = resource.getrusage(resource.RUSAGE_SELF)
        t0 = time.monotonic()
        # tree_bytes() walks with os.walk, which yields NOTHING for a directory this
        # user cannot read (the arena is mode 0700 and may be root-owned from an
        # earlier privileged run). Without this check a measured 0 would be an
        # artifact, not a fact, so report it as unknown instead.
        arena_readable = os.access(self.arena.path, os.R_OK | os.X_OK)
        if not arena_readable:
```

```
            log.warning("arena %s is not readable by uid %d: its on-disk usage will be reported as "
                        "unknown rather than 0", self.arena.path, os.geteuid())
        peak_arena = self.arena.used_bytes() if arena_readable else None
        log.info("measuring governor overhead for %.0fs (workload running, no antagonist)", dur)

        while not self._stop and time.monotonic() - t0 < dur:
            it0 = time.monotonic()
            self.step()  # one sample-and-evaluate iteration
            self.step_times.append(time.monotonic() - it0)
            self.level_counts[self.level.value] = self.level_counts.get(self.level.value, 0) + 1
            if arena_readable:
                peak_arena = max(peak_arena, self.arena.used_bytes())
            time.sleep(max(0.0, self.interval - (time.monotonic() - it0)))

        elapsed = time.monotonic() - t0
        ru1 = resource.getrusage(resource.RUSAGE_SELF)
        snap1 = self.sensor.proc(me)
        cpu = (ru1.ru_utime + ru1.ru_stime) - (ru0.ru_utime + ru0.ru_stime)
        times = sorted(self.step_times)
        n = len(times)
        rss0 = snap0.rss_bytes if snap0 else 0
        rss1 = snap1.rss_bytes if snap1 else 0
        self.overhead = {
            "mode": "overhead", "scenario": self.scenario,
            "duration_s": round(elapsed, 2),
            "sample_interval_s": self.interval,
            "sampling_iterations": n,
            "iter_mean_ms": round(1000 * sum(times) / n, 3) if n else None,
            "iter_p95_ms": round(1000 * times[min(n - 1, int(0.95 * (n - 1)))], 3) if n else None,
            "iter_max_ms": round(1000 * times[-1], 3) if n else None,
            "governor_cpu_s": round(cpu, 3),
            "governor_cpu_pct_one_core": round(100 * cpu / elapsed, 3) if elapsed else None,
            "rss_start_mib": round(rss0 / MIB, 2),
            "rss_end_mib": round(rss1 / MIB, 2),
            "rss_delta_mib": round((rss1 - rss0) / MIB, 2),
            "captures": self.captures, "restores": self.restores,
            "level_counts": dict(self.level_counts),
            "cgroup_memory_max_mib": round((self.cfg.get_size("cgroup", "memory_max") or 0) / MIB, 0),
            # The arena is a DIRECTORY on ext4 with a byte budget -- not pinned memory.
            # Arena only does mkdir / stat / statvfs plus an in-memory dict of ints.
            "arena_capacity_mib": round(self.arena.capacity_bytes / MIB, 0),
            "arena_peak_used_mib": None if peak_arena is None else round(peak_arena / MIB, 2),
            "arena_readable_by_governor": arena_readable,
            "images_written_this_run": self.captures,  # 0 captures => the arena was not written to
            "arena_ram_pinned_mib": 0.0,
            "arena_kind": "disk budget (ext4 directory), not pinned RAM",
            "fs_reserve_mib": round(self.arena.fs_reserve_bytes / MIB, 0),
            "compute_args": list(self.compute_args),
        }

    def _overhead_summary(self) -> int:
        """Log the overhead report and write it as JSON for the A/B harness."""
        o = self.overhead
        if not o:
            log.error("overhead measurement produced no data")
            return 1
        # compute_job's final state is written on SIGTERM during shutdown.
        try:
            o["compute_iterations"] = json.loads((self.run_dir / COMPUTE_STATE).read_text()).get("iteration")
        except (OSError, ValueError):
            o["compute_iterations"] = None
        reached = [lvl for lvl in ("advisory", "action") if o["level_counts"].get(lvl)]
        o["stayed_below_advisory"] = not reached
        out = self.run_dir / OVERHEAD_RESULT
        out.write_text(json.dumps(o, indent=2))
        give_to_invoking_user(out)

        log.info("GOVERNOR OVERHEAD (idle: workload running, no antagonist, no capture)")
        log.info("  window                    : %.2fs at %.2fs sample interval", o["duration_s"],
                 o["sample_interval_s"])
        log.info("  sampling iterations       : %d", o["sampling_iterations"])
        log.info("  per-iteration time        : mean %.3f ms, p95 %.3f ms, max %.3f ms",
                 o["iter_mean_ms"], o["iter_p95_ms"], o["iter_max_ms"])
        log.info("  governor CPU time         : %.3fs = %.3f%% of one core",
                 o["governor_cpu_s"], o["governor_cpu_pct_one_core"])
        log.info("  governor RSS              : %.2f MiB -> %.2f MiB (delta %+.2f MiB)",
                 o["rss_start_mib"], o["rss_end_mib"], o["rss_delta_mib"])
        log.info("  captures / restores       : %d / %d", o["captures"], o["restores"])
        log.info("  pressure levels seen      : %s%s", o["level_counts"],
                 "" if o["stayed_below_advisory"] else "  <-- reached advisory/action: NOT an idle measurement")
        log.info("  arena                     : %s", o["arena_kind"])
        peak = ("unknown (not readable by this uid)" if o["arena_peak_used_mib"] is None
                else f"{o['arena_peak_used_mib']:.2f} MiB")
        log.info("    budget %.0f MiB, peak on-disk use %s, images written this run %d, RAM pinned %.1f MiB",
```

```
                o["arena_capacity_mib"], peak, o["images_written_this_run"], o["arena_ram_pinned_mib"])
        log.info("    free-space guard (fs_reserve_mib) %.0f MiB; protected cgroup %.0f MiB",
                 o["fs_reserve_mib"], o["cgroup_memory_max_mib"])
        log.info("  compute_job iterations    : %s (this arm: governor running)", o["compute_iterations"])
        log.info("overhead result written to %s", out)
        if o["captures"]:
            log.error("a capture happened: this is not an idle-overhead measurement")
            return 1
        return 0 if o["stayed_below_advisory"] else 1

    # -- teardown ------------------------------------------------------------
    def _wait_exit(self, timeout: float) -> None:
        deadline = time.monotonic() + timeout
        while self.whitelist.with_state(RUNNING) and time.monotonic() < deadline:
            self.whitelist.refresh()
            time.sleep(self.interval)
        self.whitelist.refresh()

    def shutdown(self) -> None:
        # memory.events disappears with the cgroup: keep its final counters.
        if self.cgroup.exists():
            self.final_events = dict(read_cgroup_memory(self.cgroup.path).events)
        for sig in (signal.SIGTERM, signal.SIGKILL):
            for e in self.whitelist.with_state(RUNNING):
                try:
                    self.whitelist.signal(e.pid, sig)
                except (SafetyRefusal, ProcessLookupError) as exc:
                    log.warning("could not signal %s pid=%d: %s", e.role, e.pid, exc)
            self._wait_exit(self.grace_s)
        for e in self.whitelist.with_state(PRESERVED):
            # Verify the image is really on disk: "work recoverable" is only true
            # if the image set still exists. The governor runs as root, so it can
            # read the arena (mode 0700) that an unprivileged script cannot.
            present = e.image_dir is not None and e.image_dir.is_dir()
            detail = (f"image present, {tree_bytes(e.image_dir) / MIB:.1f}MiB" if present
                      else "IMAGE MISSING - work is NOT recoverable")
            self.event(f"{e.role} pid={e.pid} is still checkpointed at {e.image_dir} ({detail})",
                       logging.WARNING)
        if self._telemetry_file is not None:
            self._telemetry_file.close()
            self._telemetry, self._telemetry_file = None, None
        if self.cfg.get_bool("cgroup", "remove_on_exit"):
            try:
                self.cgroup.destroy()
            except (CgroupError, OSError) as exc:
                log.warning("cgroup not removed: %s", exc)

    def compute_fate(self) -> tuple[str, str]:
        """What actually happened to compute_job -- from events, not only log gaps.

        RESUMED                    checkpointed and restored (and its counter never went back)
        RAN CONTINUOUSLY           never checkpointed, alive when the run ended
        KILLED                     died during the run; its work is lost
        CHECKPOINTED, NOT RESTORED still exists only as an image when the run ended
        RESTARTED                  the counter went backwards: state was not preserved
        NO-DATA                    no compute_job was launched
        A process that died is KILLED even if its progress log has no gap: the old
        log-only check called a SIGKILLed job "RESUMED -- largest gap 0.7s".
        """
        entries = self.whitelist.by_role("compute")
        if not entries:
            return "NO-DATA", "no compute_job was launched"
        e = entries[0]
        st = self.last_state.get(e.pid, {})
        rows = progress_rows(self.run_dir / COMPUTE_PROGRESS)
        backwards = [(a[1], b[1]) for a, b in zip(rows, rows[1:]) if b[1] <= a[1]]
        last_it = st.get("iteration", rows[-1][1] if rows else None)

        if e.pid in self.exits:  # died during the run (shutdown's own SIGTERM is not recorded here)
            t, rc = self.exits[e.pid]
            if rc is None:  # a restored process is not our child: its exit status is unknowable
                how = "died (exit status unknown: not the governor's child after restore)"
            elif rc < 0:
                how = f"killed by {signal.Signals(-rc).name}"
            else:
                how = f"exited rc={rc}"
            return "KILLED", (f"{how} at {t:.1f}s; final iteration {st.get('iteration')}, "
                              f"{self._work_lost(e.pid):.2f}s of work lost")
        if e.state == PRESERVED:
            return "CHECKPOINTED, NOT RESTORED", (f"exists only as the image at {e.image_dir}; "
                                                  f"last iteration before checkpoint {last_it}")
        if backwards:
            return "RESTARTED", (f"iteration counter went backwards {backwards[:3]}: "
                                 "the job started again from scratch")
```

```
        if e.pid in self.restored_pids:
            if len(rows) >= 2:
                gap, i = largest_gap(rows)
                return "RESUMED", (f"checkpointed and restored; largest gap {gap:.1f}s between "
                                   f"iteration {rows[i][1]} and {rows[i + 1][1]}; counter never went "
                                   f"backwards (last iteration {rows[-1][1]})")
            return "RESUMED", "checkpointed and restored (too few progress lines to measure the gap)"
        return "RAN CONTINUOUSLY", f"never checkpointed, alive at the end of the run; last iteration {last_it}"

    def summary(self) -> int:
        verdict, detail = self.compute_fate()
        log.info("=" * 72)
        log.info("SUMMARY  scenario=%s  mode=%s  captures=%d  restores=%d  samples=%d",
                 self.scenario, self.mode, self.captures, self.restores, self.samples)
        for line in self.events:
            log.info("  %s", line)
        if self.workload == "pipeline":
            # Objective 6: the verdict is the GROUP's, from per-member events and hash chains.
            verdict, detail = self.group_fate()
            log.info("pipeline group continuity: %s -- %s", verdict, detail)
        else:
            log.info("compute_job continuity: %s -- %s", verdict, detail)
        log.info("=" * 72)
        if self.mode == "baseline":
            return self.baseline_report()
        if self.mode == "overhead":
            return self._overhead_summary()
        if self.observe_only:
            return 0
        if self.inject:
            return self._fault_summary()
        if self.captures == 0:
            return 2  # pressure never led to a capture: tune preserve.conf
        if self.restores == 0:
            return 3
        return 0 if verdict == "RESUMED" else 1

    # -- fault self-verification ---------------------------------------------------
    def _fault_summary(self) -> int:
        """Assert the injected fault's precondition actually occurred.

        Returns 0 only when the fault fired and produced its fail-safe outcome.
        Returns 4 (INJECTION FAILED) when the fault did not occur -- so a fault
        that silently does not fire can never be reported as a passing row.
        """
        RC_INJECTION_FAILED = 4
        preserved = bool(self.whitelist.with_state(PRESERVED))
        if self.injection_failed:
            log.critical("INJECTION FAILED - fault did not occur [%s]: %s",
                         self.inject, self.injection_reason or "precondition not satisfied")
            return RC_INJECTION_FAILED

        # Per-fault assertion: (condition that must hold, human description).
        if self.inject in ("arena-too-small", "stale-pid"):
            ok = self.fault_stood_down and self.captures == 0
            why = "stood down with no capture"
            state = "stood down, kernel left in charge"
        elif self.inject in ("dump-fail", "dump-timeout"):
            ok = self.fault_dump_attempted and self.fault_dump_failed and self.captures == 0
            if self.inject == "dump-fail":
                ok = ok and self.fault_dump_enospc
            why = "a real dump was attempted, failed, and nothing was checkpointed"
            state = "dump attempted and failed; stood down"
        elif self.inject == "restore-blocked":
            ok = self.captures == 1 and self.restores == 0 and preserved
            why = "process was dumped and then permanently denied restore"
            state = "checkpointed, not restored"
        else:
            ok, why, state = False, "unknown fault", "unknown"

        if not ok:
            self.injection_reason = f"expected that {why}; observed captures={self.captures} " \
                f"restores={self.restores} stood_down={self.fault_stood_down} " \
                f"dump_failed={self.fault_dump_failed} enospc={self.fault_dump_enospc}"
            log.critical("INJECTION FAILED - fault did not occur [%s]: %s", self.inject, self.injection_reason)
            return RC_INJECTION_FAILED

        log.info("FAULT RESULT  inject=%s  VERIFIED (%s)  captures=%d  restores=%d  state: %s",
                 self.inject, why, self.captures, self.restores, state)
        return 0

    # -- baseline comparison -------------------------------------------------------
    def _work_lost(self, pid: int) -> float:
        """Work destroyed by killing ``pid``: the cost model's R at its last observation."""
```

```
        return max(self.last_cpu.get(pid, 0.0), float(self.last_state.get(pid, {}).get("work_seconds", 0.0)))

    def baseline_report(self) -> int:
        """Who did the kernel kill, what was lost, and how that compares with PRESERVE."""
        oom_kill = self.final_events.get("oom_kill", self.last_oom_kill)
        kernel = read_kernel_oom_kills(self.start_uptime - 1.0)
        sigkilled = {pid: t for pid, (t, rc) in self.exits.items() if rc == -signal.SIGKILL}
        tolerance_s = 2 * self.interval + 1.0
        victims: dict[int, dict] = {}
        unmatched: list[dict] = []
        for k in kernel or []:
            if k.oom_memcg not in (None, self.cgroup.rel):
                continue  # an OOM in some other cgroup, not our experiment
            record = {"kernel_pid": k.pid, "anon_rss_mib": round(k.anon_rss_kib / 1024, 1), "kernel_line": k.line}
            if k.pid in sigkilled and k.pid not in victims:
                victims[k.pid] = {"pid": k.pid, "source": "kernel log (same PID)", **record}
                continue
            # The kernel logs PIDs of the *initial* PID namespace. Under WSL2 (and in
            # containers) the governor lives in a nested PID namespace and sees other
            # numbers, so match the record to the managed process that died by
            # SIGKILL at the same moment instead.
            k_elapsed = k.uptime - self.start_uptime
            # Admitted children (Objective 6) are not our children, so their exit status is
            # unknown (rc None); they are candidates too. In A/B every process is our child.
            cands = sorted((abs(t - k_elapsed), pid) for pid, t in
                           list(sigkilled.items()) + [(p, t) for p, (t, rc) in self.exits.items() if rc is None]
                           if pid not in victims)
            if cands and cands[0][0] <= tolerance_s:
                delta, pid = cands[0]
                victims[pid] = {"pid": pid, **record,
                                "source": f"kernel log pid {k.pid} (initial PID namespace) = pid {pid} here; "
                                          f"matched by time of death (delta {delta:.2f}s)"}
            else:
                unmatched.append(record)
        if kernel is None or len(victims) < oom_kill:
            for pid in sigkilled:
                # Fallback when dmesg is unreadable: SIGKILL we did not send, while oom_kill > 0.
                if pid not in victims and oom_kill > 0:
                    victims[pid] = {"pid": pid, "source": "SIGKILL exit + memory.events oom_kill"}
            # Admitted children (Objective 6) are not our children: exit status unknown. A
            # member that left no partner_lost note of its own died without exiting itself.
            for pid, (t, rc) in self.exits.items():
                e = self.whitelist.get(pid)
                if rc is None and pid not in victims and oom_kill > len(victims) and e is not None \
                        and e.parent is not None and not self._read_state(e).get("partner_lost"):
                    victims[pid] = {"pid": pid, "source": "died without exiting itself (no partner_lost "
                                                          "note) + memory.events oom_kill"}
        for v in victims.values():
            entry = self.whitelist.get(v["pid"])
            v["role"] = entry.role if entry else "?"
            v["last_observed_rss_mib"] = round(self.last_rss.get(v["pid"], 0) / MIB, 1)
        for v in victims.values():
            st = self.last_state.get(v["pid"], {})
            v.update(killed_at_s=round(self.exits.get(v["pid"], (None, None))[0] or 0, 1),
                     last_iteration=st.get("iteration"), work_lost_s=round(self._work_lost(v["pid"]), 2),
                     cpu_s=round(self.last_cpu.get(v["pid"], 0.0), 2))

        computes = self.whitelist.by_role("compute")
        compute = computes[0] if computes else None
        compute_killed = compute is not None and compute.pid in victims
        rows = progress_rows(self.run_dir / COMPUTE_PROGRESS)
        result = {
            "mode": "baseline",
            "scenario": self.scenario,
            "memory_swap_max": self.swap_max_used,
            "memory_events": self.final_events,
            "oom_kill": oom_kill,
            "kernel_log_readable": kernel is not None,
            "victims": list(victims.values()),
            "unmatched_kernel_records": unmatched,
            "compute_pid": compute.pid if compute else None,
            "compute_killed": compute_killed,
            "compute_last_iteration": (self.last_state.get(compute.pid, {}).get("iteration")
                                       if compute_killed else (rows[-1][1] if rows else None)),
            "compute_work_lost_s": round(self._work_lost(compute.pid), 2) if compute_killed else 0.0,
            "compute_final_work_s": rows[-1][2] if rows else None,
        }
        group_lines: list[str] = []
        if self.workload == "pipeline":
            # Objective 6: how many members died, how, and the total work destroyed.
            members, dead, lost = [], 0, 0.0
            for role in PIPELINE_ROLES.values():
                for e in self.whitelist.by_role(role):
                    died = e.pid in self.exits
```

```
                    st = self._read_state(e)  # final state written by the member itself, if any
                    work = max(self._work_lost(e.pid), float(st.get("work_seconds", 0.0)))
                    if e.pid in victims:
                        cause = "OOM-killed by the kernel (" + victims[e.pid]["source"] + ")"
                    elif st.get("partner_lost"):
                        cause = f"exited because its partner died (own final state: {st['partner_lost']})"
                    elif died:
                        rc = self.exits[e.pid][1]
                        cause = f"died, rc={rc}" if rc is not None else "died (exit status unknown)"
                    else:
                        cause = "survived"
                    if died:
                        dead += 1
                        lost += work
                    members.append({"pid": e.pid, "role": role, "died": died, "cause": cause,
                                    "last_iteration": st.get("iteration"),
                                    "work_lost_s": round(work, 2) if died else 0.0})
                    group_lines.append(f"    {role} pid={e.pid}: {cause}; last iteration {st.get('iteration')}, "
                                       f"work {'destroyed' if died else 'kept'} {work:.2f}s")
            result["group"] = {"members": members, "members_died": dead, "members_total": len(members),
                               "work_destroyed_s": round(lost, 2)}
            group_lines.insert(0, f"  pipeline group            : {dead} of {len(members)} members died, "
                                  f"{lost:.2f}s of work destroyed in total")
        out = self.run_dir / BASELINE_RESULT
        out.write_text(json.dumps(result, indent=2))
        give_to_invoking_user(out)

        log.info("BASELINE RESULT (kernel OOM killer, no PRESERVE intervention)")
        log.info("  memory.swap.max used      : %s", "max" if self.swap_max_used is None else self.swap_max_used)
        log.info("  memory.events             : %s", self.final_events or "(cgroup already gone)")
        log.info("  OOM killer fired?         : %s (oom_kill=%d)", "YES" if oom_kill else "NO", oom_kill)
        if kernel is None:
            log.warning("  kernel log unreadable (dmesg); victim attribution uses exit status only")
        for v in victims.values():
            log.info("  victim                    : %s pid=%d", v["role"], v["pid"])
            log.info("    identified by: %s", v["source"])
            if "kernel_line" in v:
                log.info("    kernel: %s", v["kernel_line"])
                log.info("    RSS cross-check: kernel anon-rss %.1fMiB vs last observed RSS %.1fMiB",
                         v["anon_rss_mib"], v["last_observed_rss_mib"])
            log.info("    killed at %.1fs, last iteration %s, CPU %.2fs, work destroyed %.2fs",
                     v["killed_at_s"], v["last_iteration"], v["cpu_s"], v["work_lost_s"])
        for u in unmatched:
            log.warning("  kernel OOM record not matched to any managed process: %s", u["kernel_line"])
        if oom_kill and not victims:
            log.warning("  oom_kill > 0 but the victim could not be identified")
        if compute is not None:
            if compute_killed:
                log.info("  compute_job               : KILLED at iteration %s -> %.2fs of work destroyed",
                         result["compute_last_iteration"], result["compute_work_lost_s"])
            else:
                log.info("  compute_job               : survived (reached iteration %s, %.2fs work)",
                         result["compute_last_iteration"], result["compute_final_work_s"] or 0.0)
        for line in group_lines:
            log.info(line)

        log.info("-" * 72)
        ref_setting = self.cfg.get_str("baseline", "reference_live_run")
        if not ref_setting:
            log.info("no reference live run configured for scenario %s; after the live run compare with:\n"
                     "  python3 src/compare.py --live evidence/<live dir> --baseline evidence/<this baseline dir>",
                     self.scenario)
        else:
            ref = load_reference_run(self.cfg.get_path("baseline", "reference_live_run"))
            if not ref.get("available"):
                log.warning("reference live run %s not found; no side-by-side comparison", ref["path"])
            else:
                log.info("SIDE BY SIDE  (PRESERVE reference: %s)", ref["path"])
                for name, a, b in side_by_side_table(ref, result):
                    log.info("  %-30s | %-46s | %s", name, a, b)
        log.info("-" * 72)
        log.info("baseline result written to %s", out)
        return 0 if oom_kill else 2  # 2: the kernel never had to kill anything


def main() -> int:
    ap = argparse.ArgumentParser(description="PRESERVE memory-pressure governor.")
    ap.add_argument("--config", default=None, help="path to preserve.conf")
    g = ap.add_mutually_exclusive_group()
    g.add_argument("--dry-run", action="store_true", help="score and admit, but never invoke CRIU")
    g.add_argument("--baseline", action="store_true",
                   help="control experiment: observe only, swap from baseline_memory_swap_max, kernel OOM acts")
    g.add_argument("--measure-overhead", action="store_true",
                   help="measure the governor's idle cost: workload only, no antagonist, no capture "
```

```
                        "(needs no CRIU, so no root)")
    g.add_argument("--arena-status", action="store_true",
                   help="print arena occupancy, capacity, and each image with size and age, then exit")
    g.add_argument("--reap-arena", action="store_true",
                   help="delete every image except those still checkpointed and never restored, then exit "
                        "(takes the governor lock; refuses while a governor runs)")
    ap.add_argument("--overhead-duration", type=float, default=None,
                    help="seconds to measure with --measure-overhead (default: [overhead] duration_s)")
    ap.add_argument("--no-hog", action="store_true", help="do not launch the memory hog (control run)")
    ap.add_argument("--scenario", default=None,
                    help="named scenario from preserve.conf [scenario.<name>] (default: [scenarios] default)")
    ap.add_argument("--reset-throughput", action="store_true",
                    help="discard the saved dump-throughput estimate and start from initial_dump_mib_per_s")
    ap.add_argument("--inject", default=None, choices=FAULTS,
                    help="live mode only: inject a fault to demonstrate the fail-safe stand-down path")
    args = ap.parse_args()

    cfg = Config(args.config, args.scenario)
    setup_logging(cfg)
    if args.arena_status:
        return arena_status(cfg)
    if args.reap_arena:
        return reap_arena(cfg)
    if args.baseline:
        mode = "baseline"
    elif args.measure_overhead:
        mode = "overhead"
    elif args.dry_run or cfg.get_bool("governor", "dry_run"):
        mode = "dry-run"
    else:
        mode = "live"
    if args.inject and mode != "live":
        ap.error(f"--inject {args.inject} requires live mode (not --baseline/--dry-run)")
    log.info("=" * 72)
    log.info("PRESERVE governor start: pid=%d euid=%d invoking uid=%d scenario=%s mode=%s config=%s",
             os.getpid(), os.geteuid(), invoking_ids()[0], cfg.scenario, mode, cfg.path)
    log.info("scenario %s: %s", cfg.scenario, cfg.scenario_description)
    for (section, key), value in sorted(cfg.overrides.items()):
        log.info("  scenario override [%s] %s = %s", section, key, value)

    # One governor at a time. Taken BEFORE anything is created or cleared: the
    # cgroup, the run/ files and the arena are only touched while holding it.
    lock = RunLock(cfg.get_path("paths", "lock_file"))
    try:
        lock.acquire(holder_info(mode, cfg.scenario, sys.argv))
    except LockHeld as exc:
        _refuse(str(exc))
        return EXIT_LOCKED
    except OSError as exc:
        _refuse(f"cannot open lock file {lock.path} ({exc}); refusing to run without the lock")
        return EXIT_LOCKED
    log.info("lock acquired: %s held by pid %d", lock.path, os.getpid())

    if args.inject:
        log.warning("FAULT INJECTION ARMED: --inject %s (live mode; demonstrates the fail-safe path)",
                    args.inject)
    gov = Governor(cfg, mode, reset_throughput=args.reset_throughput, inject=args.inject)
    if args.overhead_duration is not None:
        gov.overhead_duration_s = args.overhead_duration
    signal.signal(signal.SIGINT, gov.request_stop)
    signal.signal(signal.SIGTERM, gov.request_stop)
    setup_ok = False
    try:
        gov.setup()
        setup_ok = True
        lock.update(cgroup=str(gov.cgroup.path))  # lets a later run find our leftovers
        if mode == "overhead":
            gov.run_overhead()
        else:
            gov.run_demo(with_hog=not args.no_hog)
    except (CgroupError, SafetyRefusal, OSError) as exc:
        log.critical("aborting: %s", exc)
        gov.event(f"aborted: {exc}", logging.CRITICAL)
    finally:
        gov.shutdown()
    if not setup_ok:
        # Setup never completed, so no experiment ran. Do NOT fall through to
        # summary(): its exit codes describe experiment outcomes ("no OOM kill",
        # "no capture") and would misreport a refusal as a result.
        _refuse(f"setup did not complete; no experiment ran (exit {EXIT_SETUP_FAILED})")
        lock.release()
        return EXIT_SETUP_FAILED
    rc = gov.summary()
    lock.release()
```

```
    return rc


def _refuse(message: str) -> None:
    """A loud, unmissable refusal in both the console and evidence/governor.log."""
    bar = "#" * 72
    log.critical(bar)
    log.critical("REFUSING TO RUN: %s", message)
    log.critical(bar)


if __name__ == "__main__":
    sys.exit(main())
```

src/admission.py (222 lines)

```
"""
admission.py -- M3: checkpoint-arena accounting and admission tests.

The arena
---------
A checkpoint moves a process's memory from RAM to disk. That is only a
win if the disk side is bounded and the move itself cannot make things worse.
The **arena** is the directory that holds CRIU image sets, with a fixed byte
budget (``arena_size_mib``). It is managed like a simple allocator:

* ``used``: bytes of image sets already on disk (measured with ``lstat``).
* ``reserved``: bytes promised to dumps that are in flight. A dump's size
  is only known afterwards, so the cost model's estimate is reserved *before*
  CRIU starts and released when it finishes. This is the same
  reserve-then-commit pattern a kernel uses for memory overcommit
  accounting.
* ``capacity``: the configured budget.

The arena must live on a persistent, native filesystem (ext4 inside WSL2).
On ``tmpfs`` the "evicted" pages would still be in RAM, and on ``/mnt/c``
(9P to Windows, OneDrive-synced) image files could be modified behind
CRIU's back.

Admission tests
---------------
Before **capture** (checkpoint + evict), all must hold:

1. The cost model judged the process *eligible* (worth preserving).
2. Estimated dump time <= ``max_capture_s``. A dump that takes longer than
   the time left before the OOM killer acts would be wasted work.
3. Fewer than ``max_inflight_captures`` dumps are running. CRIU freezes
   the target and reads all of its memory, so parallel dumps compete for the
   same disk and prolong the stall.
4. ``used + reserved + estimate <= capacity`` (arena budget).
5. The filesystem keeps at least ``fs_reserve_mib`` free afterwards
   (``statvfs`` free blocks), so a full arena cannot fill the root disk.

Before **restore**, all must hold:

1. Pressure has *recovered*: PSI and usage stayed below the recovery
   thresholds for several consecutive samples. This hysteresis prevents
   thrashing between dump and restore.
2. The cgroup has headroom: ``limit - memory.current >= RSS * (1 + margin)``,
   where ``limit`` is the tighter of ``memory.high`` and ``memory.max``.
   Restoring into a cgroup without room for the process would itself trigger
   reclaim or an OOM kill.

Standalone use (only touches a private temporary directory)::

    python3 src/admission.py
"""
from __future__ import annotations

import argparse
import os
import sys
import tempfile
from dataclasses import dataclass
from pathlib import Path

from config import Config
from costmodel import Score
from sensor import MIB, CgroupMemory


@dataclass(frozen=True)
class Decision:
    admitted: bool
    reason: str

    def __str__(self) -> str:
        return f"{'ADMIT' if self.admitted else 'DENY'}: {self.reason}"


def tree_bytes(path: Path) -> int:
    """Total size of regular files below ``path`` (no symlink following)."""
    total = 0
    for dirpath, _dirs, files in os.walk(path):
        for name in files:
            try:
                total += os.lstat(os.path.join(dirpath, name)).st_size
            except FileNotFoundError:
                pass
    return total
```

src/admission.py (222 lines) (continued)

```python


class Arena:
    """Byte-budgeted directory of CRIU image sets."""

    def __init__(self, path: Path, capacity_bytes: int, fs_reserve_bytes: int) -> None:
        self.path = Path(path)
        self.capacity_bytes = capacity_bytes
        self.fs_reserve_bytes = fs_reserve_bytes
        self.reservations: dict[str, int] = {}

    @classmethod
    def from_config(cls, cfg: Config, path: Path | None = None) -> "Arena":
        return cls(
            path or cfg.get_path("paths", "arena_dir"),
            int(cfg.get_float("admission", "arena_size_mib") * MIB),
            int(cfg.get_float("admission", "fs_reserve_mib") * MIB),
        )

    def ensure(self) -> None:
        """Create the arena directory (mode 0700: images contain process memory)."""
        self.path.mkdir(mode=0o700, parents=True, exist_ok=True)

    def contains(self, p: Path) -> bool:
        """True if ``p`` resolves to a location strictly inside the arena."""
        try:
            return Path(p).resolve().is_relative_to(self.path.resolve()) and \
                Path(p).resolve() != self.path.resolve()
        except OSError:
            return False

    def used_bytes(self) -> int:
        return tree_bytes(self.path) if self.path.exists() else 0

    def reserved_bytes(self) -> int:
        return sum(self.reservations.values())

    def fs_free_bytes(self) -> int:
        st = os.statvfs(self.path if self.path.exists() else self.path.parent)
        return st.f_bavail * st.f_frsize

    def reserve(self, key: str, nbytes: int) -> None:
        self.reservations[key] = nbytes

    def release(self, key: str) -> None:
        self.reservations.pop(key, None)

    def summary(self) -> str:
        return (f"arena {self.path}: used={self.used_bytes() / MIB:.1f}MiB "
                f"reserved={self.reserved_bytes() / MIB:.1f}MiB "
                f"capacity={self.capacity_bytes / MIB:.0f}MiB")


class AdmissionController:
    """Implements the capture and restore admission tests above."""

    def __init__(self, cfg: Config, arena: Arena) -> None:
        s = "admission"
        self.arena = arena
        self.max_inflight = cfg.get_int(s, "max_inflight_captures")
        self.max_capture_s = cfg.get_float(s, "max_capture_s")
        self.restore_margin = cfg.get_float(s, "restore_headroom_margin")

    def admit_capture(self, score: Score) -> Decision:
        if not score.eligible:
            return Decision(False, f"not worth preserving ({score.reason})")
        if score.capture_cost_s > self.max_capture_s:
            return Decision(False, f"estimated dump {score.capture_cost_s:.2f}s > "
                                   f"max_capture_s {self.max_capture_s}s")
        if len(self.arena.reservations) >= self.max_inflight:
            return Decision(False, f"{len(self.arena.reservations)} capture(s) in flight "
                                   f"(max {self.max_inflight})")
        est = score.est_image_bytes
        committed = self.arena.used_bytes() + self.arena.reserved_bytes()
        if committed + est > self.arena.capacity_bytes:
            return Decision(False, f"arena full: {committed / MIB:.1f}+{est / MIB:.1f}MiB > "
                                   f"{self.arena.capacity_bytes / MIB:.0f}MiB")
        free = self.arena.fs_free_bytes()
        if free - est < self.arena.fs_reserve_bytes:
            return Decision(False, f"filesystem would drop below reserve: free "
                                   f"{free / MIB:.0f}MiB - {est / MIB:.1f}MiB < "
                                   f"{self.arena.fs_reserve_bytes / MIB:.0f}MiB")
        return Decision(True, f"image ~{est / MIB:.1f}MiB fits arena "
                              f"({committed / MIB:.1f}/{self.arena.capacity_bytes / MIB:.0f}MiB used), "
```

src/admission.py (222 lines) (continued)

```
                              f"dump ~{score.capture_cost_s:.2f}s")

    def admit_restore(self, rss_bytes: int, cg: CgroupMemory | None, recovered: bool) -> Decision:
        if not recovered:
            return Decision(False, "pressure has not stayed below recovery thresholds yet")
        if cg is None or cg.effective_limit is None:
            return Decision(True, "cgroup has no memory limit")
        need = int(rss_bytes * (1 + self.restore_margin))
        headroom = cg.effective_limit - cg.current
        if headroom < need:
            return Decision(False, f"headroom {headroom / MIB:.1f}MiB < needed {need / MIB:.1f}MiB "
                                   f"(RSS x {1 + self.restore_margin:.2f})")
        return Decision(True, f"headroom {headroom / MIB:.1f}MiB >= needed {need / MIB:.1f}MiB")


def _main() -> int:
    ap = argparse.ArgumentParser(description="Exercise arena accounting and admission tests (M3).")
    ap.add_argument("--config", default=None)
    args = ap.parse_args()
    cfg = Config(args.config)

    with tempfile.TemporaryDirectory(prefix="preserve-arena-test-") as tmp:
        arena = Arena.from_config(cfg, Path(tmp) / "arena")
        arena.ensure()
        ctl = AdmissionController(cfg, arena)
        print(arena.summary())

        def mk(eligible: bool, image_mib: float, cost_s: float) -> Score:
            return Score(1, "synthetic", image_mib, 60.0, cost_s, int(image_mib * MIB),
                         1.0, eligible, "synthetic")

        cases = [
            ("eligible, small", mk(True, 128, 1.0)),
            ("not eligible", mk(False, 128, 1.0)),
            ("dump too slow", mk(True, 128, ctl.max_capture_s + 1)),
            ("larger than arena", mk(True, arena.capacity_bytes / MIB + 1, 1.0)),
        ]
        for label, sc in cases:
            print(f"capture  {label:20s} -> {ctl.admit_capture(sc)}")

        arena.reserve("inflight", 64 * MIB)
        print(f"capture  {'while one in flight':20s} -> {ctl.admit_capture(mk(True, 128, 1.0))}")
        arena.release("inflight")

        limit = 512 * MIB
        for current_mib, recovered in [(300, True), (450, True), (300, False)]:
            cg = CgroupMemory(Path("/synthetic"), current_mib * MIB, limit, None, 0, None, 0, 0)
            print(f"restore  cur={current_mib}MiB/512MiB recovered={recovered!s:5} -> "
                  f"{ctl.admit_restore(128 * MIB, cg, recovered)}")
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

src/capture.py (376 lines)

```
"""
capture.py -- M4: CRIU checkpoint (dump) and restore wrapper.

What CRIU does
--------------
CRIU (Checkpoint/Restore In Userspace) saves a running process tree to a set
of image files and later recreates it, with the same PID, from those files.

**Dump** (``criu dump -t PID``):

1. *Seize and freeze*: CRIU attaches to every task in the tree with
   ``ptrace(PTRACE_SEIZE)`` and interrupts it, so nothing changes while it
   is being read.
2. *Collect state* from procfs and syscalls: memory mappings
   (``/proc/PID/smaps``, ``map_files``), registers, signal masks, open file
   descriptors with their offsets and flags, credentials, session/process
   group, namespaces, cgroup membership.
3. *Dump memory*: a small "parasite" blob is injected into the frozen task.
   It runs inside the victim's address space and streams private pages to
   CRIU through a pipe (``vmsplice``), which writes ``pages-*.img``.
4. *Finish*: by default the dumped tasks are then **killed**. For PRESERVE
   that is the point: their memory is released back to the cgroup, which is
   what relieves pressure. (``--leave-running`` and ``--leave-stopped`` would
   keep the RAM occupied, so this wrapper refuses those flags.)

**Restore** (``criu restore -D dir``): CRIU forks a task with the original
PID (``clone3`` with ``set_tid``, which needs ``CAP_CHECKPOINT_RESTORE`` or
root), re-creates mappings, reloads pages, reopens files at their saved
offsets, restores registers, and finally lets the process continue from the
instruction where it was frozen. ``--restore-detached`` lets CRIU exit once
the tree is running, so the restored process is reparented to init.

Constraints this places on workloads
------------------------------------
* The PID must be **free** at restore time. After a dump the governor
  reaps its dead child (``waitpid``) immediately, otherwise a zombie would
  keep holding the PID.
* Files open at dump time must be reopenable unchanged at restore. CRIU
  records each regular file's path and size. The workloads therefore send
  stdio to ``/dev/null`` and only hold their own state files open briefly,
  never pipes back to the governor (a pipe whose other end has gone cannot
  be restored).
* ``--shell-job`` lets CRIU dump a session leader whose session/process
  group were inherited from outside the dumped tree. The workloads are
  started with ``setsid`` (``start_new_session=True``), as in the
  feasibility test recorded in ``evidence/00_criu_feasibility.log``.

Confinement on restore
----------------------
The ``criu restore`` command itself is started inside the test cgroup (same
``preexec_fn`` as the workloads). A forked child inherits its parent's
cgroup, so the restored tree is *born confined*. The governor then checks
``/proc/PID/cgroup`` anyway.

Safety
------
:meth:`CriuCapture.dump` and :meth:`CriuCapture.restore` take a PID, and their
first statement is the whitelist gate (:meth:`cgroup.PidWhitelist.verify`).
There is no code path in this module that runs CRIU on a PID the gate did
not approve. The gate is checked a second time immediately before CRIU is
executed, to narrow the window in which the PID could change hands.

Standalone use::

    sudo python3 src/capture.py --check          # criu check (+ --extra)
    python3 src/capture.py --refusal-demo        # show dump() refusing a foreign PID
"""
from __future__ import annotations

import argparse
import logging
import os
import shutil
import subprocess
import sys
import time
from dataclasses import dataclass
from pathlib import Path

from admission import Arena, tree_bytes
from cgroup import PRESERVED, PidWhitelist, SafetyRefusal, TestCgroup
from config import Config
from sensor import MIB

```

src/capture.py (376 lines) (continued)

```python
log = logging.getLogger("preserve.capture")

FORBIDDEN_DUMP_FLAGS = frozenset({"--leave-running", "-R", "--leave-stopped", "-s"})
FAULT_DUMP_TIMEOUT_S = 0.05  # dump-timeout fault: deadline short enough that criu cannot finish
FAULT_TMPFS_SIZE = "16m"     # dump-fail fault: tmpfs far smaller than the image (~307 MiB) -> ENOSPC


@dataclass
class CaptureResult:
    ok: bool
    action: str  # "dump" | "restore"
    pid: int
    image_dir: Path | None
    seconds: float
    image_bytes: int = 0
    returncode: int | None = None
    error: str = ""
    log_tail: str = ""
    setup_failed: bool = False  # fault injection could not be established (e.g. tmpfs did not mount)
    enospc: bool = False        # the dump failed specifically with ENOSPC (disk/tmpfs full)

    def describe(self) -> str:
        if self.ok:
            return (f"{self.action} pid={self.pid} OK in {self.seconds:.2f}s, "
                    f"image {self.image_bytes / MIB:.1f}MiB at {self.image_dir}")
        return f"{self.action} pid={self.pid} FAILED (rc={self.returncode}): {self.error}"


def _tail(path: Path, n: int) -> str:
    try:
        return "\n".join(path.read_text(errors="replace").splitlines()[-n:])
    except OSError as exc:
        return f"(could not read {path}: {exc})"


class CriuCapture:
    """Runs CRIU on whitelisted processes only."""

    def __init__(self, cfg: Config, whitelist: PidWhitelist, cgroup: TestCgroup, arena: Arena) -> None:
        s = "capture"
        self.whitelist = whitelist
        self.cgroup = cgroup
        self.arena = arena
        self.criu = cfg.get_str(s, "criu_bin")
        self.shell_job = cfg.get_bool(s, "shell_job")
        self.verbosity = cfg.get_int(s, "log_level")
        self.dump_timeout_s = cfg.get_float(s, "dump_timeout_s")
        self.restore_timeout_s = cfg.get_float(s, "restore_timeout_s")
        self.keep_images = cfg.get_bool(s, "keep_images_after_restore")
        self.log_tail_lines = cfg.get_int(s, "log_tail_lines")
        self.dump_extra = cfg.get_args(s, "dump_extra_args")
        self.restore_extra = cfg.get_args(s, "restore_extra_args")
        bad = FORBIDDEN_DUMP_FLAGS.intersection(self.dump_extra)
        if bad:
            raise ValueError(f"dump_extra_args may not contain {sorted(bad)}: "
                             "eviction requires CRIU to kill the dumped task")

    # -- environment -----------------------------------------------------------
    def criu_path(self) -> str | None:
        return shutil.which(self.criu) or shutil.which(self.criu, path="/usr/sbin:/sbin")

    def available(self) -> tuple[bool, str]:
        path = self.criu_path()
        if path is None:
            return False, f"{self.criu} not found"
        if os.geteuid() != 0:
            return False, "CRIU dump/restore needs root (run the governor with sudo)"
        return True, path

    def check(self, extra: bool = False) -> tuple[int, str]:
        """Run ``criu check [--extra]`` and return (returncode, combined output)."""
        cmd = [self.criu_path() or self.criu, "check"] + (["--extra"] if extra else [])
        p = subprocess.run(cmd, stdin=subprocess.DEVNULL, capture_output=True, text=True)
        return p.returncode, (p.stdout + p.stderr).strip()

    def _common_flags(self, logname: str) -> list[str]:
        return [f"-v{self.verbosity}", "-o", logname] + (["--shell-job"] if self.shell_job else [])

    # -- dump ------------------------------------------------------------------
    def dump(self, pid: int, fault: str | None = None, members: list[int] | None = None) -> CaptureResult:
        """Checkpoint a whitelisted process into a new arena image set, then let CRIU kill it.

        ``fault`` (fail-safe demonstration, from governor --inject):
          * ``dump-fail``: a tiny tmpfs (``FAULT_TMPFS_SIZE``, far smaller than
```

```
            the image) is mounted over the image directory, so CRIU's writes
            genuinely fail with ENOSPC partway through -- a real disk-full
            failure that even root cannot bypass (unlike a chmod, which
            CAP_DAC_OVERRIDE ignores). The mount is verified before CRIU runs
            and always unmounted afterwards.
          * ``dump-timeout``: the subprocess deadline is cut to a fraction of a
            second, so ``criu dump`` is still working when it is killed, exercising
            the real timeout path.
        Neither weakens any check; both make a real dump genuinely not complete.

        ``members`` (Objective 6): a dependent group rooted at ``pid``. CRIU dumps
        the whole tree from the root, so EVERY member must pass the gate, and all
        of them are marked preserved with the same image set. The caller has
        already checked that the tree is exactly these members (deps.capturable).
        """
        others = [m for m in (members or []) if m != pid]
        entry = self.whitelist.verify(pid, "checkpoint")  # SAFETY GATE
        for m in others:
            self.whitelist.verify(m, "checkpoint")  # SAFETY GATE, every member
        image_dir = self.arena.path / f"{entry.role}-{pid}-{time.strftime('%Y%m%d-%H%M%S')}"
        image_dir.mkdir(mode=0o700)
        cmd = [self.criu_path() or self.criu, "dump", "-t", str(pid), "-D", str(image_dir)]
        cmd += self._common_flags("dump.log") + self.dump_extra

        timeout = self.dump_timeout_s
        tmpfs_mounted = False
        if fault == "dump-fail":
            # Mount a tmpfs too small to hold the image; verify it took effect, or abort.
            if not self._mount_tmpfs(image_dir, FAULT_TMPFS_SIZE):
                log.error("FAULT INJECTION [dump-fail]: tmpfs did NOT mount on %s; aborting rather "
                          "than running an unfaulted dump", image_dir)
                self._umount(image_dir)  # best effort, in case of a half-mount
                return CaptureResult(False, "dump", pid, image_dir, 0.0, setup_failed=True,
                                     error="INJECTION SETUP FAILED: tmpfs did not mount on the image dir")
            tmpfs_mounted = True
            log.warning("FAULT INJECTION [dump-fail]: %s tmpfs (%s) mounted over %s; CRIU writes will "
                        "hit ENOSPC", FAULT_TMPFS_SIZE, "size", image_dir)
        elif fault == "dump-timeout":
            timeout = FAULT_DUMP_TIMEOUT_S
            log.warning("FAULT INJECTION [dump-timeout]: dump deadline cut to %.2fs so the dump "
                        "cannot complete in time", timeout)

        self.whitelist.verify(pid, "checkpoint")  # re-check right before exec
        for m in others:
            self.whitelist.verify(m, "checkpoint")
        log.info("CRIU dump %s pid=%d%s -> %s", entry.role, pid,
                 f" (tree with members {others})" if others else "", image_dir)
        log.debug("exec: %s", " ".join(cmd))
        t0 = time.monotonic()
        try:
            try:
                p = subprocess.run(cmd, stdin=subprocess.DEVNULL, capture_output=True, text=True,
                                   timeout=timeout)
            except subprocess.TimeoutExpired:
                secs = time.monotonic() - t0
                return CaptureResult(False, "dump", pid, image_dir, secs, error=f"timed out after {secs:.1f}s",
                                     log_tail=_tail(image_dir / "dump.log", self.log_tail_lines))
            secs = time.monotonic() - t0

            if p.returncode != 0:
                # On failure CRIU resumes the frozen task; it keeps running and stays whitelisted.
                combined = p.stderr + "\n" + _tail(image_dir / "dump.log", self.log_tail_lines)
                enospc = ("ENOSPC" in combined) or ("No space left on device" in combined)
                return CaptureResult(False, "dump", pid, image_dir, secs, returncode=p.returncode,
                                     error=(p.stderr.strip().splitlines() or ["criu dump failed"])[-1],
                                     log_tail=_tail(image_dir / "dump.log", self.log_tail_lines),
                                     enospc=enospc)

            # CRIU killed the task. Reap our child now so the PID is free for restore.
            if entry.popen is not None:
                try:
                    entry.returncode = entry.popen.wait(timeout=self.dump_timeout_s)
                except subprocess.TimeoutExpired:
                    log.error("dumped pid %d did not exit after dump", pid)
            self.whitelist.mark_preserved(pid, image_dir)
            for m in others:
                self.whitelist.mark_preserved(m, image_dir)
            return CaptureResult(True, "dump", pid, image_dir, secs, tree_bytes(image_dir), 0)
        finally:
            if tmpfs_mounted:
                self._umount(image_dir)  # always unmount, even on success or exception

    # -- tmpfs helpers for the dump-fail injection ---------------------------------
    def _mount_tmpfs(self, path: Path, size: str) -> bool:
```

```
        """Mount a size-limited tmpfs over ``path`` and confirm it took effect."""
        subprocess.run(["mount", "-t", "tmpfs", "-o", f"size={size},mode=0700", "tmpfs", str(path)],
                       stdin=subprocess.DEVNULL, capture_output=True, text=True)
        return self._is_mountpoint(path)

    @staticmethod
    def _is_mountpoint(path: Path) -> bool:
        """True iff ``path`` is a tmpfs mount point right now (checked via /proc/self/mounts)."""
        target = str(Path(path).resolve())
        try:
            for line in Path("/proc/self/mounts").read_text().splitlines():
                parts = line.split()
                if len(parts) >= 3 and parts[1] == target and parts[2] == "tmpfs":
                    return True
        except OSError:
            pass
        return False

    def _umount(self, path: Path) -> None:
        """Unmount ``path`` if mounted; fall back to a lazy unmount. Never raises."""
        if not self._is_mountpoint(path):
            return
        if subprocess.run(["umount", str(path)], stdin=subprocess.DEVNULL,
                          capture_output=True, text=True).returncode != 0:
            subprocess.run(["umount", "-l", str(path)], stdin=subprocess.DEVNULL,
                           capture_output=True, text=True)
        if self._is_mountpoint(path):
            log.error("could not unmount fault tmpfs at %s", path)

    # -- restore ------------------------------------------------------------------
    def restore(self, pid: int, members: list[int] | None = None) -> CaptureResult:
        """Restore a previously dumped whitelisted process, confined to the test cgroup.

        With ``members`` the whole tree is restored from the root's image set;
        every member must pass the gate and every member must be alive and
        confined afterwards, or the restore is reported as failed.
        """
        others = [m for m in (members or []) if m != pid]
        entry = self.whitelist.verify(pid, "restore", expect_state=PRESERVED)  # SAFETY GATE
        image_dir = entry.image_dir
        if image_dir is None or not self.arena.contains(image_dir):
            raise SafetyRefusal(f"image dir {image_dir} for pid {pid} is not inside the arena")
        for m in others:
            me = self.whitelist.verify(m, "restore", expect_state=PRESERVED)  # SAFETY GATE
            if me.image_dir != image_dir:
                raise SafetyRefusal(f"member pid {m} belongs to image {me.image_dir}, not {image_dir}")
        # Dumped members that were not our children are reaped by init; give it a moment.
        deadline = time.monotonic() + 5.0
        while any(Path(f"/proc/{p}").exists() for p in [pid] + others) and time.monotonic() < deadline:
            time.sleep(0.1)
        busy = [p for p in [pid] + others if Path(f"/proc/{p}").exists()]
        if busy:
            return CaptureResult(False, "restore", pid, image_dir, 0.0,
                                 error=f"PID(s) {busy} in use by another process; CRIU cannot reuse them")

        pidfile = image_dir / "restore.pid"
        pidfile.unlink(missing_ok=True)
        cmd = [self.criu_path() or self.criu, "restore", "-D", str(image_dir),
               "--restore-detached", "--pidfile", str(pidfile)]
        cmd += self._common_flags("restore.log") + self.restore_extra
        log.info("CRIU restore %s pid=%d <- %s", entry.role, pid, image_dir)
        log.debug("exec: %s", " ".join(cmd))
        t0 = time.monotonic()
        try:
            p = subprocess.run(cmd, stdin=subprocess.DEVNULL, capture_output=True, text=True,
                               timeout=self.restore_timeout_s,
                               preexec_fn=self.cgroup.preexec_enter(None))
        except subprocess.TimeoutExpired:
            secs = time.monotonic() - t0
            return CaptureResult(False, "restore", pid, image_dir, secs, error=f"timed out after {secs:.1f}s",
                                 log_tail=_tail(image_dir / "restore.log", self.log_tail_lines))
        secs = time.monotonic() - t0
        if p.returncode != 0:
            return CaptureResult(False, "restore", pid, image_dir, secs, returncode=p.returncode,
                                 error=(p.stderr.strip().splitlines() or ["criu restore failed"])[-1],
                                 log_tail=_tail(image_dir / "restore.log", self.log_tail_lines))

        restored = int(pidfile.read_text().strip())
        if restored != pid:
            log.error("CRIU restored pid %d but image was pid %d; leaving it untouched", restored, pid)
            return CaptureResult(False, "restore", pid, image_dir, secs, returncode=0,
                                 error=f"restored under unexpected pid {restored}")
        size = tree_bytes(image_dir)
        self.whitelist.mark_restored(pid)
```

src/capture.py (376 lines) (continued)

```
        for m in others:  # raises CgroupError if a member is not running or not confined
            self.whitelist.mark_restored(m)
            log.info("restored member pid=%d alive and confined in %s", m, self.cgroup.rel)
        if not self.keep_images:
            shutil.rmtree(image_dir)
        return CaptureResult(True, "restore", pid, image_dir, secs, size, 0)


def _main() -> int:
    ap = argparse.ArgumentParser(description="CRIU wrapper (M4).")
    ap.add_argument("--config", default=None)
    g = ap.add_mutually_exclusive_group(required=True)
    g.add_argument("--check", action="store_true", help="run criu check and criu check --extra")
    g.add_argument("--refusal-demo", action="store_true",
                   help="ask dump() to checkpoint foreign PIDs; must be refused before CRIU runs")
    args = ap.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

    cfg = Config(args.config)
    cg = TestCgroup.from_config(cfg)
    cap = CriuCapture(cfg, PidWhitelist(cg), cg, Arena.from_config(cfg))
    if args.check:
        ok, msg = cap.available()
        print(f"available: {ok} ({msg})")
        for extra in (False, True):
            rc, out = cap.check(extra)
            print(f"--- criu check{' --extra' if extra else ''} (rc={rc}) ---\n{out}")
        return 0
    for pid in (1, os.getpid(), os.getppid()):
        try:
            cap.dump(pid)
            print(f"pid {pid}: dump was NOT refused -- this is a bug")
            return 1
        except SafetyRefusal as exc:
            print(f"pid {pid}: refused before CRIU ran: {exc}")
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

src/cgroup.py (504 lines)

```
"""
cgroup.py -- test-cgroup lifecycle, process confinement, and the PID whitelist.

Part 1: the test cgroup
-----------------------
Control groups (cgroups) are the kernel mechanism for grouping processes and
applying resource controllers to the group. cgroup v2 has a single
hierarchy mounted at ``/sys/fs/cgroup``. Every directory is a cgroup, and its
files are the controller interface: writing ``536870912`` to ``memory.max``
caps the group at 512 MiB, and writing a PID to ``cgroup.procs`` moves that
process into the group.

**Why experiments are safe.** Every workload runs in one leaf cgroup whose
``memory.max`` and ``memory.swap.max`` are far below the machine's RAM. When
the group exceeds its limit, the kernel reclaims or OOM-kills *inside that
group only* (cgroup-local OOM). Processes outside it are unaffected. Under
WSL2 there is a second wall: the whole Linux kernel runs in a Hyper-V VM, so
even a total failure is contained in the VM, not Windows.

**One cgroup per run, never shared.** The configured ``[cgroup] name`` is only
a *prefix*. Each governor run creates ``<name>-<pid>-<YYYYmmddHHMMSS>`` (see
:meth:`TestCgroup.for_run`) with an atomic ``mkdir``: if the directory already
exists, creation fails loudly instead of adopting it. Two runs therefore
cannot share a memory limit, even if they start at the same instant. Before
this, every run used the bare name ``preserve-test``; two concurrent runs then
shared one 512 MiB limit, pushed each other's workloads into the OOM killer,
and produced output that looked valid. ``runlock.py`` additionally stops a
second governor from starting at all.

**Where the cgroup lives: systemd delegation.** systemd owns the cgroup tree.
The documented way for anything else to create cgroups is inside a subtree
systemd has *delegated*. Every logged-in user gets one at
``user.slice/user-<uid>.slice/user@<uid>.service``, with the ``memory``
controller enabled in its ``cgroup.subtree_control``. PRESERVE creates its
leaf cgroup there and **never modifies** the parent: if the parent does not
offer the memory controller, creation fails with an explanation.

**Confinement before exec.** A process must be inside the cgroup *before* it
allocates anything, or its first allocations would be charged elsewhere and
the limit would not apply. :meth:`TestCgroup.preexec_enter` returns a
function that :class:`subprocess.Popen` runs in the child after ``fork()``
and before ``execve()``. It writes the child's own PID into
``cgroup.procs`` and only then drops root privileges. By the time the
workload's first instruction runs, it is already confined. (Popen's own
``user=``/``group=`` arguments are not used because they drop privileges
*before* ``preexec_fn`` runs, and an unprivileged child could not write to
the root-owned ``cgroup.procs``.)

Part 2: the PID whitelist (safety gate)
---------------------------------------
Checkpointing kills the dumped process, so pointing CRIU at the wrong PID is
destructive. :class:`PidWhitelist` is the **only** path through which the
governor may checkpoint or signal a process. :meth:`PidWhitelist.verify`
contains hard-coded refusals that no configuration can switch off:

1. The PID must be an int greater than 1. PID 0 (scheduler/kernel) and PID
   1 (init) are always refused.
2. It must not be the governor itself or its parent.
3. It must have been *launched by this governor* (registered by
   :meth:`admit_launched`). Membership in the cgroup alone is not enough.
4. Its recorded ``starttime`` (``/proc/<pid>/stat`` field 22) must still
   match. PIDs are recycled by the kernel, so a PID that was ours may now
   belong to a stranger. (pid, starttime) is unique for the life of the
   kernel, so this defeats PID reuse.
5. ``/proc/<pid>/cgroup`` must show that it is *currently inside the test
   cgroup*.
6. The action must be one of the known actions, and a signal must be one of
   SIGTERM, SIGINT or SIGKILL.

Every refusal raises :class:`SafetyRefusal` and is logged at ERROR level.

Standalone use::

    python3 src/cgroup.py             # show where the cgroup would live and its state (read-only)
    python3 src/cgroup.py --create    # create it with limits from preserve.conf (needs permission)
    python3 src/cgroup.py --destroy   # remove it if empty
    python3 src/cgroup.py --refusal-demo   # show the whitelist refusing foreign PIDs (no side effects)
"""
from __future__ import annotations

import argparse
import logging
import os
import re
```

src/cgroup.py (504 lines) (continued)

```python
import signal
import subprocess
import sys
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Callable

from config import Config, format_size
from sensor import CGROUP_ROOT, PAGE_SIZE, proc_cgroup, read_proc

log = logging.getLogger("preserve.cgroup")

_NAME_RE = re.compile(r"^[A-Za-z0-9_.-]+$")


class CgroupError(RuntimeError):
    """The test cgroup could not be created, configured or removed."""


class SafetyRefusal(PermissionError):
    """The governor was asked to act on a process it is not allowed to touch."""


# ============================================================================
# Part 1: the test cgroup
# ============================================================================
class TestCgroup:
    """One leaf cgroup-v2 directory with memory limits, owned by the governor."""

    def __init__(self, parent: Path, name: str) -> None:
        if not _NAME_RE.match(name) or name in (".", ".."):
            raise CgroupError(f"invalid cgroup name {name!r}")
        self.parent = Path(parent)
        if not self.parent.resolve().is_relative_to(CGROUP_ROOT):
            raise CgroupError(f"parent {self.parent} is not under {CGROUP_ROOT}")
        self.path = self.parent / name
        # Path as it appears in /proc/<pid>/cgroup ("0::/user.slice/...").
        self.rel = "/" + str(self.path.relative_to(CGROUP_ROOT))
        self.created = False  # True only once THIS object's mkdir succeeded

    @classmethod
    def from_config(cls, cfg: Config) -> "TestCgroup":
        """The bare configured name -- for the standalone inspection tool only.

        Governor runs must use :meth:`for_run`, never this.
        """
        return cls(Path(cfg.get_str("cgroup", "parent")), cfg.get_str("cgroup", "name"))

    @classmethod
    def for_run(cls, cfg: Config) -> "TestCgroup":
        """A cgroup name unique to this run: ``<prefix>-<pid>-<YYYYmmddHHMMSS>``.

        The pid makes it unique among live processes, and the timestamp keeps it
        unique even if the kernel later reuses the pid.
        """
        name = f"{cfg.get_str('cgroup', 'name')}-{os.getpid()}-{time.strftime('%Y%m%d%H%M%S')}"
        return cls(Path(cfg.get_str("cgroup", "parent")), name)

    # -- inspection ------------------------------------------------------------
    @staticmethod
    def unified_hierarchy_mounted() -> bool:
        return (CGROUP_ROOT / "cgroup.controllers").exists()

    def parent_controllers(self) -> list[str]:
        """Controllers the parent hands down to its children (subtree_control)."""
        return (self.parent / "cgroup.subtree_control").read_text().split()

    def exists(self) -> bool:
        return (self.path / "cgroup.procs").exists()

    def procs(self) -> set[int]:
        if not self.exists():
            return set()
        return {int(x) for x in (self.path / "cgroup.procs").read_text().split()}

    def contains(self, pid: int) -> bool:
        """True if ``pid`` is currently a member of this cgroup."""
        return proc_cgroup(pid) == self.rel

    def read(self, knob: str) -> str:
        return (self.path / knob).read_text().strip()

    def _write(self, knob: str, value: str) -> None:
```

src/cgroup.py (504 lines) (continued)

```
        (self.path / knob).write_text(value)

    # -- lifecycle -----------------------------------------------------------
    def create(self, memory_max: int | None, swap_max: int | None, memory_high: int | None) -> None:
        """Create the leaf cgroup and apply limits. Never touches the parent."""
        if not self.unified_hierarchy_mounted():
            raise CgroupError("cgroup v2 unified hierarchy is not mounted at /sys/fs/cgroup")
        if not self.parent.is_dir():
            raise CgroupError(f"parent cgroup {self.parent} does not exist "
                              "(is systemd running and the user logged in?)")
        if "memory" not in self.parent_controllers():
            raise CgroupError(f"memory controller is not enabled in {self.parent}/cgroup.subtree_control; "
                              "PRESERVE will not modify the parent")
        # Atomic: mkdir either creates the directory or fails with EEXIST, so there
        # is no window between "check" and "create" for another run to slip into.
        # An existing directory is NEVER reused, empty or not.
        try:
            self.path.mkdir()
        except FileExistsError:
            members = sorted(self.procs())
            raise CgroupError(
                f"REFUSING TO SHARE A CGROUP: {self.path} already exists"
                + (f" and contains PIDs {members}" if members else " (empty)")
                + ". Each run must have its own cgroup; another run is using this name, "
                  "or a previous run did not clean up.") from None
        self.created = True
        log.info("created cgroup %s (unique to this run)", self.path)

        for knob, value in (("memory.max", memory_max),
                            ("memory.swap.max", swap_max),
                            ("memory.high", memory_high)):
            self._write(knob, format_size(value))
            got = self.read(knob)
            # The kernel rounds limits down to a page boundary.
            ok = (got == "max") if value is None else (got != "max" and abs(int(got) - value) < PAGE_SIZE)
            if not ok:
                raise CgroupError(f"{knob}: wrote {format_size(value)}, kernel reports {got}")
            log.info("  %s = %s", knob, got)

    def destroy(self) -> None:
        """rmdir the cgroup -- but only one THIS run created.

        If create() failed because the directory already existed, that directory
        belongs to someone else; removing it (even when momentarily empty) would
        pull the cgroup out from under another run.
        """
        if not self.created:
            if self.exists():
                log.info("not removing %s: this run did not create it", self.path)
            return
        if not self.exists():
            return
        members = self.procs()
        if members:
            raise CgroupError(f"not removing {self.path}: still contains {sorted(members)}")
        self.path.rmdir()
        log.info("removed cgroup %s", self.path)

    # -- confinement ---------------------------------------------------------
    def preexec_enter(self, drop_to: tuple[int, int] | None) -> Callable[[], None]:
        """Build a ``preexec_fn`` that joins this cgroup, then optionally drops root.

        Runs in the forked child before exec. Only plain ``os`` calls are used,
        because the child is a copy of the parent at fork time and must not
        rely on locks other threads may have held. If anything raises here,
        Popen reports it and the child never execs.
        """
        procs_file = str(self.path / "cgroup.procs")

        def _enter() -> None:
            fd = os.open(procs_file, os.O_WRONLY)
            try:
                os.write(fd, str(os.getpid()).encode())
            finally:
                os.close(fd)
            if drop_to is not None:
                uid, gid = drop_to
                os.setgroups([])
                os.setgid(gid)
                os.setuid(uid)

        return _enter

    def migrate(self, pid: int) -> None:
```

src/cgroup.py (504 lines) (continued)

```
        """Move an existing process into this cgroup (used only after CRIU restore)."""
        self._write("cgroup.procs", str(pid))


# ============================================================================
# Part 2: the PID whitelist
# ============================================================================
RUNNING, PRESERVED, EXITED = "running", "preserved", "exited"

FORBIDDEN_PIDS = frozenset({0, 1})
ALLOWED_ACTIONS = frozenset({"checkpoint", "signal", "restore"})
ALLOWED_SIGNALS = frozenset({signal.SIGTERM, signal.SIGINT, signal.SIGKILL})


@dataclass
class ManagedProcess:
    """A process the governor launched into the test cgroup."""

    pid: int
    role: str
    start_ticks: int
    state: str = RUNNING
    popen: subprocess.Popen | None = None  # None after restore: no longer our child
    image_dir: Path | None = None  # set while PRESERVED
    state_file: Path | None = None  # workload-declared progress (JSON), optional
    returncode: int | None = None
    parent: int | None = None  # set for a child admitted via admit_child (its whitelisted parent)


class PidWhitelist:
    """The explicit set of PIDs the governor may act on (see module docstring)."""

    def __init__(self, cgroup: TestCgroup) -> None:
        self.cgroup = cgroup
        self._entries: dict[int, ManagedProcess] = {}

    # -- registration ------------------------------------------------------
    def admit_launched(self, popen: subprocess.Popen, role: str,
                       state_file: Path | None = None) -> ManagedProcess:
        """Register a child we just launched. Fails if it is not confined."""
        pid = popen.pid
        snap = read_proc(pid)
        if snap is None:
            entry = ManagedProcess(pid, role, -1, EXITED, popen, state_file=state_file,
                                   returncode=popen.poll())
            self._entries[pid] = entry
            log.warning("%s pid=%d exited immediately (rc=%s)", role, pid, entry.returncode)
            return entry
        if not self.cgroup.contains(pid):
            raise CgroupError(f"launched {role} pid={pid} is in {proc_cgroup(pid)}, "
                              f"not {self.cgroup.rel}")
        entry = ManagedProcess(pid, role, snap.start_ticks, RUNNING, popen, state_file=state_file)
        self._entries[pid] = entry
        log.info("whitelisted %s pid=%d starttime=%d cgroup=%s", role, pid, snap.start_ticks,
                 self.cgroup.rel)
        return entry

    def admit_child(self, pid: int, role: str, state_file: Path | None = None) -> ManagedProcess | None:
        """Register a process forked by a whitelisted, running parent (Objective 6).

        Admitted only if: its ppid is a whitelisted RUNNING entry whose recorded
        (pid, starttime) still matches, and the child itself is inside the test
        cgroup. The child is then keyed on its own (pid, starttime), so every
        later action still passes through verify() unchanged. Returns None
        (and admits nothing) if any condition fails.
        """
        if pid in self._entries:
            return None
        snap = read_proc(pid)
        if snap is None or not snap.alive:
            return None
        parent = self._entries.get(snap.ppid)
        if parent is None or parent.state != RUNNING:
            return None
        psnap = read_proc(parent.pid)
        if psnap is None or psnap.start_ticks != parent.start_ticks:
            log.warning("not admitting pid=%d: parent pid=%d starttime changed (PID reuse?)", pid, snap.ppid)
            return None
        if not self.cgroup.contains(pid):
            log.warning("not admitting pid=%d (child of %s pid=%d): outside the test cgroup (in %s)",
                        pid, parent.role, parent.pid, proc_cgroup(pid))
            return None
        entry = ManagedProcess(pid, role, snap.start_ticks, RUNNING, None, state_file=state_file,
                               parent=parent.pid)
```

src/cgroup.py (504 lines) (continued)

```
        self._entries[pid] = entry
        log.info("whitelisted %s pid=%d starttime=%d cgroup=%s (child of whitelisted %s pid=%d "
                 "starttime=%d)", role, pid, snap.start_ticks, self.cgroup.rel, parent.role,
                 parent.pid, parent.start_ticks)
        return entry

    # -- the safety gate -----------------------------------------------------
    def verify(self, pid: int, action: str, expect_state: str = RUNNING) -> ManagedProcess:
        """Return the entry for ``pid`` or raise SafetyRefusal. Hard-coded rules."""

        def refuse(why: str) -> None:
            log.error("SAFETY REFUSAL: %s pid=%r: %s", action, pid, why)
            raise SafetyRefusal(f"refusing to {action} pid {pid!r}: {why}")

        if action not in ALLOWED_ACTIONS:
            refuse("unknown action")
        if not isinstance(pid, int) or isinstance(pid, bool):
            refuse("pid is not an integer")
        if pid in FORBIDDEN_PIDS or pid <= 1:
            refuse("kernel/init PIDs are never touched")
        if pid in (os.getpid(), os.getppid()):
            refuse("that is the governor itself or its parent")
        entry = self._entries.get(pid)
        if entry is None:
            refuse("not launched by this governor (not in whitelist)")
        assert entry is not None
        if entry.state != expect_state:
            refuse(f"whitelisted {entry.role} is {entry.state}, expected {expect_state}")
        if expect_state == PRESERVED:
            return entry  # nothing is running under this PID by design
        snap = read_proc(pid)
        if snap is None or not snap.alive:
            refuse("process is not alive")
        assert snap is not None
        if snap.start_ticks != entry.start_ticks:
            refuse(f"PID was reused (starttime {snap.start_ticks} != recorded {entry.start_ticks})")
        if not self.cgroup.contains(pid):
            refuse(f"process is outside the test cgroup (in {proc_cgroup(pid)})")
        return entry

    def signal(self, pid: int, sig: int) -> None:
        """Send ``sig`` to a whitelisted, confined process."""
        if sig not in ALLOWED_SIGNALS:
            log.error("SAFETY REFUSAL: signal %s to pid=%r: signal not allowed", sig, pid)
            raise SafetyRefusal(f"signal {sig} is not in the allowed set")
        entry = self.verify(pid, "signal")
        os.kill(pid, sig)
        log.info("sent %s to %s pid=%d", signal.Signals(sig).name, entry.role, pid)

    # -- state transitions ---------------------------------------------------
    def mark_preserved(self, pid: int, image_dir: Path) -> None:
        entry = self._entries[pid]
        entry.state, entry.image_dir = PRESERVED, image_dir

    def mark_restored(self, pid: int) -> ManagedProcess:
        """After CRIU restore: refresh starttime and confirm confinement."""
        entry = self._entries[pid]
        snap = read_proc(pid)
        if snap is None:
            raise CgroupError(f"restored pid {pid} is not running")
        if not self.cgroup.contains(pid):
            log.warning("restored pid %d landed in %s; moving it into %s",
                        pid, proc_cgroup(pid), self.cgroup.rel)
            self.cgroup.migrate(pid)
            if not self.cgroup.contains(pid):
                raise CgroupError(f"restored pid {pid} could not be confined; "
                                  "it is NOT whitelisted and must be inspected manually")
        # The kernel assigns a new starttime to the restored task; record it so
        # the PID-reuse guard keeps working.
        entry.start_ticks = snap.start_ticks
        entry.state, entry.image_dir, entry.popen = RUNNING, None, None
        return entry

    def refresh(self) -> list[ManagedProcess]:
        """Reap children and detect exits. Returns entries that exited just now."""
        gone: list[ManagedProcess] = []
        for entry in self._entries.values():
            if entry.state != RUNNING:
                continue
            if entry.popen is not None:
                rc = entry.popen.poll()  # waitpid(WNOHANG): reaps the zombie
                if rc is None:
                    continue
                entry.returncode = rc
```

src/cgroup.py (504 lines) (continued)

```
            else:
                snap = read_proc(entry.pid)
                if snap is not None and snap.alive and snap.start_ticks == entry.start_ticks:
                    continue
            entry.state = EXITED
            gone.append(entry)
        return gone

    # -- queries -----------------------------------------------------------
    def get(self, pid: int) -> ManagedProcess | None:
        return self._entries.get(pid)

    def with_state(self, state: str) -> list[ManagedProcess]:
        return [e for e in self._entries.values() if e.state == state]

    def by_role(self, role: str) -> list[ManagedProcess]:
        return [e for e in self._entries.values() if e.role == role]


def _refusal_demo(cg: TestCgroup) -> int:
    """Show the gate refusing foreign PIDs. Never sends a signal or runs CRIU."""
    wl = PidWhitelist(cg)
    targets = [("init", 1), ("kernel", 0), ("this process", os.getpid()),
               ("parent shell", os.getppid()), ("non-integer", "1234"),
               ("not launched by us", 2)]
    for label, pid in targets:
        try:
            wl.verify(pid, "checkpoint")  # type: ignore[arg-type]
            print(f"  {label:20s} -> ALLOWED (this would be a bug)")
            return 1
        except SafetyRefusal as exc:
            print(f"  {label:20s} -> refused: {exc}")
    try:
        wl.signal(os.getpid(), signal.SIGHUP)
    except SafetyRefusal as exc:
        print(f"  {'SIGHUP to self':20s} -> refused: {exc}")
    return 0


def _main() -> int:
    ap = argparse.ArgumentParser(description="Inspect / manage the PRESERVE test cgroup.")
    ap.add_argument("--config", default=None)
    g = ap.add_mutually_exclusive_group()
    g.add_argument("--create", action="store_true")
    g.add_argument("--destroy", action="store_true")
    g.add_argument("--refusal-demo", action="store_true")
    args = ap.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

    cfg = Config(args.config)
    cg = TestCgroup.from_config(cfg)
    if args.refusal_demo:
        return _refusal_demo(cg)
    if args.create:
        cg.create(cfg.get_size("cgroup", "memory_max"), cfg.get_size("cgroup", "memory_swap_max"),
                  cfg.get_size("cgroup", "memory_high"))
    elif args.destroy:
        cg.destroy()
        return 0

    print(f"cgroup v2 mounted : {cg.unified_hierarchy_mounted()}")
    print(f"parent            : {cg.parent} (exists={cg.parent.is_dir()})")
    if cg.parent.is_dir():
        ctl = cg.parent_controllers()
        print(f"parent subtree    : {' '.join(ctl)}  (memory delegated: {'memory' in ctl})")
        print(f"parent writable   : {os.access(cg.parent, os.W_OK)}")
    # Governor runs use per-run names (<prefix>-<pid>-<timestamp>); list them all,
    # plus the bare legacy name, so leftovers and concurrent runs are visible.
    found = sorted(p for p in cg.parent.glob(f"{cg.path.name}-*") if (p / "cgroup.procs").exists()) \
        if cg.parent.is_dir() else []
    if cg.exists():
        found.insert(0, cg.path)
    print(f"run cgroups       : {len(found)} under prefix {cg.path.name!r}")
    for path in found:
        run = TestCgroup(cg.parent, path.name)
        print(f"  {path}")
        for knob in ("memory.max", "memory.swap.max", "memory.current"):
            print(f"    {knob:16s}: {run.read(knob)}")
        print(f"    members         : {sorted(run.procs())}")
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

src/compare.py (72 lines)

```
"""
compare.py -- side-by-side comparison of a PRESERVE live run and a baseline run.

An experiment is a *pair* of runs of the same scenario:

* the **baseline**: the governor only observes and the kernel's cgroup-local
  OOM killer decides (``run_demo.sh --scenario X --baseline``);
* the **live run**: PRESERVE checkpoints and restores
  (``run_demo.sh --scenario X``).

Both runs save an evidence directory (``evidence/demo-<stamp>-<mode>-<scenario>/``).
This tool reads two such directories, read-only, and prints the comparison
table that the report uses. It answers: what did the kernel kill, how much
accumulated work did that destroy, and how much did PRESERVE keep instead?

Usage::

    python3 src/compare.py --live evidence/<live dir> --baseline evidence/<baseline dir>
"""
from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

from config import PROJECT_ROOT
from governor import BASELINE_RESULT, load_reference_run, side_by_side_table


def _resolve(p: str) -> Path:
    path = Path(p)
    return path if path.is_absolute() else PROJECT_ROOT / path


def main() -> int:
    ap = argparse.ArgumentParser(description="Compare a live PRESERVE run with a kernel-OOM baseline.")
    ap.add_argument("--live", required=True, help="evidence directory of the live run")
    ap.add_argument("--baseline", required=True, help="evidence directory of the baseline run")
    args = ap.parse_args()

    live_dir, base_dir = _resolve(args.live), _resolve(args.baseline)
    ref = load_reference_run(live_dir)
    if not ref.get("available"):
        print(f"live run directory not found: {live_dir}")
        return 1
    try:
        base = json.loads((base_dir / BASELINE_RESULT).read_text())
    except (OSError, ValueError) as exc:
        print(f"cannot read {base_dir / BASELINE_RESULT}: {exc}")
        return 1
    base.setdefault("scenario", "A")  # baselines recorded before scenarios existed used the base values
    if ref.get("scenario") != base.get("scenario"):
        print(f"WARNING: comparing scenario {ref.get('scenario')} (live) with "
              f"scenario {base.get('scenario')} (baseline)")

    print(f"live run : {live_dir}")
    print(f"baseline : {base_dir}")
    for v in base.get("victims", []):
        print(f"kernel victim in baseline: {v['role']} pid={v['pid']} at iteration {v.get('last_iteration')}, "
              f"work destroyed {v.get('work_lost_s', 0):.2f}s")
        if v.get("kernel_line"):
            print(f"  {v['kernel_line']}")
    print("-" * 100)
    for name, a, b in side_by_side_table(ref, base):
        print(f"  {name:30s} | {str(a):46s} | {b}")
    print("-" * 100)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

src/config.py (242 lines)

```
"""
config.py -- configuration loading and logging shared by every PRESERVE module.

Role in the system
------------------
PRESERVE is a user-space memory-pressure governor. Every number that changes
its behaviour (cgroup limits, PSI thresholds, cost-model weights, arena size,
workload sizes) lives in ``preserve.conf`` so that experiments are reproduced
by editing one file rather than the code. This module is the single place
that reads that file, so no other module contains a magic number.

The file is parsed with :mod:`configparser` (INI syntax, standard library).
Two small conventions are added on top:

* ``{uid}`` inside a value expands to the uid of the *human* who started the
  program. Under ``sudo`` the effective uid is 0, so the real user's uid is
  taken from ``SUDO_UID``. This matters because systemd delegates a cgroup
  subtree per user (``user@<uid>.service``), and PRESERVE places its test
  cgroup inside that delegated subtree.
* Relative paths are resolved against the project root (the directory that
  holds ``preserve.conf``), never against the current working directory.
* **Named scenarios.** A section ``[scenario.<name>]`` overrides individual
  base values with keys of the form ``<section>.<key> = value``. For example,
  ``workloads.compute_working_set_mb = 300`` in ``[scenario.B]`` replaces
  ``[workloads] compute_working_set_mb`` when scenario B is selected. Every
  override must name a key that exists in the base sections, so a typo fails
  at start-up. ``[scenarios] default`` names the scenario used when none is
  given. Experiments therefore differ only in a small, readable set of
  overrides, and everything else is guaranteed identical.

Logging
-------
:func:`setup_logging` sends every record to stdout *and* to
``evidence/governor.log`` with a millisecond timestamp, so the console output
seen during a demo is preserved verbatim as evidence. When the governor runs
as root via sudo, the log file is handed back to the invoking user so that
later unprivileged runs can still append to it.

Standalone use::

    python3 src/config.py [--config PATH]     # print the parsed configuration
"""
from __future__ import annotations

import argparse
import configparser
import logging
import os
import re
import shlex
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent
DEFAULT_CONFIG_PATH = PROJECT_ROOT / "preserve.conf"

_SIZE_RE = re.compile(r"^\s*(\d+)\s*([KMGT]?)\s*$", re.IGNORECASE)
_SIZE_MULT = {"": 1, "K": 1 << 10, "M": 1 << 20, "G": 1 << 30, "T": 1 << 40}

LOGGER_NAME = "preserve"
SCENARIO_PREFIX = "scenario."


def parse_size(text: str) -> int | None:
    """Convert a cgroup-style size ("512M", "1G", "4096", "max") to bytes.

    ``max`` is the cgroup v2 spelling of "no limit" and is returned as None.
    """
    value = text.strip()
    if value.lower() == "max":
        return None
    match = _SIZE_RE.match(value)
    if not match:
        raise ValueError(f"not a size: {text!r} (expected e.g. 512M, 1G or max)")
    return int(match.group(1)) * _SIZE_MULT[match.group(2).upper()]


def format_size(nbytes: int | None) -> str:
    """Inverse of :func:`parse_size` for writing cgroup interface files."""
    return "max" if nbytes is None else str(int(nbytes))


def invoking_ids() -> tuple[int, int]:
    """Return (uid, gid) of the user who started the program.
```

src/config.py (242 lines) (continued)

```
    Under sudo the process runs with euid 0, but SUDO_UID/SUDO_GID record who
    asked for it. Outside sudo these are simply the real uid/gid.
    """
    uid = int(os.environ.get("SUDO_UID", os.getuid()))
    gid = int(os.environ.get("SUDO_GID", os.getgid()))
    return uid, gid


def give_to_invoking_user(path: Path) -> None:
    """chown a file we created as root back to the sudo caller (best effort).

    Evidence files (logs, CSVs) should stay readable and appendable by the
    student account after a privileged run.
    """
    if os.geteuid() != 0 or "SUDO_UID" not in os.environ:
        return
    uid, gid = invoking_ids()
    try:
        os.chown(path, uid, gid)
    except OSError:
        pass


class Config:
    """Typed, read-only view of ``preserve.conf``.

    Every getter raises ``KeyError`` naming the missing section/key, so a
    typo in the configuration fails loudly at start-up instead of silently
    falling back to a hidden default.
    """

    def __init__(self, path: str | Path | None = None, scenario: str | None = None) -> None:
        self.path = Path(path or DEFAULT_CONFIG_PATH).resolve()
        if not self.path.is_file():
            raise FileNotFoundError(f"configuration file not found: {self.path}")
        self.root = self.path.parent
        self._cp = configparser.ConfigParser(
            inline_comment_prefixes=("#",), interpolation=None
        )
        self._cp.read(self.path)
        self.scenario = scenario or self._cp.get("scenarios", "default", fallback=None)
        self.scenario_description = ""
        self.overrides: dict[tuple[str, str], str] = {}
        if self.scenario is not None:
            self._load_scenario(self.scenario)

    def _load_scenario(self, name: str) -> None:
        section = SCENARIO_PREFIX + name
        if not self._cp.has_section(section):
            raise KeyError(f"{self.path}: unknown scenario {name!r} (defined: {', '.join(self.scenarios())})")
        for dotted, value in self._cp.items(section):
            if dotted == "description":
                self.scenario_description = value.strip()
                continue
            base_section, _, key = dotted.partition(".")
            if not key or not self._cp.has_option(base_section, key):
                raise KeyError(f"{self.path}: [{section}] {dotted}: no base key [{base_section}] {key}")
            self.overrides[(base_section, key)] = value

    def scenarios(self) -> list[str]:
        return [s[len(SCENARIO_PREFIX):] for s in self._cp.sections() if s.startswith(SCENARIO_PREFIX)]

    # -- raw access ----------------------------------------------------------
    def _raw(self, section: str, key: str) -> str:
        if (section, key) in self.overrides:
            return self.overrides[(section, key)]
        try:
            return self._cp.get(section, key)
        except (configparser.NoSectionError, configparser.NoOptionError):
            raise KeyError(f"{self.path}: missing [{section}] {key}") from None

    def sections(self) -> list[str]:
        return [s for s in self._cp.sections() if not s.startswith(SCENARIO_PREFIX)]

    def items(self, section: str) -> list[tuple[str, str]]:
        return [(k, self._raw(section, k)) for k, _ in self._cp.items(section)]

    # -- typed getters -------------------------------------------------------
    def get_str(self, section: str, key: str) -> str:
        uid, _ = invoking_ids()
        return self._raw(section, key).strip().replace("{uid}", str(uid))

    def get_int(self, section: str, key: str) -> int:
```

src/config.py (242 lines) (continued)

```python
        return int(self.get_str(section, key))

    def get_float(self, section: str, key: str) -> float:
        return float(self.get_str(section, key))

    def get_bool(self, section: str, key: str) -> bool:
        value = self.get_str(section, key).lower()
        if value not in configparser.ConfigParser.BOOLEAN_STATES:
            raise ValueError(f"[{section}] {key}: not a boolean: {value!r}")
        return configparser.ConfigParser.BOOLEAN_STATES[value]

    def get_size(self, section: str, key: str) -> int | None:
        return parse_size(self.get_str(section, key))

    def get_path(self, section: str, key: str) -> Path:
        p = Path(self.get_str(section, key)).expanduser()
        return p if p.is_absolute() else (self.root / p)

    def get_args(self, section: str, key: str) -> list[str]:
        """Split a value into an argv list with shell quoting rules."""
        return shlex.split(self.get_str(section, key))


_logging_ready = False


def setup_logging(cfg: Config, to_file: bool = True) -> logging.Logger:
    """Configure the ``preserve`` logger hierarchy once per process.

    Modules obtain child loggers with ``logging.getLogger("preserve.<name>")``
    so each record shows which subsystem (sensor, capture, ...) produced it.
    """
    global _logging_ready
    root = logging.getLogger(LOGGER_NAME)
    if _logging_ready:
        return root
    root.setLevel(getattr(logging, cfg.get_str("logging", "level").upper()))
    root.propagate = False
    fmt = logging.Formatter(
        "%(asctime)s.%(msecs)03d %(levelname)-8s [%(name)s] %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    console = logging.StreamHandler(sys.stdout)
    console.setFormatter(fmt)
    root.addHandler(console)
    if to_file:
        log_path = cfg.get_path("paths", "log_file")
        log_path.parent.mkdir(parents=True, exist_ok=True)
        fh = logging.FileHandler(log_path, encoding="utf-8")
        fh.setFormatter(fmt)
        root.addHandler(fh)
        give_to_invoking_user(log_path)
    _logging_ready = True
    return root


def _main() -> int:
    ap = argparse.ArgumentParser(description="Print the parsed PRESERVE configuration.")
    ap.add_argument("--config", default=None, help="path to preserve.conf")
    ap.add_argument("--scenario", default=None, help="show effective values for this scenario")
    args = ap.parse_args()
    cfg = Config(args.config, args.scenario)
    print(f"# {cfg.path}  (project root {cfg.root})")
    print(f"# scenario {cfg.scenario}: {cfg.scenario_description}  (available: {', '.join(cfg.scenarios())})")
    for section in cfg.sections():
        print(f"[{section}]")
        for key, _ in cfg.items(section):
            mark = "  <- scenario override" if (section, key) in cfg.overrides else ""
            print(f"  {key:28s} = {cfg.get_str(section, key)}{mark}")
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

src/costmodel.py (259 lines)

```
"""
costmodel.py -- M2: restart cost vs. capture cost scoring.

The decision being modelled
---------------------------
When a memory-limited cgroup comes under pressure, the kernel's default
answer is the OOM killer: it picks a victim by ``oom_score`` (roughly "who
uses the most memory") and sends SIGKILL. All work done by that process is
lost. PRESERVE offers a second option, *eviction by checkpoint*: freeze the
process with CRIU, write its address space and kernel state to disk, kill it
to release its memory, and restore it later. Eviction is not free, so the
governor needs a cost model to decide **which** process to evict and
**whether** eviction is worth it at all.

Two costs are estimated, both in seconds so that they are comparable:

* **Restart cost R**: the work lost if the process were killed and had to
  start again. Without application cooperation the best OS-level proxy is
  consumed CPU time (``utime + stime`` from ``/proc/<pid>/stat``). A
  workload may also *declare* its progress in a state file; the larger of
  the two is used. The declared value is needed because CRIU restores a
  process's memory and registers but **not** the kernel's per-task CPU
  accounting, so a restored process reports a small ``utime`` again even
  though its in-memory work survived.

* **Capture cost C**: the time to dump the process. CRIU writes, among other
  things, every private anonymous page, so the image is roughly the
  resident set size. The model estimates::

        image_bytes = RSS * image_size_factor + image_fixed_overhead
        C           = dump_fixed_overhead_s + image_MiB / throughput_MiB_s

  ``throughput`` is refined after every real dump with an exponentially
  weighted moving average (EWMA), so the model calibrates itself to this
  machine's disk. The estimate is **persisted** to a small JSON state file
  after each dump and loaded at the next start, so a new run begins from
  what was measured before instead of from the configured guess
  (``initial_dump_mib_per_s``, used only when no saved estimate exists).
  Without this, every cold start would overestimate C and the admission
  test (``max_capture_s``) could refuse dumps that would have succeeded.

Score and eligibility
---------------------
A process is **eligible** for preservation only if

* ``R >= min_restart_cost_s`` (cheap processes are simply not worth it), and
* ``R >= benefit_ratio * C`` (saving the work must clearly beat the cost of
  saving it).

Among eligible processes the victim is the one with the highest score::

        score = w_restart * R + w_memory * RSS_MiB - w_capture * C

The memory term rewards evicting a process that frees a lot of memory, which
is the point of acting under pressure. The capture term penalises slow dumps
that would leave the system stalled for longer. Weights are in
``preserve.conf`` [costmodel].

Standalone use::

    python3 src/costmodel.py                # score a set of synthetic processes
    python3 src/costmodel.py --pid 1234     # score real processes (read-only)
"""
from __future__ import annotations

import argparse
import json
import logging
import math
import os
import sys
import time
from dataclasses import dataclass
from pathlib import Path

from config import Config
from sensor import MIB, ProcSnapshot, read_proc

log = logging.getLogger("preserve.costmodel")


@dataclass(frozen=True)
class Score:
    """Cost-model verdict for one process."""
```

```
    pid: int
    role: str
    mem_mib: float
    restart_cost_s: float
    capture_cost_s: float
    est_image_bytes: int
    score: float
    eligible: bool
    reason: str

    def describe(self) -> str:
        flag = "ELIGIBLE" if self.eligible else "skip"
        return (
            f"pid={self.pid} role={self.role} rss={self.mem_mib:.1f}MiB "
            f"R={self.restart_cost_s:.2f}s C={self.capture_cost_s:.2f}s "
            f"score={self.score:.2f} [{flag}: {self.reason}]"
        )


class CostModel:
    """Scores processes for eviction-by-checkpoint (see module docstring)."""

    def __init__(self, cfg: Config) -> None:
        s = "costmodel"
        self.w_restart = cfg.get_float(s, "weight_restart")
        self.w_memory = cfg.get_float(s, "weight_memory")
        self.w_capture = cfg.get_float(s, "weight_capture")
        self.min_restart_cost_s = cfg.get_float(s, "min_restart_cost_s")
        self.benefit_ratio = cfg.get_float(s, "benefit_ratio")
        self.throughput_mib_s = cfg.get_float(s, "initial_dump_mib_per_s")
        self.last_observed_mib_s: float | None = None
        self.observations = 0  # dumps folded into the estimate, across runs
        self.dump_fixed_overhead_s = cfg.get_float(s, "dump_fixed_overhead_s")
        self.ewma_alpha = cfg.get_float(s, "ewma_alpha")
        self.image_size_factor = cfg.get_float(s, "image_size_factor")
        self.image_fixed_overhead = int(cfg.get_float(s, "image_fixed_overhead_mib") * MIB)

    # -- cost estimates ------------------------------------------------------
    def estimate_image_bytes(self, rss_bytes: int) -> int:
        return int(rss_bytes * self.image_size_factor) + self.image_fixed_overhead

    def capture_cost_s(self, rss_bytes: int) -> float:
        image_mib = self.estimate_image_bytes(rss_bytes) / MIB
        return self.dump_fixed_overhead_s + image_mib / self.throughput_mib_s

    @staticmethod
    def restart_cost_s(snap: ProcSnapshot, declared_work_s: float | None = None) -> float:
        return max(snap.cpu_seconds, declared_work_s or 0.0)

    # -- scoring -------------------------------------------------------------
    def score(self, snap: ProcSnapshot, role: str, declared_work_s: float | None = None) -> Score:
        r = self.restart_cost_s(snap, declared_work_s)
        c = self.capture_cost_s(snap.rss_bytes)
        m = snap.rss_bytes / MIB
        value = self.w_restart * r + self.w_memory * m - self.w_capture * c
        if r < self.min_restart_cost_s:
            eligible, reason = False, f"R {r:.2f}s < min_restart_cost_s {self.min_restart_cost_s}s"
        elif r < self.benefit_ratio * c:
            eligible, reason = False, f"R {r:.2f}s < benefit_ratio*C {self.benefit_ratio * c:.2f}s"
        else:
            eligible, reason = True, f"R/C = {r / c:.1f}"
        return Score(snap.pid, role, m, r, c, self.estimate_image_bytes(snap.rss_bytes),
                     value, eligible, reason)

    def score_group(self, root_pid: int, role: str,
                    members: list[tuple[ProcSnapshot, float | None]]) -> Score:
        """Score a dependent group as one unit (Objective 6).

        Killing any member destroys every member's work, and CRIU dumps them
        together, so R is the SUM of the members' R, and RSS (memory freed) and
        C (one dump writing all of their pages) cover all members. The same
        eligibility rules and score formula as :meth:`score` then apply.
        ``pid`` of the result is the group's root (the pid CRIU is given).
        """
        r = sum(self.restart_cost_s(snap, declared) for snap, declared in members)
        rss = sum(snap.rss_bytes for snap, _ in members)
        merged = ProcSnapshot(root_pid, role, "R", 0, rss, 0.0, 0)
        s = self.score(merged, role, r)
        return s

    @staticmethod
    def rank(scores: list[Score]) -> list[Score]:
        """Eligible processes first, then by descending score."""
```

src/costmodel.py (259 lines) (continued)

```python
        return sorted(scores, key=lambda s: (s.eligible, s.score), reverse=True)

    # -- online calibration ------------------------------------------------
    def observe_capture(self, image_bytes: int, seconds: float) -> None:
        """Fold a measured dump into the throughput estimate (EWMA)."""
        if image_bytes <= 0 or seconds <= 0:
            return
        observed = (image_bytes / MIB) / seconds
        old = self.throughput_mib_s
        self.throughput_mib_s = self.ewma_alpha * observed + (1 - self.ewma_alpha) * old
        self.last_observed_mib_s = observed
        self.observations += 1
        log.info("dump throughput observed %.1f MiB/s; estimate %.1f -> %.1f MiB/s",
                 observed, old, self.throughput_mib_s)

    # -- persistence of the estimate across runs ---------------------------
    def load_throughput(self, path: Path, shown_as: str | None = None) -> str:
        """Adopt a saved estimate if ``path`` holds a valid one; return its provenance.

        Falls back to the configured ``initial_dump_mib_per_s`` (already set
        in ``__init__``) when the file is missing or unusable. ``shown_as`` is
        the name used in messages (e.g. the path relative to the project).
        """
        name = shown_as or str(path)
        try:
            data = json.loads(Path(path).read_text())
            value = float(data["throughput_mib_s"])
        except FileNotFoundError:
            return "default, no saved estimate"
        except (OSError, ValueError, KeyError, TypeError) as exc:
            log.warning("ignoring unreadable throughput state %s: %s", name, exc)
            return f"default, saved estimate in {name} unreadable"
        if not math.isfinite(value) or value <= 0:
            log.warning("ignoring invalid saved throughput %r in %s", value, name)
            return f"default, saved estimate in {name} invalid"
        self.throughput_mib_s = value
        self.observations = int(data.get("observations", 0))
        self.last_observed_mib_s = data.get("last_observed_mib_s")
        return f"loaded from {name}"

    def save_throughput(self, path: Path) -> None:
        """Write the current estimate atomically (tmp file + rename)."""
        path = Path(path)
        path.parent.mkdir(parents=True, exist_ok=True)
        tmp = path.with_suffix(path.suffix + ".tmp")
        tmp.write_text(json.dumps({
            "throughput_mib_s": round(self.throughput_mib_s, 3),
            "last_observed_mib_s": None if self.last_observed_mib_s is None
            else round(self.last_observed_mib_s, 3),
            "observations": self.observations,
            "ewma_alpha": self.ewma_alpha,
            "updated": time.strftime("%Y-%m-%d %H:%M:%S"),
        }, indent=2))
        os.replace(tmp, path)


def _main() -> int:
    ap = argparse.ArgumentParser(description="Score processes with the PRESERVE cost model (M2).")
    ap.add_argument("--config", default=None)
    ap.add_argument("--pid", type=int, action="append", default=[], help="score this real PID (read-only)")
    args = ap.parse_args()
    model = CostModel(Config(args.config))
    print(f"throughput estimate: {model.throughput_mib_s:.1f} MiB/s")

    if args.pid:
        snaps = [(read_proc(p), "observed") for p in args.pid]
        missing = [p for p, (s, _) in zip(args.pid, snaps) if s is None]
        if missing:
            print(f"no such process: {missing}")
        scores = [model.score(s, role) for s, role in snaps if s is not None]
    else:
        # Synthetic population: a long CPU-heavy job, a fresh memory hog, a tiny daemon.
        synthetic = [
            (ProcSnapshot(1001, "compute_job", "R", 1, 128 * MIB, 60.0, 0), "compute", None),
            (ProcSnapshot(1002, "hog", "R", 1, 400 * MIB, 0.8, 0), "hog", None),
            (ProcSnapshot(1003, "daemon", "S", 1, 8 * MIB, 0.1, 0), "daemon", None),
            (ProcSnapshot(1004, "restored", "R", 1, 128 * MIB, 0.5, 0), "compute", 75.0),
        ]
        scores = [model.score(s, role, declared) for s, role, declared in synthetic]

    for s in model.rank(scores):
        print("  " + s.describe())
    print("\nsimulated dump of 128 MiB in 0.8 s:")
    model.observe_capture(128 * MIB, 0.8)
```

src/costmodel.py (259 lines) (continued)

```
    print(f"throughput estimate now {model.throughput_mib_s:.1f} MiB/s")
    return 0


if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO, format="%(message)s")
    sys.exit(_main())
```

src/deps.py (225 lines)

```
"""
deps.py -- dependency detection between managed processes (Objective 6).

Why groups
----------
When processes depend on each other, evicting or killing one of them destroys
the others' work too: a consumer whose producer is gone reads EOF and stops.
So a dependent set must be scored, checkpointed and restored as **one unit**.

Evidence for an edge (read-only procfs, nothing is inferred from labels)
------------------------------------------------------------------------
* **parent/child**: field 4 (``ppid``) of ``/proc/<child>/stat`` names the
  parent.
* **shared pipe / socket**: ``readlink /proc/<pid>/fd/<n>`` gives
  ``pipe:[<inode>]`` or ``socket:[<inode>]``. Two processes that hold the
  same inode are connected through that kernel object. pipefs and sockfs
  inode numbers are unique system-wide, so equal inodes mean the same object.

Each :class:`Edge` records the proof (the ppid field or the inode and the fd
numbers on both sides) so the log shows *why* two processes were grouped.
Connected components of the edge graph (union-find) are the groups; a process
with no edges is a group of one, which is how single-process scenarios
behave exactly as before.

Capturability (CRIU dumps a whole tree from its root pid)
---------------------------------------------------------
``criu dump -t ROOT`` takes ROOT and *all* of its descendants. A group can
therefore be captured only if

1. it is exactly one subtree: one member whose parent is outside the group
   (the root), and the root's full set of live descendants, found by scanning
   every ``/proc/<pid>/stat``, equals the group (no member outside the tree,
   no unmanaged descendant that the dump would take with it); and
2. none of its pipes or sockets is also held by a process outside the group.
   Such a link could not be restored, and the outside process would see its
   partner vanish. This needs every process's fd table, so any process whose
   fds cannot be read makes isolation unprovable, and the group is refused.

Standalone use (read-only)::

    python3 src/deps.py --pid 1234 --pid 1235
"""
from __future__ import annotations

import argparse
import os
import sys
from dataclasses import dataclass, field
from pathlib import Path

LINK_KINDS = ("pipe", "socket")


@dataclass(frozen=True)
class Edge:
    a: int
    b: int
    kind: str   # "parent/child" | "pipe" | "socket"
    proof: str  # the procfs evidence

    @property
    def key(self) -> tuple:
        return (min(self.a, self.b), max(self.a, self.b), self.kind, self.proof.split(" ")[0])

    def describe(self) -> str:
        return f"{self.kind} edge {self.a} <-> {self.b}: {self.proof}"


@dataclass
class Group:
    members: list[int]
    edges: list[Edge] = field(default_factory=list)

    @property
    def size(self) -> int:
        return len(self.members)


def read_ppid(pid: int) -> int | None:
    try:
        stat = Path(f"/proc/{pid}/stat").read_text()
    except (FileNotFoundError, ProcessLookupError, PermissionError):
        return None
    fields = stat[stat.rindex(")") + 2:].split()
```

src/deps.py (225 lines) (continued)

```python
    return int(fields[1]) if fields[0] not in ("Z", "X", "x") else None


def fd_links(pid: int) -> dict[tuple[str, int], list[int]] | None:
    """{(kind, inode): [fd, ...]} for every pipe/socket ``pid`` holds; None if unreadable."""
    out: dict[tuple[str, int], list[int]] = {}
    base = f"/proc/{pid}/fd"
    try:
        names = os.listdir(base)
    except (FileNotFoundError, ProcessLookupError):
        return {}
    except PermissionError:
        return None
    for name in names:
        try:
            target = os.readlink(f"{base}/{name}")
        except OSError:
            continue
        for kind in LINK_KINDS:
            if target.startswith(f"{kind}:["):
                out.setdefault((kind, int(target[len(kind) + 2:-1])), []).append(int(name))
    return out


def all_pids() -> list[int]:
    return [int(n) for n in os.listdir("/proc") if n.isdigit()]


def find_edges(pids: list[int]) -> list[Edge]:
    """Edges among ``pids`` only (parent/child and shared pipe/socket inodes)."""
    pidset = set(pids)
    edges: list[Edge] = []
    for p in pids:
        pp = read_ppid(p)
        if pp in pidset:
            edges.append(Edge(pp, p, "parent/child", f"/proc/{p}/stat ppid={pp}"))
    holders: dict[tuple[str, int], list[tuple[int, list[int]]]] = {}
    for p in pids:
        for link, fds in (fd_links(p) or {}).items():
            holders.setdefault(link, []).append((p, fds))
    for (kind, inode), hs in sorted(holders.items()):
        for i in range(len(hs)):
            for j in range(i + 1, len(hs)):
                (pa, fa), (pb, fb) = hs[i], hs[j]
                edges.append(Edge(pa, pb, kind, f"{kind}:[{inode}] held by {pa} fd {fa} and {pb} fd {fb}"))
    return edges


def build_groups(pids: list[int]) -> list[Group]:
    """Connected components of the edge graph over ``pids`` (union-find)."""
    edges = find_edges(pids)
    parent = {p: p for p in pids}

    def root(x: int) -> int:
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for e in edges:
        parent[root(e.a)] = root(e.b)
    comps: dict[int, Group] = {}
    for p in sorted(pids):
        comps.setdefault(root(p), Group([])).members.append(p)
    for e in edges:
        comps[root(e.a)].edges.append(e)
    return list(comps.values())


def capturable(group: Group) -> tuple[bool, int | None, str]:
    """(ok, root_pid, reason): can ``criu dump -t root`` take exactly this group?"""
    members = set(group.members)
    ppids = {p: read_ppid(p) for p in members}
    dead = [p for p, pp in ppids.items() if pp is None]
    if dead:
        return False, None, f"member(s) {sorted(dead)} not alive"
    roots = [p for p, pp in ppids.items() if pp not in members]
    if len(roots) != 1:
        return False, None, (f"not one subtree: {len(roots)} members have a parent outside the group "
                             f"{sorted(roots)} (CRIU dumps one tree from one root pid)")
    root_pid = roots[0]

    # Whole-system scan: the root's real descendants, and who else holds our inodes.
    children: dict[int, list[int]] = {}
```

src/deps.py (225 lines) (continued)

```
    outside_links: dict[tuple[str, int], list[int]] = {}
    ours: dict[tuple[str, int], int] = {}
    for p in members:
        for link in (fd_links(p) or {}):
            ours[link] = p
    unreadable: list[int] = []
    for p in all_pids():
        pp = read_ppid(p)
        if pp is not None:
            children.setdefault(pp, []).append(p)
        if p in members or p == os.getpid():
            continue
        links = fd_links(p)
        if links is None:
            unreadable.append(p)
            continue
        for link in links:
            if link in ours:
                outside_links.setdefault(link, []).append(p)

    tree, todo = set(), [root_pid]
    while todo:
        p = todo.pop()
        if p not in tree:
            tree.add(p)
            todo.extend(children.get(p, []))
    if tree != members:
        extra, missing = sorted(tree - members), sorted(members - tree)
        return False, root_pid, (f"not exactly one subtree of root {root_pid}: dump would also take "
                                 f"{extra or 'nothing'}, would miss {missing or 'nothing'}")
    if outside_links:
        (kind, inode), who = next(iter(sorted(outside_links.items())))
        return False, root_pid, (f"{kind}:[{inode}] of member {ours[(kind, inode)]} is also held by "
                                 f"outside pid(s) {who}; that link could not be restored")
    if unreadable:
        return False, root_pid, (f"isolation unprovable: fd tables of {len(unreadable)} process(es) "
                                 f"unreadable (e.g. {unreadable[:5]}); not root?")
    return True, root_pid, (f"one subtree rooted at {root_pid} ({len(members)} members), no pipe/socket "
                            f"shared with any of {len(all_pids())} other processes")


def _main() -> int:
    ap = argparse.ArgumentParser(description="Show dependency edges and groups among PIDs (read-only).")
    ap.add_argument("--pid", type=int, action="append", required=True)
    args = ap.parse_args()
    for g in build_groups(args.pid):
        print(f"group {g.members}")
        for e in g.edges:
            print(f"  {e.describe()}")
        if g.size > 1:
            ok, root, why = capturable(g)
            print(f"  capturable={ok} root={root}: {why}")
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

src/matrix.py (351 lines)

```
"""
matrix.py -- summarise PRESERVE experiment runs into one results table.

Helper for ``run_all.sh``. It never runs an experiment. It only reads what a
run has already written to its evidence directory:

* ``baseline_result.json``: the structured baseline report (OOM count,
  kernel-identified victims, work destroyed);
* ``console.log``: the governor's log, including its SUMMARY block
  (captures/restores, PRESERVED/RESTORED lines with timings, continuity
  verdict, per-process exit events);
* ``compute_job.progress.log``: read through ``governor.load_reference_run``,
  the same extraction ``compare.py`` uses, to get the work held at the
  checkpoint.

The evidence directory is always the one the run itself reported. Nothing is
inferred from timestamps.

Subcommands::

    defaults      --scenario S                          KEY=VALUE lines for shell prompts
    custom-config --base S --compute N --hog N --delay S --out PATH
    row           --scenario S --mode M --rc N --batch K [--evidence DIR]  -> one JSON line
    table         --session FILE.jsonl --out FILE.txt [--repeat N]
    latest                                              print the newest evidence/matrix-*.txt
"""
from __future__ import annotations

import argparse
import json
import re
import shlex
import statistics
import sys
import time
from collections import Counter
from pathlib import Path

from config import PROJECT_ROOT, Config
from governor import BASELINE_RESULT, load_reference_run
from sensor import MIB

MODE_LABEL = {"baseline": "baseline", "live": "PRESERVE", "dry-run": "dry run"}
OK_RCS = {"baseline": {0, 2}, "live": {0}, "dry-run": {0}}
_REFUSED = {75: "REFUSED: another governor was running (lock held); no experiment ran",
            76: "REFUSED: setup aborted (no exclusive cgroup); no experiment ran"}
RC_MEANING = {
    "live": {1: "error, or compute_job restarted instead of resuming", 2: "no checkpoint happened",
             3: "no restore happened", 126: "sudo not available (live runs need root)", **_REFUSED},
    "baseline": {1: "error", **_REFUSED},
    "dry-run": {1: "error", 126: "sudo not available", **_REFUSED},
}
COLUMNS = ["Scenario", "Mode", "OOM kills", "Victim", "Work destroyed", "Work preserved", "Dump", "Restore",
           "Benefit"]

KILLED_RE = re.compile(r"\] (\w+) pid=(\d+) killed by SIGKILL(?: \(last reported iteration (\d+), work ([\d.]+)s\))?
 + ")
DUMP_RE = re.compile(r"PRESERVED dump pid=\d+ OK in ([\d.]+)s, image ([\d.]+)MiB")
RESTORE_RE = re.compile(r"RESTORED restore pid=\d+ OK in ([\d.]+)s")
SUMMARY_RE = re.compile(r"SUMMARY .*captures=(\d+)\s+restores=(\d+)")
# The verdict may contain spaces ("RAN CONTINUOUSLY", "CHECKPOINTED, NOT RESTORED"); it ends at " -- ".
CONTINUITY_RE = re.compile(r"compute_job continuity: (.+?) -- ")


# ---------------------------------------------------------------------------
# defaults / custom-config
# ---------------------------------------------------------------------------
def cmd_defaults(args: argparse.Namespace) -> int:
    cfg = Config(None, args.scenario)
    mem_max = cfg.get_size("cgroup", "memory_max")
    values = {
        "COMPUTE": cfg.get_str("workloads", "compute_working_set_mb"),
        "HOG": cfg.get_str("workloads", "hog_target_mb"),
        "DELAY": cfg.get_str("workloads", "hog_start_delay_s"),
        "MEMMAX_MIB": "0" if mem_max is None else str(mem_max // MIB),
    }
    for k, v in values.items():
        print(f"{k}={shlex.quote(v)}")
    return 0


def cmd_custom_config(args: argparse.Namespace) -> int:
    """Write a copy of preserve.conf plus a [scenario.custom] section. preserve.conf is untouched."""
    out = Path(args.out).resolve()
```

```python
    if out.parent != PROJECT_ROOT:
        print(f"custom config must live in the project root {PROJECT_ROOT} so relative paths resolve")
        return 1
    base = Config(None, args.base)
    overrides = dict(base.overrides)
    overrides[("workloads", "compute_working_set_mb")] = str(args.compute)
    overrides[("workloads", "hog_target_mb")] = str(args.hog)
    overrides[("workloads", "hog_start_delay_s")] = str(args.delay)
    overrides[("baseline", "reference_live_run")] = ""
    lines = ["", "# ---- generated by run_all.sh for one custom run; preserve.conf itself is unchanged ----",
             "[scenario.custom]",
             f"description = Custom run from run_all.sh based on scenario {args.base}: "
             f"compute_working_set_mb={args.compute} hog_target_mb={args.hog} hog_start_delay_s={args.delay}"]
    lines += [f"{s}.{k} = {v}" for (s, k), v in sorted(overrides.items())]
    out.write_text((PROJECT_ROOT / "preserve.conf").read_text() + "\n".join(lines) + "\n")
    Config(out, "custom")  # validate: every override must name a real key
    return 0


# ---------------------------------------------------------------------------
# row: extract one run's numbers from its evidence directory
# ---------------------------------------------------------------------------
def build_row(scenario: str, mode: str, rc: int, batch: int, evidence: Path | None) -> dict:
    row: dict = {"scenario": scenario, "mode": mode, "rc": rc, "batch": batch,
                 "evidence": str(evidence) if evidence else None, "failed": False, "reason": "",
                 "has_data": False, "oom_kills": None, "victim": None, "destroyed_s": None,
                 "destroyed_partial": False, "preserved_s": None, "dump_s": None, "image_mib": None,
                 "restore_s": None}
    if rc not in OK_RCS.get(mode, {0}):
        row["failed"] = True
        row["reason"] = RC_MEANING.get(mode, {}).get(rc, f"exit code {rc}")
    if evidence is None or not (evidence / "console.log").exists():
        row["failed"] = True
        row["reason"] = row["reason"] or "run reported no evidence folder"
        return row
    # A FAILED marker written by run_demo.sh is authoritative: the run did not
    # complete, so its cell must never show a stale or partial value.
    if (evidence / "FAILED").exists():
        row["failed"] = True
        marker = (evidence / "FAILED").read_text(errors="replace").splitlines()
        row["reason"] = row["reason"] or (marker[0] if marker else "run marked FAILED")
        return row
    console = (evidence / "console.log").read_text(errors="replace")
    if "SUMMARY" not in console:
        row["failed"] = True
        row["reason"] = row["reason"] or "governor did not reach its summary (crash?)"
        return row
    row["has_data"] = True

    if mode == "baseline":
        try:
            result = json.loads((evidence / BASELINE_RESULT).read_text())
        except (OSError, ValueError):
            row["failed"], row["reason"] = True, f"missing or unreadable {BASELINE_RESULT}"
            return row
        row["oom_kills"] = result.get("oom_kill", 0)
        victims = result.get("victims", [])
        row["victim"] = ", ".join(sorted({v.get("role", "?") for v in victims})) or "none"
        row["destroyed_s"] = round(sum(v.get("work_lost_s", 0.0) for v in victims), 2)
        return row

    kills = [int(x) for x in re.findall(r"oom_kill=(\d+)", console)]
    row["oom_kills"] = max(kills) if kills else 0
    victims: dict[str, tuple[str, float | None]] = {}
    for m in KILLED_RE.finditer(console):
        victims[m[2]] = (m[1], float(m[4]) if m[4] else None)  # dedupe: events repeat in SUMMARY
    row["victim"] = ", ".join(sorted({role for role, _ in victims.values()})) or "none"
    row["destroyed_s"] = round(sum(w or 0.0 for _, w in victims.values()), 2)
    row["destroyed_partial"] = any(w is None for _, w in victims.values())

    if mode == "live":
        dump, restore = DUMP_RE.search(console), RESTORE_RE.search(console)
        summary, continuity = SUMMARY_RE.search(console), CONTINUITY_RE.search(console)
        if dump:
            row["dump_s"], row["image_mib"] = float(dump[1]), float(dump[2])
        if restore:
            row["restore_s"] = float(restore[1])
        resumed = continuity is not None and continuity[1] == "RESUMED"
        if summary and int(summary[1]) > 0 and int(summary[2]) > 0 and resumed:
            ref = load_reference_run(evidence)
            row["preserved_s"] = ref.get("before_work_s")
        else:
            row["preserved_s"] = 0.0
    return row
```

```python


def cmd_row(args: argparse.Namespace) -> int:
    evidence = Path(args.evidence) if args.evidence else None
    print(json.dumps(build_row(args.scenario, args.mode, args.rc, args.batch, evidence)))
    return 0


# ---------------------------------------------------------------------------
# table
# ---------------------------------------------------------------------------
def _agg(values: list, unit: str = "s") -> str:
    vals = [v for v in values if v is not None]
    if not vals:
        return "n/a"
    fmt = (lambda x: f"{x:.2f}{unit}") if unit else (lambda x: f"{x:g}")
    if len(vals) == 1:
        return fmt(vals[0])
    return f"{fmt(statistics.median(vals))} [{fmt(min(vals))}-{fmt(max(vals))}]"


def _kills_compute(row: dict) -> bool:
    return "compute" in (row.get("victim") or "")


def benefit(first: dict, data: list[dict], session: list[dict]) -> str:
    """Did PRESERVE save work that the kernel would actually have destroyed?

    Compares a PRESERVE row with the successful baseline runs of the *same
    scenario in the same session*: work preserved only counts as a benefit if
    the baseline's OOM killer chose compute_job.
    """
    if first["mode"] != "live":
        return "n/a"
    baselines = [r for r in session if r["scenario"] == first["scenario"]
                 and r["mode"] == "baseline" and not r["failed"]]
    if not baselines:
        return "unknown (no baseline this session)"
    at_risk = [r for r in baselines if _kills_compute(r)]
    if not at_risk:
        return "none"  # the kernel never picked compute_job: the preserved work was never at risk
    saved = [r for r in data if not _kills_compute(r) and (r["preserved_s"] or 0) > 0]
    if not saved:
        return "no (compute_job lost)" if any(_kills_compute(r) for r in data) else "no (not checkpointed)"
    cell = f"yes (+{statistics.median(r['preserved_s'] for r in saved):.2f}s)"
    if len(at_risk) < len(baselines):
        cell += f" [baseline killed compute_job in {len(at_risk)}/{len(baselines)}]"
    return cell


def group_cells(rows: list[dict], session: list[dict]) -> list[str]:
    first = rows[0]
    n, failed = len(rows), [r for r in rows if r["failed"]]
    ok = [r for r in rows if not r["failed"]]
    mode = MODE_LABEL.get(first["mode"], first["mode"]) + (f" x{n}" if n > 1 else "")
    if failed:
        mode += " FAILED" if not ok else f" ({len(failed)} FAILED)"
    data = ok or [r for r in rows if r["has_data"]]
    if not data:
        return [first["scenario"], mode] + ["FAILED"] * (len(COLUMNS) - 2)

    victims = Counter(r["victim"] for r in data)
    victim, count = victims.most_common(1)[0]
    victim_cell = victim if len(data) == 1 else f"{victim} ({count}/{len(data)})"
    destroyed = _agg([r["destroyed_s"] for r in data])
    if any(r["destroyed_partial"] for r in data):
        destroyed += " +?"  # a victim declared no work (e.g. the hog)
    is_live = first["mode"] == "live"
    return [
        first["scenario"], mode,
        _agg([r["oom_kills"] for r in data], unit=""),
        victim_cell,
        destroyed,
        _agg([r["preserved_s"] for r in data]) if is_live else "n/a",
        _agg([r["dump_s"] for r in data]) if is_live else "n/a",
        _agg([r["restore_s"] for r in data]) if is_live else "n/a",
        benefit(first, data, session),
    ]


def headline(rows: list[dict]) -> str:
    def pick(mode: str, key: str) -> list[float]:
        return [r[key] for r in rows if r["scenario"] == "B" and r["mode"] == mode
                and not r["failed"] and r[key] is not None]
```

src/matrix.py (351 lines) (continued)

```
252
253     destroyed, preserved = pick("baseline", "destroyed_s"), pick("live", "preserved_s")
254     if not destroyed or not preserved:
255         missing = [name for name, v in (("Scenario B baseline", destroyed), ("Scenario B PRESERVE", preserved))
256                    if not v]
257         return f"Headline: needs a successful {' and '.join(missing)} run in this session."
258     d, p = statistics.median(destroyed), statistics.median(preserved)
259     suffix = f" (medians, n={len(destroyed)} / n={len(preserved)})" if len(destroyed) > 1 or len(preserved) > 1 else
  + ""
260     return (f"Headline (Scenario B): the kernel OOM killer destroyed {d:.2f}s of compute_job's work; "
261             f"PRESERVE checkpointed {p:.2f}s of work and resumed it with nothing lost{suffix}.")
262
263
264 def render(rows: list[dict], repeat: int, arena_lines: list[str] | None = None) -> str:
265     batches: dict[int, list[dict]] = {}
266     for r in rows:
267         batches.setdefault(r["batch"], []).append(r)
268     table = [COLUMNS] + [group_cells(b, rows) for b in batches.values()]
269     widths = [max(len(str(line[i])) for line in table) for i in range(len(COLUMNS))]
270     sep = "-+-".join("-" * w for w in widths)
271     out = [f"PRESERVE experiment matrix -- {time.strftime('%Y-%m-%d %H:%M:%S')}  (repeat={repeat})"]
272     if arena_lines:
273         out.append("Arena at batch start (reaped before each batch):")
274         out += [f"  {line}" for line in arena_lines]
275     out.append("")
276     out.append(" | ".join(str(c).ljust(w) for c, w in zip(table[0], widths)))
277     out.append(sep)
278     out += [" | ".join(str(c).ljust(w) for c, w in zip(line, widths)) for line in table[1:]]
279     out += ["", headline(rows), "",
280             "Work destroyed = accumulated work of processes the kernel killed (cost-model R at death).",
281             "Work preserved = compute_job work held at the checkpoint and resumed after restore.",
282             "Benefit        = PRESERVE row vs the same scenario's baseline in this session: yes (+Xs) if the "
283             "baseline OOM killer killed compute_job and PRESERVE kept it (X = work preserved); none if the "
284             "baseline never killed compute_job, so the preserved work was never at risk; n/a for baseline "
285             "and dry-run rows.",
286             "With repeats: median [min-max]."]
287     out += ["", "Runs and evidence:"]
288     for i, r in enumerate(rows, 1):
289         status = f"FAILED: {r['reason']}" if r["failed"] else "ok"
290         out.append(f"  {i:2d}. {r['scenario']:<16} {MODE_LABEL.get(r['mode'], r['mode']):<9} rc={r['rc']:<3} "
291                    f"{status:<12} {r['evidence'] or '-'}")
292     return "\n".join(out) + "\n"
293
294
295 def cmd_table(args: argparse.Namespace) -> int:
296     session = Path(args.session)
297     rows = [json.loads(line) for line in session.read_text().splitlines() if line.strip()] \
298         if session.exists() else []
299     if not rows:
300         print("(no runs completed in this session yet)")
301         return 0
302     arena_lines = []
303     if args.arena_log and Path(args.arena_log).exists():
304         arena_lines = [ln for ln in Path(args.arena_log).read_text().splitlines() if ln.strip()]
305     text = render(rows, args.repeat, arena_lines)
306     Path(args.out).write_text(text)
307     print(text, end="")
308     print(f"saved to {args.out}")
309     return 0
310
311
312 def cmd_latest(_args: argparse.Namespace) -> int:
313     files = sorted((PROJECT_ROOT / "evidence").glob("matrix-*.txt"), key=lambda p: p.stat().st_mtime)
314     if not files:
315         print("no saved results table yet (evidence/matrix-*.txt)")
316         return 0
317     print(f"# {files[-1]}")
318     print(files[-1].read_text(), end="")
319     return 0
320
321
322 def main() -> int:
323     ap = argparse.ArgumentParser(description="Summarise PRESERVE runs (helper for run_all.sh).")
324     sub = ap.add_subparsers(dest="cmd", required=True)
325     p = sub.add_parser("defaults")
326     p.add_argument("--scenario", default=None)
327     p = sub.add_parser("custom-config")
328     p.add_argument("--base", required=True)
329     p.add_argument("--compute", type=int, required=True)
330     p.add_argument("--hog", type=int, required=True)
331     p.add_argument("--delay", type=float, required=True)
332     p.add_argument("--out", required=True)
333     p = sub.add_parser("row")
334     p.add_argument("--scenario", required=True)
```

src/matrix.py (351 lines) (continued)

```
    p.add_argument("--mode", required=True, choices=sorted(MODE_LABEL))
    p.add_argument("--rc", type=int, required=True)
    p.add_argument("--batch", type=int, required=True)
    p.add_argument("--evidence", default=None)
    p = sub.add_parser("table")
    p.add_argument("--session", required=True)
    p.add_argument("--out", required=True)
    p.add_argument("--repeat", type=int, default=1)
    p.add_argument("--arena-log", default=None, help="file of per-batch 'ARENA:' lines for the header")
    sub.add_parser("latest")
    args = ap.parse_args()
    return {"defaults": cmd_defaults, "custom-config": cmd_custom_config, "row": cmd_row,
            "table": cmd_table, "latest": cmd_latest}[args.cmd](args)


if __name__ == "__main__":
    sys.exit(main())
```

src/runlock.py (226 lines)

```
"""
runlock.py -- at most one governor at a time, with a named holder.

Why this exists
---------------
Every governor run used the same cgroup name and the same ``run/`` directory.
Two runs started at once therefore shared one memory limit (each one's
workloads pushed the other's into the OOM killer) and deleted each other's
state files. Nothing detected it: both runs produced output that looked valid.

The fix has two halves. ``cgroup.py`` gives every run its own cgroup, and this
module makes a second governor *refuse to start* while one is running.

Mechanism
---------
An exclusive ``flock(2)`` on ``[paths] lock_file``, taken with ``LOCK_NB`` so a
second governor fails immediately instead of waiting. ``flock`` is used rather
than a PID file because the kernel releases the lock the moment its holder
dies -- normal exit, crash, SIGKILL, or the ``os._exit`` of the
``governor-dies`` fault injection -- so a dead run can never leave a stale lock
behind that blocks the next one.

While holding the lock, the governor writes a JSON description of itself
into the file (pid, uid, mode, scenario, start time, argv and, once created,
its cgroup path). That text is only *trusted* as "the current holder" when the
lock is actually held. When the lock is free, it describes the most recent run,
which is how ``run_failsafe.sh`` finds the leftover cgroup of a run that died.

The file descriptor is opened ``O_CLOEXEC`` (Python's default since PEP 446),
so workloads and CRIU, which are started as subprocesses, never inherit the
lock and cannot keep it alive after the governor has gone.

Standalone use (the shell scripts call these)::

    python3 src/runlock.py --status        # exit 0 if free, 75 if held (prints holder)
    python3 src/runlock.py --last-cgroup   # exit 0 + cgroup of the last run if free; 75 if held
    python3 src/runlock.py --hold --label "what I am" -- CMD ARGS...
                                           # hold the lock (described) while CMD runs;
                                           # exit 75 if held, else CMD's exit code
"""
from __future__ import annotations

import argparse
import fcntl
import json
import os
import subprocess
import sys
import time
from pathlib import Path

from config import Config, give_to_invoking_user

EXIT_LOCKED = 75  # EX_TEMPFAIL: another governor is running


class LockHeld(RuntimeError):
    """Another governor holds the lock."""


def read_holder(path: Path) -> dict:
    """The JSON a governor wrote into the lock file, or {} if absent/unreadable."""
    try:
        return json.loads(Path(path).read_text() or "{}")
    except (OSError, ValueError):
        return {}


def _alive(pid: object) -> str:
    if not isinstance(pid, int):
        return "unknown"
    return "alive" if Path(f"/proc/{pid}").exists() else "NOT RUNNING"


def describe(holder: dict) -> str:
    """One line naming the process a lock-holder record describes."""
    if not holder:
        return "holder unknown (the lock file has no description yet)"
    started = holder.get("started")
    when = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(started)) if started else "?"
    parts = [
        f"pid {holder.get('pid')} ({_alive(holder.get('pid'))})",
        f"uid {holder.get('uid')}",
        f"mode {holder.get('mode')}",
```

src/runlock.py (226 lines) (continued)

```
        f"scenario {holder.get('scenario')}",
        f"started {when}",
    ]
    if holder.get("cgroup"):
        parts.append(f"cgroup {holder['cgroup']}")
    if holder.get("argv"):
        parts.append("cmd: " + " ".join(holder["argv"]))
    return ", ".join(parts)


def describe_held(holder: dict) -> str:
    """Describe the holder of a lock that IS held, without trusting a stale record.

    The record is only rewritten by holders that go through RunLock. If its pid is
    not running, the real holder took the lock some other way (e.g. a bare
    flock(1)), so say exactly that rather than naming a dead process.
    """
    pid = holder.get("pid")
    if isinstance(pid, int) and not Path(f"/proc/{pid}").exists():
        return ("held by a process that did not describe itself (the recorded holder, "
                f"pid {pid}, is not running); last record: {describe(holder)}")
    return describe(holder)


def is_held(path: Path) -> bool:
    """True if some process currently holds the lock (probed without taking it)."""
    try:
        fd = os.open(path, os.O_RDONLY)
    except FileNotFoundError:
        return False
    try:
        fcntl.flock(fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
    except BlockingIOError:
        return True
    else:
        fcntl.flock(fd, fcntl.LOCK_UN)
        return False
    finally:
        os.close(fd)


class RunLock:
    """Exclusive, non-blocking, crash-safe lock held for the governor's lifetime."""

    def __init__(self, path: Path) -> None:
        self.path = Path(path)
        self.fd: int | None = None
        self.info: dict = {}

    def acquire(self, info: dict) -> None:
        """Take the lock or raise LockHeld naming the holder. Never waits."""
        self.path.parent.mkdir(parents=True, exist_ok=True)
        # O_RDWR so the holder can describe itself. A PermissionError here (e.g. a
        # root-owned lock file and an unprivileged governor) propagates: the
        # caller must refuse to start rather than run unlocked.
        fd = os.open(self.path, os.O_RDWR | os.O_CREAT | os.O_CLOEXEC, 0o666)
        try:
            fcntl.flock(fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
        except BlockingIOError:
            os.close(fd)
            raise LockHeld(f"another governor holds {self.path}: "
                           f"{describe_held(read_holder(self.path))}") from None
        self.fd = fd
        # Hand the file to the sudo caller so later unprivileged runs can open it.
        give_to_invoking_user(self.path)
        self.update(**info)

    def update(self, **fields: object) -> None:
        """Rewrite the holder description in place (only while holding the lock)."""
        if self.fd is None:
            return
        self.info.update(fields)
        data = json.dumps(self.info, indent=2).encode()
        os.ftruncate(self.fd, 0)
        os.pwrite(self.fd, data, 0)
        os.fsync(self.fd)

    def release(self) -> None:
        if self.fd is not None:
            fcntl.flock(self.fd, fcntl.LOCK_UN)
            os.close(self.fd)
            self.fd = None


```

src/runlock.py (226 lines) (continued)

```python
def holder_info(mode: str, scenario: str | None, argv: list[str]) -> dict:
    return {"pid": os.getpid(), "uid": os.getuid(), "euid": os.geteuid(),
            "mode": mode, "scenario": scenario, "started": time.time(), "argv": argv}


def _main() -> int:
    ap = argparse.ArgumentParser(description="Inspect the PRESERVE governor lock.")
    ap.add_argument("--config", default=None)
    g = ap.add_mutually_exclusive_group(required=True)
    g.add_argument("--status", action="store_true", help="exit 0 if free, 75 if held")
    g.add_argument("--last-cgroup", action="store_true",
                   help="print the cgroup recorded by the most recent run (only when the lock is free)")
    g.add_argument("--hold", action="store_true",
                   help="take the lock, describe this holder (--label), run CMD, release; "
                        "exit 75 if already held, else CMD's exit code")
    ap.add_argument("--since", type=float, default=None,
                    help="with --last-cgroup: print nothing unless that run started at/after this epoch "
                         "time (so a caller only ever sees a run it launched itself)")
    ap.add_argument("--label", default="script",
                    help="with --hold: what the holder is, shown to anyone who is refused")
    ap.add_argument("cmd", nargs=argparse.REMAINDER, help="with --hold: -- command [args...]")
    args = ap.parse_args()
    path = Config(args.config).get_path("paths", "lock_file")

    if args.hold:
        # Scripts take the lock through this wrapper rather than bare flock(1), so a
        # refused governor is told exactly what holds it (e.g. "overhead arm B run 2").
        cmd = args.cmd[1:] if args.cmd[:1] == ["--"] else args.cmd
        if not cmd:
            ap.error("--hold needs a command after --")
        lock = RunLock(path)
        try:
            lock.acquire(holder_info(args.label, None, cmd))
        except LockHeld as exc:
            print(f"HELD: {exc}", file=sys.stderr)
            return EXIT_LOCKED
        try:
            return subprocess.call(cmd)
        finally:
            lock.release()

    holder = read_holder(path)
    if is_held(path):
        print(f"HELD: {describe_held(holder)}")
        return EXIT_LOCKED
    if args.status:
        print(f"free (lock file {path}; last run: {describe(holder)})")
        return 0
    cg = holder.get("cgroup")
    if args.since is not None and float(holder.get("started") or 0) < args.since:
        cg = None  # the last run predates the caller's launch: not the caller's run
    if cg:
        print(cg)
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

src/sensor.py (387 lines)

```
"""
sensor.py -- M1: memory-pressure telemetry from PSI and cgroup v2.

What this module observes, in OS terms
--------------------------------------
**Pressure Stall Information (PSI).** Since Linux 4.20 the kernel accounts,
for each resource (cpu, io, memory), the share of wall-clock time in which
tasks were *stalled* waiting for that resource. For memory, a task is stalled
while it performs direct reclaim, waits on a refault of a recently evicted
page (thrashing), or swaps a page back in. Two lines are reported:

* ``some``: % of time at least one runnable task was stalled on memory.
  It measures added latency.
* ``full``: % of time *all* non-idle tasks were stalled at once. Nothing
  productive happened, so it measures lost throughput.

Each line carries exponentially-decaying averages over 10 s, 60 s and 300 s
windows (``avg10/avg60/avg300``, in percent) plus a monotonically increasing
``total`` stall time in microseconds. PSI is exposed system-wide in
``/proc/pressure/memory`` and per cgroup in ``<cgroup>/memory.pressure``.
The per-cgroup file is the key signal here: it shows pressure *inside the
test cgroup*, caused by its ``memory.max`` limit, even when the host as a
whole has plenty of free RAM.

**cgroup v2 memory controller** (all values in bytes):

* ``memory.current``: memory currently charged to the cgroup (anonymous
  pages, page cache, kernel memory).
* ``memory.max``: hard limit. Above it the kernel reclaims and, failing that,
  invokes the cgroup-local OOM killer.
* ``memory.high``: throttle point. Above it allocating tasks are forced into
  reclaim, which shows up as PSI stall long before an OOM kill.
* ``memory.swap.current`` / ``memory.swap.max``: swap usage and its limit.
* ``memory.events``: counters of how often the cgroup hit ``high``/``max``
  and how many ``oom`` / ``oom_kill`` events happened.
* ``memory.stat``: breakdown, of which ``anon`` (heap/stack; must be swapped
  to be reclaimed) and ``file`` (page cache; can be dropped) are recorded.

**Per-process accounting** from procfs:

* ``/proc/<pid>/stat`` field 14/15 ``utime``/``stime``: CPU time consumed,
  in clock ticks (``SC_CLK_TCK`` per second). The cost model uses it as the
  work a process would lose if it were killed.
* field 22 ``starttime``: when the process started, in ticks since boot.
  The pair (pid, starttime) uniquely names a process even if the kernel
  later *reuses* the numeric PID. The safety whitelist relies on this.
* ``/proc/<pid>/statm`` field 2: resident set size in pages.
* ``/proc/<pid>/cgroup``: on a pure cgroup-v2 system, one line ``0::<path>``
  naming the cgroup the process belongs to.

**Kernel OOM-killer records.** When the cgroup-local OOM killer fires, the
kernel prints to its ring buffer (read with ``dmesg``)::

    oom-kill:constraint=CONSTRAINT_MEMCG,...,oom_memcg=<cg>,task_memcg=<cg>,task=python3,pid=1234,uid=1000
    Memory cgroup out of memory: Killed process 1234 (python3) total-vm:..., anon-rss:351232kB, ...

``memory.events`` only *counts* kills (``oom_kill``). These lines say *which*
process the kernel chose. The victim is the task with the highest
``oom_badness`` (RSS + swap entries + page tables, adjusted by
``oom_score_adj``). Lines are timestamped in seconds since boot, so they are
filtered against ``/proc/uptime`` taken at the start of a run.

This module is strictly **read-only**: it never writes to /proc or /sys.

Standalone use::

    python3 src/sensor.py                    # one sample (system + test cgroup if present)
    python3 src/sensor.py --count 20         # 20 samples at the configured interval
    python3 src/sensor.py --cgroup /sys/fs/cgroup/some/group --pid 1234
"""
from __future__ import annotations

import argparse
import os
import re
import subprocess
import sys
import time
from dataclasses import dataclass, field
from pathlib import Path

from config import Config

CLK_TCK = os.sysconf("SC_CLK_TCK")
```

src/sensor.py (387 lines) (continued)

```python
PAGE_SIZE = os.sysconf("SC_PAGE_SIZE")
MIB = 1 << 20
CGROUP_ROOT = Path("/sys/fs/cgroup")
SYSTEM_PSI_PATH = Path("/proc/pressure/memory")


# ---------------------------------------------------------------------------
# PSI
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class PsiLine:
    """One line of a PSI file: averaged stall percentages and total stall time."""

    avg10: float
    avg60: float
    avg300: float
    total_us: int


@dataclass(frozen=True)
class Psi:
    """Parsed PSI file. ``full`` is None for resources that do not report it."""

    some: PsiLine
    full: PsiLine | None

    def get(self, metric: str, window: str) -> float:
        """Return e.g. ``get("some", "avg10")`` as a percentage."""
        line = self.some if metric == "some" else self.full
        return 0.0 if line is None else float(getattr(line, window))


ZERO_PSI = Psi(PsiLine(0.0, 0.0, 0.0, 0), PsiLine(0.0, 0.0, 0.0, 0))


def parse_psi(text: str) -> Psi:
    """Parse ``some avg10=.. avg60=.. avg300=.. total=..`` (+ optional ``full``)."""
    lines: dict[str, PsiLine] = {}
    for raw in text.splitlines():
        parts = raw.split()
        if not parts:
            continue
        kv = dict(p.split("=", 1) for p in parts[1:])
        lines[parts[0]] = PsiLine(
            float(kv["avg10"]), float(kv["avg60"]), float(kv["avg300"]), int(kv["total"])
        )
    if "some" not in lines:
        raise ValueError(f"unrecognised PSI content: {text!r}")
    return Psi(lines["some"], lines.get("full"))


def read_psi(path: Path) -> Psi:
    return parse_psi(Path(path).read_text())


# ---------------------------------------------------------------------------
# cgroup v2 memory controller
# ---------------------------------------------------------------------------
def _read_limit(path: Path) -> int | None:
    """Read a cgroup limit file; ``max`` (unlimited) becomes None."""
    text = path.read_text().strip()
    return None if text == "max" else int(text)


def _read_keyed(path: Path) -> dict[str, int]:
    """Read a flat-keyed cgroup file ("key value" per line)."""
    out: dict[str, int] = {}
    for line in path.read_text().splitlines():
        key, _, value = line.partition(" ")
        if value:
            out[key] = int(value)
    return out


@dataclass(frozen=True)
class CgroupMemory:
    """Snapshot of one cgroup's memory controller state (bytes)."""

    path: Path
    current: int
    max: int | None
    high: int | None
    swap_current: int
    swap_max: int | None
```

```python
    anon: int
    file: int
    events: dict[str, int] = field(default_factory=dict)
    pressure: Psi = ZERO_PSI

    @property
    def effective_limit(self) -> int | None:
        """The tighter of memory.high and memory.max (None if both unlimited)."""
        limits = [v for v in (self.max, self.high) if v is not None]
        return min(limits) if limits else None

    @property
    def usage_frac(self) -> float:
        """memory.current as a fraction of memory.max (0.0 when unlimited)."""
        return self.current / self.max if self.max else 0.0


def read_cgroup_memory(path: Path) -> CgroupMemory:
    """Read all memory-controller files of a cgroup directory."""
    path = Path(path)
    stat = _read_keyed(path / "memory.stat")
    swap_cur = path / "memory.swap.current"
    swap_max = path / "memory.swap.max"
    pressure_file = path / "memory.pressure"
    return CgroupMemory(
        path=path,
        current=int((path / "memory.current").read_text()),
        max=_read_limit(path / "memory.max"),
        high=_read_limit(path / "memory.high"),
        swap_current=int(swap_cur.read_text()) if swap_cur.exists() else 0,
        swap_max=_read_limit(swap_max) if swap_max.exists() else None,
        anon=stat.get("anon", 0),
        file=stat.get("file", 0),
        events=_read_keyed(path / "memory.events"),
        pressure=read_psi(pressure_file) if pressure_file.exists() else ZERO_PSI,
    )


# ---------------------------------------------------------------------------
# Per-process accounting
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class ProcSnapshot:
    """What procfs says about one process at one instant."""

    pid: int
    comm: str
    state: str  # R running, S sleeping, D uninterruptible, T stopped, Z zombie ...
    ppid: int
    rss_bytes: int
    cpu_seconds: float  # utime + stime
    start_ticks: int  # field 22: start time in clock ticks since boot

    @property
    def alive(self) -> bool:
        return self.state not in ("Z", "X", "x")


def read_proc(pid: int) -> ProcSnapshot | None:
    """Snapshot ``/proc/<pid>``; None if the process does not exist (any more)."""
    try:
        stat = Path(f"/proc/{pid}/stat").read_text()
        statm = Path(f"/proc/{pid}/statm").read_text()
    except (FileNotFoundError, ProcessLookupError, PermissionError):
        return None
    # comm is wrapped in parentheses and may itself contain spaces or ')',
    # so split on the *last* ')'. After it, fields[0] is stat field 3.
    rparen = stat.rindex(")")
    comm = stat[stat.index("(") + 1 : rparen]
    fields = stat[rparen + 2 :].split()
    utime, stime = int(fields[11]), int(fields[12])  # fields 14, 15
    return ProcSnapshot(
        pid=pid,
        comm=comm,
        state=fields[0],  # field 3
        ppid=int(fields[1]),  # field 4
        rss_bytes=int(statm.split()[1]) * PAGE_SIZE,
        cpu_seconds=(utime + stime) / CLK_TCK,
        start_ticks=int(fields[19]),  # field 22
    )


def proc_cgroup(pid: int) -> str | None:
    """Return the cgroup-v2 path of ``pid`` (e.g. ``/user.slice/...``), or None."""
```

```
    try:
        text = Path(f"/proc/{pid}/cgroup").read_text()
    except (FileNotFoundError, ProcessLookupError, PermissionError):
        return None
    for line in text.splitlines():
        if line.startswith("0::"):
            return line[3:].strip()
    return None


# ---------------------------------------------------------------------------
# Kernel OOM-killer records
# ---------------------------------------------------------------------------
_DMESG_RE = re.compile(r"^\[\s*(\d+\.\d+)\]\s?(.*)$")
_OOM_CTX_RE = re.compile(r"oom-kill:.*?oom_memcg=([^,]+),task_memcg=([^,]+),task=([^,]+),pid=(\d+)")
_KILLED_RE = re.compile(r"Killed process (\d+) \(([^)]*)\).*?anon-rss:(\d+)kB")


@dataclass(frozen=True)
class KernelOomKill:
    """One OOM-killer victim as reported by the kernel."""

    uptime: float  # seconds since boot when the kill was logged
    pid: int
    comm: str
    anon_rss_kib: int
    oom_memcg: str | None  # cgroup whose limit was exceeded
    line: str


def read_uptime() -> float:
    """Seconds since boot (first field of /proc/uptime); same clock as dmesg."""
    return float(Path("/proc/uptime").read_text().split()[0])


def read_kernel_oom_kills(since_uptime: float) -> list[KernelOomKill] | None:
    """OOM-killer victims logged since ``since_uptime``; None if dmesg is unreadable."""
    try:
        p = subprocess.run(["dmesg"], stdin=subprocess.DEVNULL, capture_output=True, text=True, timeout=10)
    except (OSError, subprocess.TimeoutExpired):
        return None
    if p.returncode != 0:
        return None
    memcg_of: dict[int, str] = {}
    kills: list[KernelOomKill] = []
    for raw in p.stdout.splitlines():
        m = _DMESG_RE.match(raw)
        if not m or float(m.group(1)) < since_uptime:
            continue
        ts, msg = float(m.group(1)), m.group(2)
        ctx = _OOM_CTX_RE.search(msg)
        if ctx:
            memcg_of[int(ctx.group(4))] = ctx.group(1)
            continue
        k = _KILLED_RE.search(msg)
        if k:
            pid = int(k.group(1))
            kills.append(KernelOomKill(ts, pid, k.group(2), int(k.group(3)), memcg_of.get(pid), raw.strip()))
    return kills


# ---------------------------------------------------------------------------
# Sampler
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class Sample:
    """One telemetry sample: system PSI plus (optionally) the test cgroup."""

    monotonic: float
    wall: float
    system_psi: Psi
    cgroup: CgroupMemory | None


class Sensor:
    """Periodic sampler of system-wide and per-cgroup memory pressure."""

    def __init__(self, cgroup_path: Path | None = None) -> None:
        self.cgroup_path = Path(cgroup_path) if cgroup_path else None

    def sample(self) -> Sample:
        cg = None
        if self.cgroup_path is not None and (self.cgroup_path / "memory.current").exists():
            cg = read_cgroup_memory(self.cgroup_path)
```

src/sensor.py (387 lines) (continued)

```
        system = read_psi(SYSTEM_PSI_PATH) if SYSTEM_PSI_PATH.exists() else ZERO_PSI
        return Sample(time.monotonic(), time.time(), system, cg)

    @staticmethod
    def proc(pid: int) -> ProcSnapshot | None:
        return read_proc(pid)


def format_sample(s: Sample) -> str:
    """One human-readable line summarising a sample."""
    out = f"sys.some.avg10={s.system_psi.get('some', 'avg10'):5.2f}%"
    if s.cgroup is not None:
        c = s.cgroup
        lim = "max" if c.max is None else f"{c.max / MIB:.0f}M"
        out += (
            f" | cg.some.avg10={c.pressure.get('some', 'avg10'):5.2f}%"
            f" cg.full.avg10={c.pressure.get('full', 'avg10'):5.2f}%"
            f" cur={c.current / MIB:7.1f}M/{lim} ({c.usage_frac * 100:4.1f}%)"
            f" swap={c.swap_current / MIB:6.1f}M"
            f" oom_kill={c.events.get('oom_kill', 0)}"
        )
    return out


def _main() -> int:
    ap = argparse.ArgumentParser(description="Read-only PSI / cgroup memory sampler (M1).")
    ap.add_argument("--config", default=None)
    ap.add_argument("--cgroup", default=None, help="cgroup directory (default: test cgroup from config)")
    ap.add_argument("--pid", type=int, action="append", default=[], help="also snapshot this PID")
    ap.add_argument("--count", type=int, default=1, help="number of samples")
    args = ap.parse_args()

    cfg = Config(args.config)
    interval = cfg.get_float("sensor", "sample_interval_s")
    cg_path = Path(args.cgroup) if args.cgroup else (
        Path(cfg.get_str("cgroup", "parent")) / cfg.get_str("cgroup", "name")
    )
    if not (cg_path / "memory.current").exists():
        print(f"(cgroup {cg_path} does not exist; system PSI only)")
    sensor = Sensor(cg_path)
    for i in range(args.count):
        print(f"{time.strftime('%H:%M:%S')} {format_sample(sensor.sample())}")
        for pid in args.pid:
            print(f"    {read_proc(pid)}  cgroup={proc_cgroup(pid)}")
        if i + 1 < args.count:
            time.sleep(interval)
    return 0


if __name__ == "__main__":
    sys.exit(_main())
```

workloads/compute_job.py (188 lines)

```
"""
compute_job.py -- a long-running, CPU-bound job whose progress is measurable.

Purpose in the experiment
-------------------------
This is the process PRESERVE is supposed to *save*. It stands for any job
whose value lies in accumulated in-memory state (a simulation, a training
loop, a long build): killing it means redoing all the work.

Each iteration extends a SHA-256 hash chain::

        d_0 = SHA256(seed)
        d_k = SHA256^h(d_{k-1})        (h = hashes per iteration)

The chain has two useful properties:

* **Progress is unambiguous.** Iteration ``k`` can only be reached by
  computing iterations ``1..k-1`` first, so "the job resumed at iteration
  N+1" is a strong claim.
* **State is verifiable.** The digest at iteration ``k`` is a pure function
  of ``k``. ``--verify-log`` recomputes the chain and checks every logged
  digest. That proves the restored process continued from *correct* memory
  contents, not just from the right counter value.

Memory footprint
----------------
The job also holds a working set of ``--working-set-mb`` MiB (a
``bytearray``). Every ``--touch-every`` iterations it writes one byte per page
so all pages stay dirty and resident. This makes the job a realistic eviction
target: dumping it frees that much memory inside the cgroup.

CRIU-friendliness
-----------------
* stdin/stdout/stderr are expected to be ``/dev/null``, which the governor
  arranges.
* The state file is written atomically (write ``.tmp``, then
  ``os.replace``), and the progress log is opened, appended and closed each
  time. No file descriptor stays open across the sleep in which a dump is
  most likely to happen, so there is nothing fragile to restore.
* ``work_seconds`` is accumulated in a Python variable, not read from
  ``time.process_time()``. CRIU restores memory but not the kernel's
  per-task CPU accounting, so the process CPU clock restarts near zero after
  a restore, while a variable in memory survives.

Outputs (paths given on the command line)
-----------------------------------------
* state file (JSON): ``pid, iteration, work_seconds, digest, updated_wall``
* progress log, one line per report::

      <unix_time> iteration=<k> work_s=<s> digest=<hex16> pid=<pid>

Standalone use::

    python3 workloads/compute_job.py --iterations 50 --state-file /tmp/cj.json --progress-log /tmp/cj.log
    python3 workloads/compute_job.py --verify-log /tmp/cj.log
"""
from __future__ import annotations

import argparse
import configparser
import hashlib
import json
import mmap
import os
import re
import signal
import sys
import time
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent
PAGE = mmap.PAGESIZE
SEED = b"PRESERVE compute_job seed v1"
LINE_RE = re.compile(r"iteration=(\d+) .*digest=([0-9a-f]+)")

_stop = False


def _on_term(signum, _frame) -> None:
    global _stop
    _stop = True


def conf_defaults(path: Path) -> dict[str, str]:
```

workloads/compute_job.py (188 lines) (continued)

```
    """Read [workloads] from preserve.conf so standalone runs need no flags."""
    cp = configparser.ConfigParser(inline_comment_prefixes=("#",), interpolation=None)
    cp.read(path)
    return dict(cp["workloads"]) if cp.has_section("workloads") else {}


def advance(digest: bytes, hashes: int) -> bytes:
    for _ in range(hashes):
        digest = hashlib.sha256(digest).digest()
    return digest


def touch(buf: bytearray, value: int) -> None:
    """Write one byte in every page so the whole buffer is dirty and resident."""
    n = len(range(0, len(buf), PAGE))
    buf[::PAGE] = bytes([value & 0xFF or 1]) * n


def write_state(path: Path, state: dict) -> None:
    tmp = path.with_suffix(path.suffix + ".tmp")
    tmp.write_text(json.dumps(state))
    os.replace(tmp, path)  # atomic rename: readers never see a half-written file


def run(args: argparse.Namespace) -> int:
    signal.signal(signal.SIGTERM, _on_term)
    signal.signal(signal.SIGINT, _on_term)
    state_file, progress = Path(args.state_file), Path(args.progress_log)
    working_set = bytearray(args.working_set_mb << 20)
    touch(working_set, 1)

    digest = hashlib.sha256(SEED).digest()
    iteration, work_s = 0, 0.0
    max_credit = args.max_iteration_credit_s

    while not _stop and (args.iterations == 0 or iteration < args.iterations):
        t0 = time.perf_counter()
        digest = advance(digest, args.hashes_per_iteration)
        iteration += 1
        if iteration % args.touch_every == 0:
            touch(working_set, iteration)
        dt = time.perf_counter() - t0
        # An iteration that spans a checkpoint would count the downtime as work.
        if 0 <= dt <= max_credit:
            work_s += dt
        if iteration % args.report_every == 0:
            now = time.time()
            write_state(state_file, {"pid": os.getpid(), "iteration": iteration,
                                     "work_seconds": round(work_s, 4), "digest": digest.hex(),
                                     "updated_wall": now})
            with open(progress, "a") as f:
                f.write(f"{now:.3f} iteration={iteration} work_s={work_s:.3f} "
                        f"digest={digest.hex()[:16]} pid={os.getpid()}\n")
        time.sleep(args.step_sleep_s)

    write_state(state_file, {"pid": os.getpid(), "iteration": iteration,
                             "work_seconds": round(work_s, 4), "digest": digest.hex(),
                             "updated_wall": time.time(), "final": True})
    return 0


def verify_log(path: Path, hashes: int) -> int:
    """Recompute the hash chain and check every digest recorded in a progress log."""
    entries = [(int(m.group(1)), m.group(2)) for m in map(LINE_RE.search, path.read_text().splitlines()) if m]
    if not entries:
        print(f"no progress lines in {path}")
        return 1
    digest, at, bad = hashlib.sha256(SEED).digest(), 0, 0
    resets = 0
    for i, (iteration, logged) in enumerate(entries):
        if iteration <= at:  # counter went backwards: job restarted from scratch
            resets += 1
            digest, at = hashlib.sha256(SEED).digest(), 0
        digest = advance(digest, hashes * (iteration - at))
        at = iteration
        if not digest.hex().startswith(logged):
            bad += 1
            print(f"MISMATCH line {i + 1}: iteration {iteration} logged {logged}, expected {digest.hex()[:16]}")
    print(f"verified {len(entries)} lines up to iteration {at}: {bad} mismatches, {resets} restart(s) from scratch")
    return 0 if bad == 0 else 1


def main() -> int:
    d = conf_defaults(PROJECT_ROOT / "preserve.conf")
```

workloads/compute_job.py (188 lines) (continued)

```
    ap = argparse.ArgumentParser(description="Hash-chain compute job with a resident working set.")
    ap.add_argument("--state-file", default="compute_job.state.json")
    ap.add_argument("--progress-log", default="compute_job.progress.log")
    ap.add_argument("--working-set-mb", type=int, default=int(d.get("compute_working_set_mb", 0)))
    ap.add_argument("--hashes-per-iteration", type=int, default=int(d.get("compute_hashes_per_iteration", 1)))
    ap.add_argument("--step-sleep-s", type=float, default=float(d.get("compute_step_sleep_s", 0)))
    ap.add_argument("--report-every", type=int, default=int(d.get("compute_report_every", 1)))
    ap.add_argument("--touch-every", type=int, default=int(d.get("compute_touch_every", 1)))
    ap.add_argument("--max-iteration-credit-s", type=float,
                    default=float(d.get("compute_max_iteration_credit_s", 1)))
    ap.add_argument("--iterations", type=int, default=0, help="stop after N iterations (0 = run until SIGTERM)")
    ap.add_argument("--verify-log", default=None, help="verify a progress log instead of running")
    args = ap.parse_args()
    if args.verify_log:
        return verify_log(Path(args.verify_log), args.hashes_per_iteration)
    return run(args)


if __name__ == "__main__":
    sys.exit(main())
```

workloads/hog.py (129 lines)

```
"""
hog.py -- allocates memory steadily to drive the test cgroup into pressure.

Purpose in the experiment
-------------------------
The hog is the *antagonist*. Every ``--interval-s`` seconds it allocates
another ``--chunk-mb`` MiB until it holds ``--target-mb``. It then keeps the
memory for ``--hold-s`` seconds and exits, which lets pressure recover so the
governor can demonstrate restore.

How it produces pressure (and why the pages must be touched)
------------------------------------------------------------
``bytearray(n)`` asks the allocator for zeroed memory. For large sizes that
becomes an anonymous ``mmap`` whose pages are *not yet backed by RAM*. Linux
allocates a physical page only on the first write (demand paging). Untouched,
a 400 MiB bytearray would barely move ``memory.current``. The hog therefore
writes one byte per page, forcing a page fault and a charged page for each.

Once the cgroup approaches ``memory.max`` (or ``memory.high``), each new
page fault must first reclaim memory: drop page cache, or swap out anonymous
pages up to ``memory.swap.max``. Tasks waiting in reclaim are counted as
*stalled* by PSI, which is the signal the governor watches. While holding,
the hog re-touches its pages (``--retouch``), so pages that were swapped out
fault back in. That sustains the stall instead of letting it decay.

If the governor does nothing, or decides nothing is worth preserving, the
cgroup's OOM killer eventually picks a victim. That baseline is exactly what
PRESERVE is compared against.

The hog never touches the CPU-bound job and has no knowledge of the governor.

Outputs
-------
State file (JSON): ``pid, allocated_mib, phase`` with phase in
``allocating | holding | done``.

Standalone use (it is safe *inside a cgroup*; outside one it really does
allocate the target)::

    python3 workloads/hog.py --target-mb 64 --hold-s 2 --state-file /tmp/hog.json
"""
from __future__ import annotations

import argparse
import configparser
import json
import mmap
import os
import signal
import sys
import time
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent
PAGE = mmap.PAGESIZE

_stop = False


def _on_term(signum, _frame) -> None:
    global _stop
    _stop = True


def conf_defaults(path: Path) -> dict[str, str]:
    cp = configparser.ConfigParser(inline_comment_prefixes=("#",), interpolation=None)
    cp.read(path)
    return dict(cp["workloads"]) if cp.has_section("workloads") else {}


def touch(buf: bytearray, value: int) -> None:
    n = len(range(0, len(buf), PAGE))
    buf[::PAGE] = bytes([value & 0xFF or 1]) * n


def write_state(path: Path, state: dict) -> None:
    tmp = path.with_suffix(path.suffix + ".tmp")
    tmp.write_text(json.dumps(state))
    os.replace(tmp, path)


def main() -> int:
    d = conf_defaults(PROJECT_ROOT / "preserve.conf")
    ap = argparse.ArgumentParser(description="Steady memory allocator for inducing cgroup pressure.")
```

workloads/hog.py (129 lines) (continued)

```
    ap.add_argument("--chunk-mb", type=int, default=int(d.get("hog_chunk_mb", 1)))
    ap.add_argument("--interval-s", type=float, default=float(d.get("hog_interval_s", 1)))
    ap.add_argument("--target-mb", type=int, default=int(d.get("hog_target_mb", 1)))
    ap.add_argument("--hold-s", type=float, default=float(d.get("hog_hold_s", 0)),
                    help="seconds to hold the memory after reaching target (0 = until SIGTERM)")
    ap.add_argument("--retouch-interval-s", type=float, default=float(d.get("hog_retouch_interval_s", 0)),
                    help="while holding, re-dirty all pages this often (0 = never)")
    ap.add_argument("--state-file", default="hog.state.json")
    args = ap.parse_args()

    signal.signal(signal.SIGTERM, _on_term)
    signal.signal(signal.SIGINT, _on_term)
    state = Path(args.state_file)
    chunks: list[bytearray] = []
    allocated = 0

    def report(phase: str) -> None:
        write_state(state, {"pid": os.getpid(), "allocated_mib": allocated, "phase": phase,
                            "updated_wall": time.time()})

    while not _stop and allocated < args.target_mb:
        size = min(args.chunk_mb, args.target_mb - allocated)
        buf = bytearray(size << 20)
        touch(buf, len(chunks) + 1)  # fault every page in: this is what charges the cgroup
        chunks.append(buf)
        allocated += size
        report("allocating")
        time.sleep(args.interval_s)

    report("holding")
    end = time.monotonic() + args.hold_s
    last_touch = time.monotonic()
    while not _stop and (args.hold_s == 0 or time.monotonic() < end):
        if args.retouch_interval_s > 0 and time.monotonic() - last_touch >= args.retouch_interval_s:
            for i, buf in enumerate(chunks):
                touch(buf, i + 2)  # swapped-out pages fault back in -> sustained pressure
            last_touch = time.monotonic()
        time.sleep(args.interval_s)

    report("done")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

workloads/pipeline_job.py (206 lines)

```
"""
pipeline_job.py -- two dependent processes: a parent and one forked child joined by a pipe.

Purpose in the experiment (Objective 6)
---------------------------------------
compute_job is one process. Real jobs are often several cooperating processes
(a producer and a consumer, a driver and a worker), and they depend on each
other: if the kernel OOM-kills one, the others lose their work as well. This
workload is the smallest such group.

* The parent creates a pipe and forks one child. The parent keeps the read
  end, the child keeps the write end; the unused ends are closed, so both
  processes hold the same ``pipe:[inode]`` in ``/proc/<pid>/fd``. That shared
  inode, plus the child's ``ppid``, is the evidence the governor uses to
  detect the dependency.
* Each process extends its own SHA-256 hash chain exactly like compute_job
  (``d_k = SHA256^h(d_{k-1})``), but from its own seed, and holds its own
  working set (``--parent-mb`` / ``--child-mb``).
* Every report the child writes one line ``<iteration> <digest>`` into the
  pipe; the parent drains the pipe each iteration.
* **The dependency is real.** If the child dies, the parent reads EOF and
  exits with ``EXIT_PARTNER_LOST`` without saving anything. If the parent dies,
  the child's next write fails with EPIPE and it exits the same way. Killing
  either member destroys the work of both.

Outputs (in the working directory, or under ``--run-dir``)
----------------------------------------------------------
* ``pipeline_job.<role>.state.json``  ``pid, role, iteration, work_seconds, digest, ...``
* ``pipeline_job.<role>.progress.log`` one line per report, same format as compute_job::

      <unix_time> iteration=<k> work_s=<s> digest=<hex16> pid=<pid>

CRIU-friendliness
-----------------
Same rules as compute_job (stdio on /dev/null, state written atomically,
progress log opened and closed per report). The pipe is the only long-lived
descriptor and both of its ends are inside the dumped tree, which CRIU can
restore. ``work_seconds`` lives in memory, so it survives a restore.

Standalone use::

    python3 workloads/pipeline_job.py --iterations 40 --run-dir /tmp/pj
    python3 workloads/pipeline_job.py --verify-log /tmp/pj/pipeline_job.child.progress.log --role child
"""
from __future__ import annotations

import argparse
import hashlib
import os
import re
import select
import signal
import sys
import time
from pathlib import Path

from compute_job import PROJECT_ROOT, advance, conf_defaults, touch, write_state

SEEDS = {"parent": b"PRESERVE pipeline_job parent seed v1",
         "child": b"PRESERVE pipeline_job child seed v1"}
ROLES = tuple(SEEDS)
EXIT_PARTNER_LOST = 3  # the other member died: this member's work is lost too
LINE_RE = re.compile(r"iteration=(\d+) .*digest=([0-9a-f]+)")

_stop = False


def _on_term(signum, _frame) -> None:
    global _stop
    _stop = True


def files(run_dir: Path, role: str) -> tuple[Path, Path]:
    return (run_dir / f"pipeline_job.{role}.state.json",
            run_dir / f"pipeline_job.{role}.progress.log")


def member(role: str, args: argparse.Namespace, rfd: int | None, wfd: int | None) -> int:
    """One member's main loop. ``rfd`` (parent) or ``wfd`` (child) is the shared pipe."""
    signal.signal(signal.SIGTERM, _on_term)
    signal.signal(signal.SIGINT, _on_term)
    state_file, progress = files(Path(args.run_dir), role)
    working_set = bytearray((args.parent_mb if role == "parent" else args.child_mb) << 20)
    touch(working_set, 1)
```

```python

    digest = hashlib.sha256(SEEDS[role]).digest()
    iteration, work_s, received = 0, 0.0, 0
    max_credit = args.max_iteration_credit_s
    partner_lost = ""

    while not _stop and (args.iterations == 0 or iteration < args.iterations):
        t0 = time.perf_counter()
        digest = advance(digest, args.hashes_per_iteration)
        iteration += 1
        if iteration % args.touch_every == 0:
            touch(working_set, iteration)
        dt = time.perf_counter() - t0
        if 0 <= dt <= max_credit:  # an iteration spanning a checkpoint is not work
            work_s += dt
        if rfd is not None:  # parent: drain whatever the child sent
            while select.select([rfd], [], [], 0)[0]:
                chunk = os.read(rfd, 65536)
                if not chunk:
                    partner_lost = "pipe EOF: child is gone"
                    break
                received += chunk.count(b"\n")
        if iteration % args.report_every == 0 and not partner_lost:
            if wfd is not None:  # child: tell the parent how far it is
                try:
                    os.write(wfd, f"{iteration} {digest.hex()[:16]}\n".encode())
                except BrokenPipeError:
                    partner_lost = "EPIPE: parent is gone"
            now = time.time()
            write_state(state_file, {"pid": os.getpid(), "role": role, "iteration": iteration,
                                     "work_seconds": round(work_s, 4), "digest": digest.hex(),
                                     "received": received, "updated_wall": now})
            with open(progress, "a") as f:
                f.write(f"{now:.3f} iteration={iteration} work_s={work_s:.3f} "
                        f"digest={digest.hex()[:16]} pid={os.getpid()}\n")
        if partner_lost:
            # The dependency: without its partner this member's work is worthless.
            write_state(state_file, {"pid": os.getpid(), "role": role, "iteration": iteration,
                                     "work_seconds": round(work_s, 4), "digest": digest.hex(),
                                     "received": received, "updated_wall": time.time(),
                                     "partner_lost": partner_lost})
            return EXIT_PARTNER_LOST
        time.sleep(args.step_sleep_s)

    write_state(state_file, {"pid": os.getpid(), "role": role, "iteration": iteration,
                             "work_seconds": round(work_s, 4), "digest": digest.hex(),
                             "received": received, "updated_wall": time.time(), "final": True})
    return 0


def run(args: argparse.Namespace) -> int:
    signal.signal(signal.SIGPIPE, signal.SIG_IGN)  # EPIPE as an exception, not a silent death
    rfd, wfd = os.pipe()
    pid = os.fork()
    if pid == 0:
        os.close(rfd)
        rc = 1
        try:
            rc = member("child", args, None, wfd)
        finally:
            os._exit(rc)
    os.close(wfd)
    rc = member("parent", args, rfd, None)
    if rc == 0:  # clean stop: let the child stop too, then reap it
        try:
            os.kill(pid, signal.SIGTERM)
        except ProcessLookupError:
            pass
    try:
        os.waitpid(pid, 0)
    except ChildProcessError:
        pass  # after a CRIU restore the child may already have been reaped elsewhere
    return rc


def verify_log(path: Path, role: str, hashes: int) -> int:
    """Recompute ``role``'s hash chain and check every digest in its progress log."""
    seed = SEEDS[role]
    entries = [(int(m.group(1)), m.group(2)) for m in map(LINE_RE.search, path.read_text().splitlines()) if m]
    if not entries:
        print(f"no progress lines in {path}")
        return 1
    digest, at, bad, resets = hashlib.sha256(seed).digest(), 0, 0, 0
    for i, (iteration, logged) in enumerate(entries):
```

```
        if iteration <= at:  # counter went backwards: restarted from scratch
            resets += 1
            digest, at = hashlib.sha256(seed).digest(), 0
        digest = advance(digest, hashes * (iteration - at))
        at = iteration
        if not digest.hex().startswith(logged):
            bad += 1
            print(f"MISMATCH line {i + 1}: iteration {iteration} logged {logged}, expected {digest.hex()[:16]}")
    print(f"[{role}] verified {len(entries)} lines up to iteration {at}: {bad} mismatches, "
          f"{resets} restart(s) from scratch")
    return 0 if bad == 0 and resets == 0 else 1


def main() -> int:
    d = conf_defaults(PROJECT_ROOT / "preserve.conf")
    ap = argparse.ArgumentParser(description="Parent + forked child joined by a pipe, each with a hash chain.")
    ap.add_argument("--run-dir", default=".")
    ap.add_argument("--parent-mb", type=int, default=int(d.get("pipeline_parent_mb", 0)))
    ap.add_argument("--child-mb", type=int, default=int(d.get("pipeline_child_mb", 0)))
    ap.add_argument("--hashes-per-iteration", type=int, default=int(d.get("compute_hashes_per_iteration", 1)))
    ap.add_argument("--step-sleep-s", type=float, default=float(d.get("compute_step_sleep_s", 0)))
    ap.add_argument("--report-every", type=int, default=int(d.get("compute_report_every", 1)))
    ap.add_argument("--touch-every", type=int, default=int(d.get("compute_touch_every", 1)))
    ap.add_argument("--max-iteration-credit-s", type=float,
                    default=float(d.get("compute_max_iteration_credit_s", 1)))
    ap.add_argument("--iterations", type=int, default=0, help="stop after N iterations (0 = run until SIGTERM)")
    ap.add_argument("--verify-log", default=None, help="verify a progress log instead of running")
    ap.add_argument("--role", choices=ROLES, default=None, help="whose chain --verify-log checks")
    args = ap.parse_args()
    if args.verify_log:
        if args.role is None:
            ap.error("--verify-log needs --role parent|child")
        return verify_log(Path(args.verify_log), args.role, args.hashes_per_iteration)
    return run(args)


if __name__ == "__main__":
    sys.exit(main())
```

preserve.conf (159 lines)

```
# ====================================================================
# preserve.conf -- every tunable of the PRESERVE governor.
# The code contains no magic numbers: change behaviour here.
#
# Conventions
#   * Units are in the key name: _s seconds, _mib / _mb MiB, _pct percent,
#     _frac fraction 0..1.
#   * cgroup sizes accept K/M/G suffixes or the literal "max" (no limit).
#   * Relative paths are relative to this file's directory.
#   * {uid} expands to the invoking user's uid (SUDO_UID under sudo).
#   * Named scenarios at the end of this file override individual keys
#     ("<section>.<key> = value"); select one with --scenario <name>.
# ====================================================================

[paths]
log_file      = evidence/governor.log     # timestamped log (also echoed to stdout)
telemetry_csv = evidence/telemetry.csv    # one row per sample, for plots
arena_dir     = arena                     # CRIU images: native ext4 only, never /mnt/c or tmpfs
run_dir       = run                       # workload state / progress files
throughput_state = run/throughput.json    # persisted dump-throughput EWMA; --reset-throughput deletes it
lock_file     = run/governor.lock         # flock: one governor at a time; holds a description of its holder

[logging]
level = INFO                              # DEBUG also prints every CRIU command line

[cgroup]
# Leaf cgroup is created inside systemd's per-user delegated subtree.
# The parent is never modified; it must already delegate "memory".
parent          = /sys/fs/cgroup/user.slice/user-{uid}.slice/user@{uid}.service
name            = preserve-test           # PREFIX: each run creates <name>-<pid>-<timestamp>, never shared
memory_max      = 512M                    # hard limit -> cgroup-local OOM killer above it
memory_swap_max = 256M                    # swap the group may use; 0 = none
baseline_memory_swap_max = 0              # --baseline only: no swap, so the cgroup OOM killer must act
memory_high     = max                     # throttle point; "max" = disabled
remove_on_exit  = yes                     # rmdir the cgroup at shutdown (only if empty)

[psi]
source               = cgroup             # cgroup = <cg>/memory.pressure, system = /proc/pressure/memory
metric               = some               # some = latency added, full = throughput lost
window               = avg10              # avg10 | avg60 | avg300
advisory_pct         = 10.0               # >= : log and score, no action
action_pct           = 25.0               # >= : checkpoint the best candidate
recovery_pct         = 5.0                # <= : counts toward recovery (must be < advisory_pct)
recovery_hold_samples = 5                 # consecutive calm samples before a restore (hysteresis)

[sensor]
sample_interval_s   = 0.5                 # control-loop period
log_every_n_samples = 4                   # periodic telemetry line in the log

[governor]
usage_advisory_frac     = 0.75            # memory.current / memory.max thresholds,
usage_action_frac       = 0.90            # combined with PSI (either one triggers)
usage_recovery_frac     = 0.60
capture_cooldown_s      = 5.0             # minimum gap between two checkpoints
decision_log_interval_s = 2.0             # rate limit for scoring/admission log lines
restore_max_attempts    = 3               # per checkpointed process
terminate_grace_s       = 3.0             # SIGTERM -> wait -> SIGKILL at shutdown
demo_duration_s         = 180             # hard stop for the demo
demo_post_restore_s     = 20              # keep observing the restored job this long
dry_run                 = no              # yes = never invoke CRIU (same as --dry-run)

[overhead]
# --measure-overhead: what the governor costs while doing nothing (the normal case).
duration_s = 120                          # measurement window per run
repeat     = 3                            # runs per arm in the A/B throughput comparison

[baseline]
# Control experiment (--baseline): governor observes only, kernel OOM killer acts.
post_oom_observe_s = 15                   # keep observing this long after the first OOM kill
reference_live_run = evidence/GOLD-live-run   # saved live run for the side-by-side summary (read-only); empty = non
e

[costmodel]
# score = weight_restart*R + weight_memory*RSS_MiB - weight_capture*C
weight_restart           = 1.0            # per second of work that would be lost
weight_memory            = 0.02           # per MiB freed by eviction
weight_capture           = 2.0            # per second the dump is estimated to take
min_restart_cost_s       = 5.0            # R below this: not worth preserving
benefit_ratio            = 2.0            # require R >= benefit_ratio * C
initial_dump_mib_per_s   = 200.0          # starting guess, refined by EWMA after each dump
dump_fixed_overhead_s    = 0.5            # freeze + parasite + metadata, independent of size
ewma_alpha               = 0.3            # weight of the newest throughput observation
image_size_factor        = 1.05           # image bytes per resident byte
image_fixed_overhead_mib = 2.0            # metadata images (fds, mm, core, ...)
```

preserve.conf (159 lines) (continued)

```
84
85 [admission]
86 arena_size_mib          = 2048            # byte budget for all image sets
87 fs_reserve_mib          = 1024            # always leave this much free on the arena's filesystem
88 max_inflight_captures   = 1               # concurrent dumps
89 max_capture_s           = 10.0            # refuse dumps estimated to take longer
90 restore_headroom_margin = 0.25            # restore needs RSS * (1 + margin) of free cgroup room
91
92 [capture]
93 criu_bin                  = criu          # looked up in PATH, then /usr/sbin
94 shell_job                 = yes           # --shell-job (workloads are setsid session leaders)
95 log_level                 = 4             # criu -v<N>; 4 = full debug, as in the feasibility test
96 dump_timeout_s            = 60
97 restore_timeout_s         = 60
98 keep_images_after_restore = no            # yes = keep image sets as evidence (they count against the arena)
99 log_tail_lines            = 40            # lines of dump.log / restore.log shown on failure
100 dump_extra_args          =               # --leave-running / --leave-stopped are rejected
101 restore_extra_args       =
102
103 [workloads]
104 python_bin           =                   # empty = the interpreter running the governor
105 run_as_invoking_user = yes               # workloads drop root and run as the sudo caller
106 workload             = compute           # valuable workload: compute (one process) | pipeline (dependent group)
107
108 # pipeline_job (Objective 6): parent + forked child joined by a pipe; hash-chain
109 # parameters are shared with compute_job (compute_hashes_per_iteration, ...).
110 pipeline_parent_mb   = 40                # parent's resident working set
111 pipeline_child_mb    = 260               # child's resident working set
112
113 # compute_job: the process worth preserving
114 compute_working_set_mb         = 128     # resident, dirtied memory (the eviction payoff)
115 compute_hashes_per_iteration   = 50000   # CPU work per iteration (SHA-256 rounds)
116 compute_step_sleep_s           = 0.05    # pause between iterations
117 compute_report_every          = 10      # iterations between state/progress writes
118 compute_touch_every           = 10      # iterations between re-dirtying the working set
119 compute_max_iteration_credit_s = 1.0     # an iteration longer than this spanned a checkpoint
120
121 # hog: the antagonist that creates pressure
122 hog_start_delay_s      = 20              # let compute_job accumulate work first
123 hog_chunk_mb           = 16              # allocation step
124 hog_interval_s         = 0.5             # time between steps
125 hog_target_mb          = 400             # 400 + 128 > 512 -> pressure unless something is evicted
126 hog_hold_s             = 25              # hold at target, then exit so pressure can recover
127 hog_retouch_interval_s = 2.0             # re-dirty all pages while holding (keeps PSI up)
128
129 # ======================================================================
130 # Named scenarios. Each overrides only the keys listed; everything else
131 # is the base configuration above, identical across scenarios.
132 # Select with:  ./run_demo.sh --scenario B [--baseline]
133 # ======================================================================
134 [scenarios]
135 default = A
136
137 [scenario.A]
138 description = Kernel already right - the hog is the largest process, so footprint-based OOM selection kills it and c
  + ompute_job (128 MiB) is never at risk; PRESERVE offers no benefit
139 # (no overrides: the base values above are scenario A)
140
141 [scenario.B]
142 description = Large long-running job vs smaller burst - compute_job is the largest process, so the kernel's footprin
  + t-based OOM selection kills the valuable job
143 workloads.compute_working_set_mb = 300    # compute_job charged ~309 MiB (measured overhead ~8.7 MiB)
144 # hog 240: 309 + 240 + ~7 overhead = ~556 MiB > 512M (180 would give ~496 MiB: no OOM at all).
145 # At the OOM moment compute_job ~309 MiB > hog ~200 MiB, so the kernel picks compute_job.
146 workloads.hog_target_mb          = 240
147 workloads.hog_start_delay_s      = 40     # compute_job reaches R ~12 s (~2.4x min_restart_cost_s) before pressure
148 cgroup.memory_swap_max           = 0      # same cgroup as the baseline: PRESERVE must beat the OOM killer without s
  + wap
149 baseline.reference_live_run      =        # no saved live run yet; compare with src/compare.py afterwards
150
151 [scenario.C]
152 description = Dependent group - pipeline_job parent (40 MiB) + child (260 MiB) joined by a pipe, vs the same hog as
  + B; the kernel OOM-kills the child (largest process) and the parent loses its work through the pipe
153 workloads.workload               = pipeline
154 # child ~269 + parent ~49 (incl. ~9 MiB interpreter each) = ~318 MiB; hog 240 -> ~558 MiB > 512M.
155 # At the OOM moment the hog holds <= ~194 MiB < child ~269 MiB, so the kernel picks the child.
156 workloads.hog_target_mb          = 240
157 workloads.hog_start_delay_s      = 40     # group R (sum of both members) well above min_restart_cost_s
158 cgroup.memory_swap_max           = 0      # as B: PRESERVE must beat the OOM killer without swap
159 baseline.reference_live_run      =
```

run_demo.sh (212 lines)

```
#!/usr/bin/env bash
# ======================================================================
# run_demo.sh -- PRESERVE Review I demonstration.
#
# What it shows:
#   1. Pre-flight: cgroup v2, PSI, CRIU and the project location are sane.
#   2. Safety: the PID whitelist refuses to touch foreign processes.
#   3. The governor run: compute_job + hog in a 512 MiB cgroup; under
#      pressure compute_job is checkpointed, later restored, and resumes.
#   4. Verification: the compute_job hash chain is recomputed from scratch
#      and every logged digest is checked.
#
# Modes:
#   (default)    live: PRESERVE checkpoints and restores (sudo, CRIU needs root)
#   --dry-run    decisions logged, CRIU never invoked (sudo)
#   --baseline   control experiment: governor observes only, swap set to
#                [cgroup] baseline_memory_swap_max, kernel OOM killer acts.
#                No CRIU, so no root: the governor is started with
#                `systemd-run --user --scope`, which places it inside the
#                user's delegated cgroup subtree where it may move its
#                children into preserve-test without privileges.
#
# Everything printed is also saved under evidence/demo-<timestamp>-<mode>/.
# The workloads always run as the invoking user.
#
# Scenarios (named sections in preserve.conf):
#   --scenario A   kernel already right: hog is the largest process (default)
#   --scenario B   large long-running job vs smaller burst: the kernel's
#                  footprint-based OOM selection picks compute_job
#   --scenario C   dependent group: pipeline_job parent+child (pipe); the
#                  kernel kills the child and the parent loses its work too
#
# Usage:  ./run_demo.sh [--scenario A|B] [--dry-run | --baseline] [--no-hog]
#         (all args are passed to governor.py)
# ======================================================================
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
STAMP="$(date +%Y%m%d-%H%M%S)"
MODE=live
SCENARIO=A
prev=""
for arg in "$@"; do
  case "$arg" in
    --dry-run)  MODE=dry-run ;;
    --baseline) MODE=baseline ;;
    --scenario=*) SCENARIO="${arg#--scenario=}" ;;
  esac
  [ "$prev" = "--scenario" ] && SCENARIO="$arg"
  prev="$arg"
done
OUT="$ROOT/evidence/demo-$STAMP-$MODE-$SCENARIO"
PY="$(command -v python3)"

banner() { printf '\n=== %s ===\n' "$*"; }

# Meaning of a governor exit code, from the documented exit-code table (per mode).
rc_meaning() {  # mode rc
  case "$2" in   # codes meaning "no experiment ran", the same in every mode
    75) echo "REFUSED: another governor is running (lock held); no experiment ran"; return ;;
    76) echo "REFUSED: setup aborted (e.g. could not create an exclusive cgroup); no experiment ran"; return ;;
  esac
  case "$1" in
    baseline) case "$2" in
        0) echo "kernel OOM killer fired and was recorded" ;;
        2) echo "no OOM kill happened (no memory crisis occurred)" ;;
        1) echo "error" ;;  *) echo "unexpected exit code" ;; esac ;;
    *) case "$2" in
        0) echo "preserved and resumed" ;;
        2) echo "no capture happened" ;;
        3) echo "no restore happened" ;;
        1) echo "error, or compute_job restarted instead of resuming" ;;
        *) echo "unexpected exit code" ;; esac ;;
  esac
}

# A run is only a success at rc 0 (and, for baseline, the no-crisis rc 2 is
# reported but still treated as a non-success so it cannot masquerade as a demo).
fail_banner() {  # rc
  local meaning; meaning="$(rc_meaning "$MODE" "$1")"
  printf '\n'
  echo "######################################################################"
  echo "##  RUN FAILED  --  governor exited $1: $meaning"
  echo "##  scenario $SCENARIO, mode $MODE.  Verification and evidence skipped."
```

run_demo.sh (212 lines) (continued)

```
  echo "########################################################################"
  { echo "FAILED: governor exit $1 ($meaning)"; echo "scenario=$SCENARIO mode=$MODE stamp=$STAMP"; } > "$OUT/FAILED"
  echo "marked $OUT/FAILED"
  echo "console log : $OUT/console.log"
  exit "$1"
}

case "$ROOT" in
  /mnt/*) echo "REFUSING: project is under $ROOT (Windows/OneDrive filesystem); CRIU images must live on native ext4
 + ." >&2
          exit 1 ;;
esac

mkdir -p "$OUT"
exec > >(tee "$OUT/console.log") 2>&1

banner "PRESERVE Review I demo  ($STAMP, scenario: $SCENARIO, mode: $MODE)"
echo "project : $ROOT"
echo "kernel  : $(uname -r)"
echo "python  : $($PY --version 2>&1)"
(cd "$ROOT/src" && "$PY" config.py --scenario "$SCENARIO" | grep -E "^# scenario|scenario override") || {
  echo "unknown scenario '$SCENARIO'"; echo "FAILED: unknown scenario '$SCENARIO'" > "$OUT/FAILED"; exit 1; }

banner "1. Pre-flight checks"
fail=0
check() { if eval "$2" >/dev/null 2>&1; then echo "  PASS  $1"; else echo "  FAIL  $1"; fail=1; fi; }
check "cgroup v2 mounted"            "test -f /sys/fs/cgroup/cgroup.controllers"
check "memory controller available"  "grep -qw memory /sys/fs/cgroup/cgroup.controllers"
check "PSI /proc/pressure/memory"    "test -r /proc/pressure/memory"
check "criu installed"               "command -v criu || test -x /usr/sbin/criu"
check "systemd user delegation"      "test -d /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service"
[ "$fail" -eq 0 ] || { echo "pre-flight failed; not starting the governor"
  echo "FAILED: pre-flight checks failed" > "$OUT/FAILED"; exit 1; }

banner "2. Safety gate: whitelist refuses foreign PIDs"
(cd "$ROOT/src" && "$PY" cgroup.py --refusal-demo)

if [ "$MODE" = baseline ] && command -v systemd-run >/dev/null && systemctl --user show-environment >/dev/null 2>&1;
 +  then
  banner "3. Governor run (mode: baseline, unprivileged via systemd-run --user --scope)"
  LAUNCH=(systemd-run --user --scope --quiet --)
else
  banner "3. Governor run (mode: $MODE, sudo)"
  sudo -v
  LAUNCH=(sudo)
fi
cp "$ROOT/preserve.conf" "$OUT/preserve.conf"

# (defence 2) Clear stale workload state BEFORE the run, so a governor that
# exits before writing (bad argument, --help, early abort) cannot leave an
# old progress log behind for the verification step to read as success.
#
# The clearing runs UNDER the governor lock (runlock.py --hold, which also records
# what holds it). If another governor is running, its live run/ files are left
# alone and this run is refused: earlier, an unguarded rm here deleted another
# run's state mid-run. While we hold the lock for the rm, no governor can start,
# so there is no check-then-act gap.
set +e
"$PY" "$ROOT/src/runlock.py" --config "$ROOT/preserve.conf" --hold \
  --label "run_demo.sh: clearing run/ before a $MODE run" -- \
  rm -f "$ROOT/run/"*.progress.log "$ROOT/run/"*.state.json 2>/dev/null
clr=$?
set -e
if [ "$clr" -eq 75 ]; then
  # `|| true`: --status exits 75 when held, and under `set -e` a failing command
  # substitution in an assignment would kill the script before the banner printed.
  holder="$(cd "$ROOT/src" && "$PY" runlock.py --status 2>&1)" || true
  echo "########################################################################"
  echo "##  REFUSED -- another governor is running; nothing was cleared or started"
  echo "##  $holder"
  echo "########################################################################"
  { echo "FAILED: refused, another governor is running"; echo "$holder"; } > "$OUT/FAILED"
  echo "console log : $OUT/console.log"
  exit 75
elif [ "$clr" -ne 0 ]; then
  echo "FATAL: could not clear stale run/ state files; aborting" >&2
  { echo "FAILED: could not clear stale run/ state"; } > "$OUT/FAILED"
  echo "console log : $OUT/console.log"
  exit 1
fi
RUN_START=$(date +%s)   # (defence 3) verification refuses progress older than this

set +e
"${LAUNCH[@]}" "$PY" "$ROOT/src/governor.py" --config "$ROOT/preserve.conf" "$@"
```

run_demo.sh (212 lines) (continued)

```
rc=$?
set -e
echo "governor exit code: $rc  ($(rc_meaning "$MODE" "$rc"))"

# (requirement 1) Any non-zero exit stops here with a failure banner; the
# verification and evidence steps below never run for a failed run.
[ "$rc" -eq 0 ] || fail_banner "$rc"

if [ "$MODE" = baseline ]; then
  banner "Kernel log: OOM-killer records for this run"
  dmesg 2>/dev/null | grep -E "oom-kill:|Killed process" | tail -6 || echo "(dmesg not readable)"
fi

if ls "$ROOT/run/"pipeline_job.*.progress.log >/dev/null 2>&1; then
  # Scenario C (Objective 6): the governor already recomputed every member's hash
  # chain and printed the group verdict in its SUMMARY; keep the logs as evidence.
  banner "4. Pipeline group: per-member hash chains (verified by the governor, see SUMMARY)"
  grep -hE "member pipeline|pipeline group continuity" "$ROOT/evidence/governor.log" | tail -3 || true
  cp "$ROOT/run/"*.json "$ROOT/run/"*.log "$OUT/" 2>/dev/null || true
else
banner "4. Verify compute_job hash chain"
PROG="$ROOT/run/compute_job.progress.log"
if [ ! -f "$PROG" ]; then
  echo "no progress log found (nothing was run); not reporting a result"
elif [ "$(awk 'END{print int($1)}' "$PROG" 2>/dev/null || echo 0)" -lt "$RUN_START" ]; then
  # (defence 3) newest line predates this run: a leftover we must not trust.
  echo "STALE progress log (newest line older than this run's start $RUN_START); refusing to verify"
  { echo "FAILED: only stale progress data (newest line < run start)"; } > "$OUT/FAILED"
  echo "console log : $OUT/console.log"
  exit 1
else
  "$PY" "$ROOT/workloads/compute_job.py" --verify-log "$PROG" || true
  echo "--- progress lines around the largest gap ---"
  awk 'NR>1 { gap=$1-prev; if (gap>best) { best=gap; line=NR } } { prev=$1; rows[NR]=$0 }
       END { for (i=line-3; i<=line+2; i++) if (i in rows) print (i==line ? ">> " : "   ") rows[i];
             printf "largest gap: %.1f s\n", best }' "$PROG"
  cp "$ROOT/run/"*.json "$ROOT/run/"*.log "$OUT/" 2>/dev/null || true
fi

fi

banner "Evidence"
echo "console log : $OUT/console.log"
echo "governor log: $ROOT/evidence/governor.log"
echo "telemetry   : $ROOT/evidence/telemetry.csv"
exit "$rc"
```

run_failsafe.sh (296 lines)

```
#!/usr/bin/env bash
# ======================================================================
# run_failsafe.sh -- demonstrate PRESERVE's fail-safe property.
#
# The report claims: every path that does not reach ADMIT ends in STAND
# DOWN, and in STAND DOWN the system behaves no worse than the unmodified
# kernel. This script exercises that claim by running Scenario B once per
# injected fault (governor --inject FAULT), plus:
#   * one clean baseline (kernel OOM killer acts, no governor intervention)
#   * one clean live run with NO fault, to confirm unchanged behaviour
#     (it must still capture AND restore)
#
# For each run it prints, in the matrix style:
#   Fault | Reached capture? | Process state at exit | Same as baseline?
# and saves the table to evidence/failsafe-<timestamp>.txt.
#
# Live runs (the clean live run and every fault) need root for CRIU; the
# baseline does not. Safe to run non-interactively with cached sudo.
# ======================================================================
set -uo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PY="$(command -v python3)"
SCEN=B
STAMP="$(date +%Y%m%d-%H%M%S)"
TABLE="$ROOT/evidence/failsafe-$STAMP.txt"
FAULTS=(arena-too-small dump-fail dump-timeout restore-blocked stale-pid governor-dies)
LOCKCLI=("$PY" "$ROOT/src/runlock.py" --config "$ROOT/preserve.conf")
# Every governor run now creates its own <prefix>-<pid>-<timestamp> cgroup.
CG_PARENT="/sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service"
CG_PREFIX="$(cd "$ROOT/src" && "$PY" -c 'from config import Config; print(Config().get_str("cgroup", "name"))')"

case "$ROOT" in
  /mnt/*) echo "REFUSING: project under $ROOT; must be on native ext4." >&2; exit 1 ;;
esac

refuse_if_locked() {  # abort the whole session loudly if another governor is running
  local st
  if ! st="$("${LOCKCLI[@]}" --status 2>&1)"; then
    echo "################################################################################"
    echo "##  REFUSED -- another governor is running. Nothing was started or killed."
    echo "##  $st"
    echo "################################################################################"
    exit 75
  fi
}

# ---- leftover cleanup: ONLY the cgroup of a run this script launched ----------
# Earlier versions SIGKILLed every process in a cgroup found by NAME. With a shared
# name that killed another session's workloads. Now the only cgroup ever reaped is
# the one recorded in the lock file by the governor this script started (started
# at/after $1), and only while no governor holds the lock.
reap_own() {  # launched_at_epoch
  local cg procs
  cg="$("${LOCKCLI[@]}" --last-cgroup --since "$1" 2>/dev/null)" || return 0   # 75: a governor is running
  [ -n "$cg" ] || return 0
  case "$cg" in
    "$CG_PARENT/$CG_PREFIX"-[0-9]*-[0-9]*) ;;
    *) echo "  NOT reaping unexpected cgroup path '$cg'"; return 0 ;;
  esac
  [ -d "$cg" ] || return 0
  procs="$(cat "$cg/cgroup.procs" 2>/dev/null)"
  if [ -n "$procs" ]; then
    echo "  cleaning up this run's leftover processes in $cg: $procs"
    # shellcheck disable=SC2086
    sudo kill -KILL $procs 2>/dev/null
    sleep 1
  fi
  sudo rmdir "$cg" 2>/dev/null || true
}

ROWS="$ROOT/evidence/.failsafe-$STAMP.rows"   # one "fault|reached|state|same" line per run
: > "$ROWS"

# ---- run one scenario-B governor invocation, capture its evidence dir -------
# args: label  inject-or-empty  mode(live|baseline)
run_one() {
  local label=$1 inject=$2 mode=$3 dir rc gargs log
  dir="$ROOT/evidence/failsafe-$STAMP-$label"
  mkdir -p "$dir"
  log="$dir/console.log"
  gargs=(--config "$ROOT/preserve.conf" --scenario "$SCEN")
  [ "$mode" = baseline ] && gargs+=(--baseline)
  [ -n "$inject" ] && gargs+=(--inject "$inject")
```

```
 85
 86   echo; echo "########## $label ##########"
 87   # Clear stale workload state UNDER the governor lock (same defence as run_demo.sh):
 88   # if another governor is running, refuse instead of deleting its live files.
 89   if ! "${LOCKCLI[@]}" --hold --label "run_failsafe.sh: clearing run/ before $label" -- \
 90          rm -f "$ROOT/run/"*.progress.log "$ROOT/run/"*.state.json 2>/dev/null; then
 91     refuse_if_locked
 92     echo "FATAL: could not clear stale run/ state files"; exit 1
 93   fi
 94   cp "$ROOT/preserve.conf" "$dir/preserve.conf"
 95
 96   local launched_at; launched_at="$(date +%s)"
 97   if [ "$mode" = baseline ] && command -v systemd-run >/dev/null && systemctl --user show-environment >/dev/null 2>&
 + 1; then
 98     systemd-run --user --scope --quiet -- "$PY" "$ROOT/src/governor.py" "${gargs[@]}" 2>&1 | tee -i "$log"
 99   else
100     sudo "$PY" "$ROOT/src/governor.py" "${gargs[@]}" 2>&1 | tee -i "$log"
101   fi
102   rc=${PIPESTATUS[0]}
103   echo "exit code: $rc"
104   if [ "$rc" -eq 75 ] || [ "$rc" -eq 76 ]; then
105     # Refused before any experiment ran: never collect run/ files or classify.
106     echo "$label|REFUSED - governor exit $rc (another governor running or no exclusive cgroup)|$dir|REFUSED" >> "$RO
 + WS"
107     echo "FAILED: governor exit $rc (refused; no experiment ran)" > "$dir/FAILED"
108     return
109   fi
110   cp "$ROOT/run/"*.json "$ROOT/run/"*.log "$dir/" 2>/dev/null || true
111   # Record reality for governor-dies, whose governor cannot log its own aftermath:
112   # the members of THIS run's own cgroup (path recorded in the lock file).
113   local leftover="none" own
114   own="$("${LOCKCLI[@]}" --last-cgroup --since "$launched_at" 2>/dev/null)"
115   if [ -n "$own" ] && [ -d "$own" ]; then leftover="$(cat "$own/cgroup.procs" 2>/dev/null | tr '\n' ' ')"; fi
116   classify "$label" "$inject" "$mode" "$rc" "$log" "$leftover" "$dir"
117   reap_own "$launched_at"
118 }
119
120 # ---- derive the four columns from a run's console log + reality -------------
121 # Every column must be backed by a string the governor actually logged. If the
122 # required evidence is not present, the row is emitted as UNCLASSIFIED with its
123 # evidence path -- never a guessed or confident-looking wrong value.
124 classify() {
125   local label=$1 inject=$2 mode=$3 rc=$4 log=$5 leftover=$6 dir=$7
126   local reached state same
127   # emit an UNCLASSIFIED row (naming the label and the evidence dir) and return
128   unclass() { echo "$label|UNCLASSIFIED - check log|$dir|UNCLASSIFIED" >> "$ROWS"; }
129
130   if [ "$mode" = baseline ]; then
131     if grep -q "compute_job .*KILLED" "$log"; then
132       echo "baseline (clean)|n/a|compute_job killed by kernel OOM|reference" >> "$ROWS"
133     elif [ "$rc" = 2 ]; then
134       echo "baseline (clean)|n/a|no OOM kill (no memory crisis)|reference (no kill)" >> "$ROWS"
135     else unclass; fi
136     return
137   fi
138
139   if [ -z "$inject" ]; then
140     # clean live run: must capture AND restore, else it is a regression (loud).
141     local caps res; caps=$(grep -c "PRESERVED dump" "$log"); res=$(grep -c "RESTORED restore" "$log")
142     if [ "$caps" -ge 1 ] && [ "$res" -ge 1 ]; then
143       echo "none (clean live)|yes (dumped+restored)|restored, resumed|better (work preserved)" >> "$ROWS"
144     else
145       echo "none (clean live)|INCOMPLETE|clean run did NOT capture+restore -- regression!|WORSE" >> "$ROWS"
146     fi
147     return
148   fi
149
150   # --- fault runs: each requires positive evidence of its own precondition ---
151   # A self-verified injection that did not fire aborts as INJECTION FAILED; never a pass.
152   if grep -q "INJECTION FAILED\|INJECTION SETUP FAILED" "$log"; then
153     local reason; reason="$(grep -m1 "INJECTION \(SETUP \)\?FAILED" "$log" | sed 's/.*INJECTION/INJECTION/')"
154     echo "$inject|INJECTION FAILED - fault did not occur|$dir|${reason:-INJECTION FAILED}" >> "$ROWS"; return
155   fi
156   case "$inject" in
157     arena-too-small|stale-pid)
158       # Must show a stand-down for THIS fault. Then the kernel is in charge; in
159       # scenario B (no swap) it kills compute_job. Require that kill to claim
160       # "== baseline"; if the kill is absent, do not guess -- UNCLASSIFIED.
161       grep -q "STAND DOWN \[fault=$inject\]" "$log" || { unclass; return; }
162       reached="no (stood down)"
163       if grep -q "compute.*killed by SIGKILL" "$log" || grep -q "oom_kill 0 -> 1" "$log"; then
164         state="stood down; compute_job killed by kernel OOM"; same="yes (compute killed == baseline)"
165       else
166         state="stood down; compute_job NOT killed in window"; same="UNCLASSIFIED - check log"
```

run_failsafe.sh (296 lines) (continued)

```
      fi ;;
    dump-fail|dump-timeout)
      # Must have attempted a real dump and then stood down after it failed.
      grep -q "ACTION: checkpoint" "$log" || { unclass; return; }
      grep -q "STAND DOWN \[fault=$inject\]" "$log" || { unclass; return; }
      reached="attempted (dump failed)"
      local how="dump failed"
      [ "$inject" = dump-fail ]    && grep -q "ENOSPC" "$log" && how="dump failed (ENOSPC)"
      [ "$inject" = dump-timeout ] && how="dump failed (timeout)"
      # "Process state at exit" is the FINAL state. A process that survives the
      # failed dump but is then killed by the kernel OOM is dead at exit == baseline.
      local killed=""
      grep -q "compute.*killed by SIGKILL" "$log" && killed=1
      grep -q "oom_kill 0 -> 1" "$log" && killed=1
      if grep -q "is GONE" "$log"; then
        state="$how; process LOST by the failed dump"; same="yes (process lost == baseline)"
      elif grep -q "SURVIVED" "$log"; then
        if [ -n "$killed" ]; then
          state="$how; survived the failure, then killed by kernel OOM"; same="yes (== baseline)"
        else
          # survived the failure AND still alive when the run ended: verify no kill in log
          state="$how; survived the failure, still running at exit"; same="better (alive at exit)"
        fi
      else unclass; return; fi ;;
    restore-blocked)
      # Must have dumped, then been permanently denied restore.
      grep -q "PRESERVED dump" "$log" || { unclass; return; }
      grep -q "restore admission will DENY\|STAND DOWN \[fault=restore-blocked\]" "$log" || { unclass; return; }
      reached="yes (dumped)"
      # "work recoverable" is only true if the image set is still on disk. The
      # governor (root) verifies that at shutdown and logs it; trust nothing else.
      if grep -q "still checkpointed at .*image present" "$log"; then
        state="checkpointed, not restored; image present on disk"
        same="work recoverable, job incomplete"
      elif grep -q "IMAGE MISSING" "$log"; then
        state="checkpointed but IMAGE MISSING; work NOT recoverable"
        same="worse than baseline (work lost and job dead)"
      else
        unclass; return   # no evidence either way: do not claim recoverability
      fi ;;
    governor-dies)
      grep -q "crash.*before any capture" "$log" || { unclass; return; }
      reached="no (crashed first)"; same="yes (kernel in charge, == no governor)"
      if [ -n "$leftover" ] && [ "$leftover" != "none" ]; then
        state="governor died before capture; compute_job still under kernel at that point (pids: $leftover)"
      else
        state="governor died before capture; workloads already gone by then"
      fi ;;
    *) unclass; return ;;
  esac
  echo "$inject|$reached|$state|$same" >> "$ROWS"
}

# ---- render the matrix-style table -----------------------------------------
render() {
  {
    echo "PRESERVE fail-safe matrix -- $(date '+%Y-%m-%d %H:%M:%S')  (scenario $SCEN)"
    echo "Arena at session start (reaped first): ${ARENA_START:-not reaped}"
    echo
    # Formatted with awk, not column(1): column is not installed everywhere and
    # its absence silently produced an EMPTY table body in earlier runs.
    { echo "Fault|Reached capture?|Process state at exit|Same as baseline?"
      cat "$ROWS"; } | awk -F'|' '
      function pad(s, n) { while (length(s) < n) s = s " "; return s }
      { rows++; if (NF > nc) nc = NF
        for (i = 1; i <= NF; i++) { cell[rows, i] = $i; if (length($i) > w[i]) w[i] = length($i) } }
      END {
        for (r = 1; r <= rows; r++) {
          out = ""
          for (i = 1; i <= nc; i++) { out = out pad(cell[r, i], w[i]); if (i < nc) out = out " | " }
          sub(/ +$/, "", out); print out
          if (r == 1) { sep = ""
            for (i = 1; i <= nc; i++) { d = ""; for (k = 0; k < w[i]; k++) d = d "-"
              sep = sep d; if (i < nc) sep = sep "-+-" } print sep }
        } }'
    echo
    echo "Reached capture?  = did the governor get as far as attempting a CRIU dump."
    echo "Process state     = the valuable compute_job's fate when the run ended."
    echo "Same as baseline? = baseline kills compute_job and destroys its work; a fail-safe"
    echo "                    outcome is no worse ('yes'). A cell may say 'better' ONLY if the"
    echo "                    valuable process was in a better state AT EXIT than in baseline."
    echo "failed dumps      : a dump that fails after the process is frozen and partially written"
    echo "                    does not cost the process -- PRESERVE does not destroy work the"
    echo "                    kernel would have kept. The process resumes unfrozen and is then left"
```

run_failsafe.sh (296 lines) (continued)

```
    echo "                    to the kernel, which kills it exactly as in baseline. That is a"
    echo "                    fail-safe guarantee, not an improvement over baseline."
    echo "restore-blocked   : a permanently blocked restore leaves the work intact on disk but the"
    echo "                    job unfinished ('work recoverable, job incomplete'). That is better"
    echo "                    than baseline only if the restore can eventually proceed; within the"
    echo "                    run itself the job does not complete, so it does not beat baseline on"
    echo "                    completion."
    echo "UNCLASSIFIED      : the run's log lacked the evidence to decide this row; read the named"
    echo "                    evidence directory. Never treated as a pass."
    echo
    echo "Per-run evidence: evidence/failsafe-$STAMP-<label>/"
  } | tee "$TABLE"
}

# Allow sourcing (FAILSAFE_LIB=1) to unit-test classify()/render without running.
if [ "${FAILSAFE_LIB:-}" != 1 ]; then
  refuse_if_locked   # another governor running: do nothing at all
  # ---- sudo guard (works interactively and non-interactively) -----------
  if [ -t 0 ]; then
    sudo -v || exit 1
  else
    sudo -n true 2>/dev/null || { echo "need cached sudo (run 'sudo -v' first, or run interactively)"; exit 1; }
  fi
  # keep the timestamp fresh for the whole session
  ( while kill -0 "$$" 2>/dev/null; do sudo -n true 2>/dev/null; sleep 50; done ) & KEEPALIVE=$!
  trap 'kill "$KEEPALIVE" 2>/dev/null' EXIT

  # Reap finished images before the batch (never one still checkpointed and not
  # restored, e.g. a previous restore-blocked run's), and record the starting state.
  ARENA_OUT="$(sudo "$PY" "$ROOT/src/governor.py" --config "$ROOT/preserve.conf" --reap-arena 2>&1)"; arc=$?
  ARENA_START="$(printf '%s\n' "$ARENA_OUT" | grep -o 'ARENA: .*' | tail -1)"
  [ -n "$ARENA_START" ] || ARENA_START="ARENA: NOT REAPED (exit $arc): $(printf '%s\n' "$ARENA_OUT" | tail -1)"
  [ "$arc" -eq 0 ] || ARENA_START="$ARENA_START  [reap exit $arc -- starting occupancy unverified]"
  echo "$ARENA_START"

  echo "Fail-safe demonstration (scenario $SCEN). Each fault forces one real precondition to fail."
  run_one baseline      ""              baseline   # reference: kernel OOM acts
  run_one clean         ""              live       # no fault: must still capture+restore
  for f in "${FAULTS[@]}"; do
    run_one "$f" "$f" live
  done
  echo
  render
  echo
  echo "table saved to $TABLE"
fi
```

run_all.sh (266 lines)

```
#!/usr/bin/env bash
# ======================================================================
# run_all.sh -- interactive runner for the PRESERVE experiment matrix.
#
# Menu-driven: runs the scenario A/B baseline and PRESERVE experiments,
# a dry run, or a custom run with chosen workload sizes, and after every
# run prints a results table (also saved to evidence/matrix-<ts>.txt).
#
#   ./run_all.sh              each chosen run executes once
#   ./run_all.sh --repeat 5   each chosen run executes 5 times; the table
#                             then shows median [min-max] per column
#
# How runs are executed
#   * Scenario A/B baseline and PRESERVE runs go through run_demo.sh, so
#     they produce the same evidence folders as before. The folder is read
#     from the "console log : <path>" line run_demo.sh prints.
#   * Dry runs and custom runs call src/governor.py directly: run_demo.sh
#     would use sudo for a dry run and does not know custom scenarios.
#     Baseline and dry runs run unprivileged (systemd-run --user --scope);
#     only PRESERVE (live) runs use sudo, because CRIU needs root.
#   * Custom sizes are written to a generated copy of preserve.conf with a
#     [scenario.custom] section; preserve.conf itself is never edited.
#
# Numbers in the table come from src/matrix.py, which reads only what each
# run wrote (baseline_result.json, the governor's console log and SUMMARY
# block, the compute_job progress log).
# ======================================================================
set -uo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PY="$(command -v python3)"
MATRIX=("$PY" "$ROOT/src/matrix.py")
REPEAT=1

usage() { sed -n '2,16p' "$0" | sed 's/^# \{0,1\}//'; }

while [ $# -gt 0 ]; do
  case "$1" in
    --repeat)   REPEAT="${2:-}"; shift 2 ;;
    --repeat=*) REPEAT="${1#--repeat=}"; shift ;;
    -h|--help)  usage; exit 0 ;;
    *) echo "unknown argument: $1" >&2; usage; exit 2 ;;
  esac
done
if ! [[ "$REPEAT" =~ ^[1-9][0-9]*$ ]]; then
  echo "--repeat needs a positive integer (got '$REPEAT')" >&2; exit 2
fi
case "$ROOT" in
  /mnt/*) echo "REFUSING: project is under $ROOT; it must live on the native Linux filesystem." >&2; exit 1 ;;
esac

SESSION_TS="$(date +%Y%m%d-%H%M%S)"
SESSION="$ROOT/evidence/matrix-$SESSION_TS.jsonl"     # raw per-run rows (Review II statistics)
TABLE="$ROOT/evidence/matrix-$SESSION_TS.txt"         # the printed table
ARENA_LOG="$ROOT/evidence/matrix-$SESSION_TS.arena"   # per-batch arena occupancy, shown in the table header
CUSTOM_CONF="$ROOT/.run_all-custom.conf"              # project root, so relative paths resolve
BATCH=0
KEEPALIVE=""

cleanup() {
  [ -n "$KEEPALIVE" ] && kill "$KEEPALIVE" 2>/dev/null
  rm -f "$CUSTOM_CONF"
}
trap cleanup EXIT
# Ctrl-C stops the current run (the governor shuts down cleanly) and returns to the menu.
trap 'echo; echo "(interrupted -- returning to the menu)"' INT

# ---- sudo once, kept alive for the session ----------------------------
echo "PRESERVE runs need root for CRIU (checkpoint/restore); baseline and dry runs do not."
echo "Enter your sudo password once now; it is kept alive for this session."
SUDO_OK=0
if sudo -v; then
  SUDO_OK=1
  ( while kill -0 "$$" 2>/dev/null; do sudo -n -v 2>/dev/null; sleep 50; done ) &
  KEEPALIVE=$!
else
  echo "sudo not available: PRESERVE runs (options 2 and 4, PRESERVE custom runs) will be marked FAILED."
fi

can_run_unprivileged() {
  command -v systemd-run >/dev/null && systemctl --user show-environment >/dev/null 2>&1
}

# ---- recording and table ----------------------------------------------
```

run_all.sh (266 lines) (continued)

```
record() {  # scenario mode rc evidence_dir
  local args=(row --scenario "$1" --mode "$2" --rc "$3" --batch "$BATCH")
  [ -n "$4" ] && args+=(--evidence "$4")
  if ! "${MATRIX[@]}" "${args[@]}" >> "$SESSION"; then
    echo "!! could not summarise the $1 $2 run (evidence: ${4:-none})"
  fi
}

show_table() {
  echo
  "${MATRIX[@]}" table --session "$SESSION" --out "$TABLE" --repeat "$REPEAT" --arena-log "$ARENA_LOG" \
    || echo "!! could not build the results table"
}

# Before each batch: delete finished images (never one still checkpointed and not
# restored) and record the starting occupancy for the table header. The arena is
# root-owned after live runs, so this uses sudo when the session has it.
reap_arena() {
  local out rc line
  if [ "$SUDO_OK" -eq 1 ]; then
    out="$(sudo "$PY" "$ROOT/src/governor.py" --config "$ROOT/preserve.conf" --reap-arena 2>&1)"; rc=$?
  else
    out="$("$PY" "$ROOT/src/governor.py" --config "$ROOT/preserve.conf" --reap-arena 2>&1)"; rc=$?
  fi
  line="$(printf '%s\n' "$out" | grep -o 'ARENA: .*' | tail -1)"
  [ -n "$line" ] || line="ARENA: NOT REAPED (exit $rc): $(printf '%s\n' "$out" | tail -1)"
  [ "$rc" -eq 0 ] || line="$line  [reap exit $rc -- starting occupancy unverified]"
  echo "batch $BATCH: $line" >> "$ARENA_LOG"
  echo ">> batch $BATCH $line"
}

report_rc() {  # label rc
  if [ "$2" -eq 0 ]; then echo ">> $1: finished (exit 0)"; else echo ">> $1: exit code $2 (see table)"; fi
}

# ---- run executors -----------------------------------------------------
run_standard() {  # scenario(A|B) mode(baseline|live)
  local scen=$1 mode=$2 log rc dir
  local args=(--scenario "$scen")
  [ "$mode" = baseline ] && args+=(--baseline)
  if [ "$mode" = live ] && [ "$SUDO_OK" -ne 1 ]; then
    echo ">> Scenario $scen PRESERVE: FAILED -- needs sudo"; record "$scen" live 126 ""; return
  fi
  echo; echo "########## Scenario $scen -- ${mode/live/PRESERVE} ##########"
  log="$(mktemp)"
  "$ROOT/run_demo.sh" "${args[@]}" 2>&1 | tee -i "$log"   # -i: Ctrl-C must not cut the log
  rc=${PIPESTATUS[0]}
  dir="$(grep -E '^console log : ' "$log" | tail -1 | sed 's/^console log : //')"
  rm -f "$log"
  [ -n "$dir" ] && dir="$(dirname "$dir")"
  [ -z "$dir" ] && echo ">> run_demo.sh did not report an evidence folder"
  report_rc "Scenario $scen ${mode/live/PRESERVE}" "$rc"
  record "$scen" "$mode" "$rc" "$dir"
}

run_direct() {  # label scenario-name mode(baseline|live|dry-run) config-file
  local label=$1 scen=$2 mode=$3 conf=$4 rc launch
  local dir="$ROOT/evidence/demo-$(date +%Y%m%d-%H%M%S)-$mode-$label"
  local gargs=(--config "$conf" --scenario "$scen")
  [ "$mode" = baseline ] && gargs+=(--baseline)
  [ "$mode" = dry-run ] && gargs+=(--dry-run)
  if [ "$mode" != live ] && can_run_unprivileged; then
    launch=(systemd-run --user --scope --quiet --)
  elif [ "$SUDO_OK" -eq 1 ]; then
    launch=(sudo)
  else
    echo ">> $label $mode: FAILED -- needs sudo"; record "$label" "$mode" 126 ""; return
  fi
  echo; echo "########## $label -- $mode ##########"
  mkdir -p "$dir"
  cp "$conf" "$dir/preserve.conf"
  "${launch[@]}" "$PY" "$ROOT/src/governor.py" "${gargs[@]}" 2>&1 | tee -i "$dir/console.log"
  rc=${PIPESTATUS[0]}
  if [ "$rc" -eq 0 ]; then
    cp "$ROOT/run/"*.json "$ROOT/run/"*.log "$dir/" 2>/dev/null
    if [ -f "$dir/compute_job.progress.log" ]; then
      "$PY" "$ROOT/workloads/compute_job.py" --verify-log "$dir/compute_job.progress.log" 2>&1 | tee -a "$dir/consol
e.log"
    fi
  else
    # A refused or failed run must not pick up run/ files: they may belong to a
    # different run that is still going (rc 75 = another governor holds the lock).
    echo "FAILED: governor exit $rc; run/ files not collected" > "$dir/FAILED"
    echo ">> $label $mode: governor exit $rc -- marked FAILED, run/ not collected"
```

run_all.sh (266 lines) (continued)

```
  fi
  echo "console log : $dir/console.log"
  report_rc "$label $mode" "$rc"
  record "$label" "$mode" "$rc" "$dir"
}

batch() {  # run one menu choice REPEAT times as one table row
  BATCH=$((BATCH + 1))
  reap_arena
  local i
  for ((i = 1; i <= REPEAT; i++)); do
    [ "$REPEAT" -gt 1 ] && echo && echo "=== repetition $i of $REPEAT ==="
    "$@"
  done
}

# ---- prompts -----------------------------------------------------------
ask_scenario() {  # sets SCEN
  local v
  read -r -p "Scenario A or B? [B]: " v
  SCEN="${v:-B}"; SCEN="${SCEN^^}"
  [[ "$SCEN" =~ ^[AB]$ ]] || { echo "unknown scenario '$SCEN'"; return 1; }
}

ask_number() {  # prompt default regex -> REPLY_VALUE
  local v
  while true; do
    read -r -p "$1 [$2]: " v
    v="${v:-$2}"
    [[ "$v" =~ $3 ]] && { REPLY_VALUE="$v"; return 0; }
    echo "   not a valid number: '$v'"
  done
}

custom_run() {
  local defaults c h d v mode
  defaults="$("${MATRIX[@]}" defaults --scenario B)" || { echo "could not read preserve.conf"; return; }
  eval "$defaults"   # COMPUTE HOG DELAY MEMMAX_MIB (shell-quoted by matrix.py)
  echo "Custom run (based on scenario B: no swap, same thresholds). Press Enter to keep a value."
  ask_number "compute_working_set_mb" "$COMPUTE" '^[0-9]+$'; c=$REPLY_VALUE
  ask_number "hog_target_mb"          "$HOG"     '^[0-9]+$'; h=$REPLY_VALUE
  ask_number "hog_start_delay_s"      "$DELAY"   '^[0-9]+([.][0-9]+)?$'; d=$REPLY_VALUE
  if [ $((c + h)) -le "$MEMMAX_MIB" ]; then
    echo
    echo "  !!! WARNING: compute ($c) + hog ($h) = $((c + h)) MiB does NOT exceed memory_max ($MEMMAX_MIB MiB)."
    echo "  !!! No memory crisis will occur: the OOM killer will not fire and PRESERVE has nothing to save."
    echo "  !!! (Each Python process adds ~8 MiB of its own, so values just under the limit are borderline.)"
    read -r -p "  Run anyway? [y/N]: " v
    [[ "$v" =~ ^[yY] ]] || { echo "cancelled"; return; }
  fi
  read -r -p "Mode: 1) baseline  2) PRESERVE  3) dry run  [1]: " v
  case "${v:-1}" in 1) mode=baseline ;; 2) mode=live ;; 3) mode=dry-run ;; *) echo "invalid mode"; return ;; esac
  "${MATRIX[@]}" custom-config --base B --compute "$c" --hog "$h" --delay "$d" --out "$CUSTOM_CONF" \
    || { echo "could not generate the custom configuration"; return; }
  batch run_direct "custom-c${c}-h${h}-d${d}" custom "$mode" "$CUSTOM_CONF"
}

menu() {
  cat <<EOF

  PRESERVE experiment runner$( [ "$REPEAT" -gt 1 ] && echo "   (each run repeated $REPEAT times)" )
  ------------------------------------------------
   1) Scenario B - baseline   (kernel OOM kills the valuable job)
   2) Scenario B - PRESERVE   (we checkpoint and restore it)
   3) Scenario A - baseline   (kernel correctly kills the hog)
   4) Scenario A - PRESERVE   (no benefit - the honest negative case)
   5) Run all four, in order
   6) Dry run (decisions only, nothing frozen)
   7) Show the last saved results table
   8) Custom run - choose workload sizes
   q) Quit
  ------------------------------------------------
EOF
}

# ---- main loop ---------------------------------------------------------
while true; do
  menu
  read -r -p "  Choose: " choice || break
  case "$choice" in
    1) batch run_standard B baseline ;;
    2) batch run_standard B live ;;
    3) batch run_standard A baseline ;;
    4) batch run_standard A live ;;
```

run_all.sh (266 lines) (continued)

```
    5) batch run_standard B baseline
       batch run_standard B live
       batch run_standard A baseline
       batch run_standard A live ;;
    6) ask_scenario || continue
       batch run_direct "$SCEN" "$SCEN" dry-run "$ROOT/preserve.conf" ;;
    7) "${MATRIX[@]}" latest; continue ;;
    8) custom_run ;;
    q|Q) break ;;
    "") continue ;;
    *) echo "  unknown choice '$choice'"; continue ;;
  esac
  show_table
done
echo "bye. session table: $( [ -f "$TABLE" ] && echo "$TABLE" || echo "(none: no runs completed)")"
```

run_overhead.sh (302 lines)

```
#!/usr/bin/env bash
# ======================================================================
# run_overhead.sh -- what PRESERVE costs when it is doing nothing.
#
# The governor is idle in the normal case: it samples PSI and cgroup
# counters and decides not to act. This measures that cost.
#
#   Part 1  governor self-cost over a window (CPU, % of one core, per-
#           iteration mean/p95/max, RSS start vs end, arena cost)
#   Part 2  A/B workload throughput: the SAME compute_job for the SAME
#           duration, with the governor running and without it, repeated
#           so the difference can be compared against run-to-run noise.
#
# Neither part needs CRIU, so neither needs root: the governor runs in the
# user's own delegated cgroup subtree via systemd-run --user --scope.
#
# Usage: ./run_overhead.sh [--duration N] [--repeat N] [--scenario A|B]
# Writes evidence/overhead-<timestamp>.txt (rendered with awk, not column).
# ======================================================================
set -uo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PY="$(command -v python3)"
STAMP="$(date +%Y%m%d-%H%M%S)"
OUT="$ROOT/evidence/overhead-$STAMP.txt"
WORK="$ROOT/evidence/overhead-$STAMP"
SCEN=B
DURATION=""
REPEAT=""

while [ $# -gt 0 ]; do
  case "$1" in
    --duration) DURATION="$2"; shift 2 ;;
    --repeat)   REPEAT="$2";   shift 2 ;;
    --scenario) SCEN="$2";     shift 2 ;;
    -h|--help)  sed -n '2,18p' "$0" | sed 's/^# \{0,1\}//'; exit 0 ;;
    *) echo "unknown argument: $1" >&2; exit 2 ;;
  esac
done
case "$ROOT" in /mnt/*) echo "REFUSING: project under $ROOT; must be on native ext4." >&2; exit 1 ;; esac

# Defaults from preserve.conf unless overridden.
conf() { "$PY" - "$1" <<EOF
import sys
sys.path.insert(0, "$ROOT/src")
from config import Config
print(Config(None, "$SCEN").get_str("overhead", sys.argv[1]))
EOF
}
[ -n "$DURATION" ] || DURATION="$(conf duration_s)"
[ -n "$REPEAT" ]   || REPEAT="$(conf repeat)"
mkdir -p "$WORK"

# ---- governor lock and per-run cgroups -----------------------------------
# Every governor run creates its own <prefix>-<pid>-<timestamp> cgroup and holds an
# exclusive flock while it runs (src/runlock.py). This script NEVER kills by cgroup
# name: an earlier reap() of "preserve-test" SIGKILLed another session's workloads.
# It only reaps the cgroup its own governor recorded in the lock file.
LOCKCLI=("$PY" "$ROOT/src/runlock.py" --config "$ROOT/preserve.conf")
LOCK="$(cd "$ROOT/src" && "$PY" -c 'from config import Config; print(Config().get_path("paths", "lock_file"))')"
mkdir -p "$(dirname "$LOCK")"
CG_PARENT="/sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service"
CG_PREFIX="$(cd "$ROOT/src" && "$PY" -c 'from config import Config; print(Config().get_str("cgroup", "name"))')"

refuse() {  # loud abort: nothing further is started and nothing is killed
  echo "################################################################################"
  echo "##  REFUSED -- $1"
  echo "##  Nothing further was started or killed; this session's data is incomplete."
  echo "################################################################################"
  exit 75
}
refuse_if_locked() {
  local st
  st="$("${LOCKCLI[@]}" --status 2>&1)" || refuse "another governor is running: $st"
}
reap_own() {  # launched_at_epoch -- leftovers of the governor THIS script launched only
  local cg p
  cg="$("${LOCKCLI[@]}" --last-cgroup --since "$1" 2>/dev/null)" || return 0   # 75: a governor is running
  [ -n "$cg" ] || return 0
  case "$cg" in
    "$CG_PARENT/$CG_PREFIX"-[0-9]*-[0-9]*) ;;
    *) echo "  NOT reaping unexpected cgroup path '$cg'"; return 0 ;;
  esac
  [ -d "$cg" ] || return 0
```

```
  p="$(cat "$cg/cgroup.procs" 2>/dev/null)"
  if [ -n "$p" ]; then
    echo "  cleaning up this run's leftover processes in $cg: $p"
    # shellcheck disable=SC2086
    kill -KILL $p 2>/dev/null || echo "  WARNING: could not kill $p"
    sleep 1
  fi
  rmdir "$cg" 2>/dev/null || echo "  WARNING: could not remove $cg"
}
unpriv() { command -v systemd-run >/dev/null && systemctl --user show-environment >/dev/null 2>&1; }
launch() { if unpriv; then systemd-run --user --scope --quiet -- "$@"; else "$@"; fi; }

refuse_if_locked
echo "PRESERVE overhead measurement: scenario $SCEN, ${DURATION}s window, $REPEAT run(s) per arm"
echo "(no CRIU involved, so no root required)"

# ---- arm A: governor running (also yields the governor self-cost) -----------
for i in $(seq 1 "$REPEAT"); do
  echo; echo "### arm A (governor running) run $i/$REPEAT"
  # Clear run/ UNDER the lock, so a running governor's live files are never deleted.
  "${LOCKCLI[@]}" --hold --label "run_overhead.sh: clearing run/ before arm A run $i" -- \
    rm -f "$ROOT/run/"*.progress.log "$ROOT/run/"*.state.json "$ROOT/run/overhead_result.json" 2>/dev/null
  case $? in
    0) ;;
    75) refuse "a governor started before arm A run $i (lock held)" ;;
    *)  refuse "could not clear run/ state before arm A run $i" ;;
  esac
  launched_at="$(date +%s)"
  launch "$PY" "$ROOT/src/governor.py" --config "$ROOT/preserve.conf" --scenario "$SCEN" \
         --measure-overhead --overhead-duration "$DURATION" 2>&1 | tee -i "$WORK/A$i.log"
  rc=${PIPESTATUS[0]}
  echo "$rc" > "$WORK/A$i.rc"
  case "$rc" in 75|76) refuse "arm A run $i: governor refused to start (exit $rc)" ;; esac
  if [ -f "$ROOT/run/overhead_result.json" ]; then cp "$ROOT/run/overhead_result.json" "$WORK/A$i.json"
  else echo "WARNING: arm A run $i produced no overhead_result.json"; fi
  # The governor exits 1 if pressure left 'normal' or anything was captured: not idle.
  [ "$rc" = 0 ] || echo "  arm A run $i: governor exit $rc -> INVALID, excluded from the aggregate"
  reap_own "$launched_at"
done

# ---- arm B: identical workload, NO governor, but HOLDING THE LOCK ------------
# With no governor running, nothing would stop another governor (and its workload)
# from starting mid-window and competing for the CPU. So each arm B run holds the
# same exclusive lock for its whole duration; a governor that tries to start in the
# meantime refuses with exit 75.
REF=""
for i in $(seq 1 "$REPEAT"); do
  if [ "$(cat "$WORK/A$i.rc" 2>/dev/null)" = 0 ] && [ -f "$WORK/A$i.json" ]; then REF="$WORK/A$i.json"; break; fi
done
[ -n "$REF" ] || refuse "no valid arm A run to take the workload arguments from"
MEMMAX="$("$PY" -c "import json; print(int(json.load(open('$REF'))['cgroup_memory_max_mib']))")"
# Decide the memory environment ONCE with a probe. The old version used
# 'confined || unconfined' around the run itself, and because timeout(1) exits 124
# at the normal end of the window, the unconfined fallback ran every time.
CONFINE=()
if unpriv && systemd-run --user --scope --quiet -p "MemoryMax=${MEMMAX}M" -p MemorySwapMax=0 -- true 2>/dev/null; th
+ en
  CONFINE=(systemd-run --user --scope --quiet -p "MemoryMax=${MEMMAX}M" -p MemorySwapMax=0 --)
  BCONF="confined: MemoryMax=${MEMMAX}M, MemorySwapMax=0 (same limits as arm A)"
else
  BCONF="UNCONFINED: systemd-run with memory properties unavailable (arms not comparable)"
fi
echo; echo "arm B memory environment: $BCONF"
echo "$BCONF" > "$WORK/B.memenv"

for i in $(seq 1 "$REPEAT"); do
  echo; echo "### arm B (no governor, holding the lock) run $i/$REPEAT"
  mapfile -t ARGS < <("$PY" -c "
import json
a = json.load(open('$REF'))['compute_args']
repl = {'--state-file': '$WORK/B$i.state.json', '--progress-log': '$WORK/B$i.progress.log'}
out, k = [], 0
while k < len(a):
    if a[k] in repl: out += [a[k], repl[a[k]]]; k += 2
    else: out.append(a[k]); k += 1
print('\n'.join(out))")
  "${LOCKCLI[@]}" --hold --label "run_overhead.sh: arm B run $i/$REPEAT (compute_job, no governor)" -- \
    "${CONFINE[@]}" timeout -s TERM "$DURATION" "$PY" "$ROOT/workloads/compute_job.py" "${ARGS[@]}" \
    > "$WORK/B$i.log" 2>&1
  rc=$?
  echo "$rc" > "$WORK/B$i.rc"
  case "$rc" in
    0|124) ;;   # 124 = timeout expired: the normal end of the window
    75) refuse "a governor holds the lock; arm B run $i was not started" ;;
```

run_overhead.sh (302 lines) (continued)

```
    *)  echo "  arm B run $i: exit $rc -> INVALID, excluded from the aggregate" ;;
  esac
  echo "arm B run $i: $(grep -o '"iteration": [0-9]*' "$WORK/B$i.state.json" 2>/dev/null | head -1)"
done

# ---- aggregate + render ----------------------------------------------------
"$PY" - "$WORK" "$REPEAT" "$DURATION" "$SCEN" <<'PYEOF' > "$WORK/rows.psv"
import json, statistics, sys
from pathlib import Path
work, repeat, duration, scen = Path(sys.argv[1]), int(sys.argv[2]), float(sys.argv[3]), sys.argv[4]

def rc(path):
    try: return int(path.read_text().strip())
    except Exception: return None

# Only runs that passed their own validity checks are aggregated: an arm A run must
# have exited 0 (the governor exits 1 if pressure left 'normal' or anything was
# captured), and an arm B run must have ended normally (0, or 124 = window over).
A, excluded = [], []
for i in range(1, repeat + 1):
    j, r = work / f"A{i}.json", rc(work / f"A{i}.rc")
    if r != 0 or not j.exists():
        excluded.append(f"A{i} (governor exit {r}{'' if j.exists() else ', no result'})"); continue
    x = json.loads(j.read_text())
    if not x.get("stayed_below_advisory", False) or x.get("captures"):
        excluded.append(f"A{i} (not idle: levels {x.get('level_counts')}, captures {x.get('captures')})"); continue
    A.append(x)
b_it = []
for i in range(1, repeat + 1):
    r = rc(work / f"B{i}.rc")
    try: it = json.loads((work / f"B{i}.state.json").read_text()).get("iteration")
    except Exception: it = None
    if r in (0, 124) and it: b_it.append(it)
    else: excluded.append(f"B{i} (exit {r}, iterations {it})")
a_it = [x.get("compute_iterations") for x in A if x.get("compute_iterations")]

rows = []
def row(k, v): rows.append(f"{k}|{v}")

if not A:
    print("measurement|FAILED: no valid arm A run" + (f" (excluded: {'; '.join(excluded)})" if excluded else ""))
    raise SystemExit(0)
o = A[0]
def agg(vals, unit="", nd=3):
    vals = [v for v in vals if v is not None]
    if not vals: return "n/a"
    f = lambda x: f"{x:.{nd}f}{unit}" if isinstance(x, float) else f"{x}{unit}"
    if len(vals) == 1: return f(vals[0])
    return f"{f(statistics.median(vals))} [{f(min(vals))}-{f(max(vals))}]"

row("scenario / window", f"{scen} / {o['duration_s']:.1f}s at {o['sample_interval_s']}s interval")
row("runs per arm", f"{repeat} requested; used A={len(A)} B={len(b_it)}")
row("excluded runs", "; ".join(excluded) if excluded else "none")
try: memenv = (work / "B.memenv").read_text().strip()
except Exception: memenv = "unknown"
row("arm B memory environment", memenv)
row("-- GOVERNOR SELF-COST --", "")
row("sampling iterations", agg([x["sampling_iterations"] for x in A], nd=0))
row("per-iteration mean", agg([x["iter_mean_ms"] for x in A], " ms"))
row("per-iteration p95", agg([x["iter_p95_ms"] for x in A], " ms"))
row("per-iteration max", agg([x["iter_max_ms"] for x in A], " ms"))
row("governor CPU time", agg([x["governor_cpu_s"] for x in A], " s"))
row("governor CPU, % of one core", agg([x["governor_cpu_pct_one_core"] for x in A], " %"))
row("governor RSS start", agg([x["rss_start_mib"] for x in A], " MiB", 2))
row("governor RSS end", agg([x["rss_end_mib"] for x in A], " MiB", 2))
row("governor RSS delta (leak check)", agg([x["rss_delta_mib"] for x in A], " MiB", 2))
row("captures / restores", f"{o['captures']} / {o['restores']}  (idle: must be 0/0)")
row("pressure levels seen", f"{o['level_counts']}  stayed below advisory: {o['stayed_below_advisory']}")
row("-- ARENA COST --", "")
row("arena kind", o["arena_kind"])
row("arena budget (disk cap)", f"{o['arena_capacity_mib']:.0f} MiB")
peak = o["arena_peak_used_mib"]
row("arena peak on-disk use", f"{peak:.2f} MiB" if peak is not None
    else f"unknown - arena not readable by this uid (readable={o['arena_readable_by_governor']})")
row("images written while idle", f"{o['images_written_this_run']} (captures=0 => the arena was not written to)")
row("arena RAM pinned", f"{o['arena_ram_pinned_mib']:.1f} MiB")
row("free-space guard (fs_reserve)", f"{o['fs_reserve_mib']:.0f} MiB kept free on the filesystem")
row("budget vs protected cgroup", f"{o['arena_capacity_mib']:.0f} MiB disk / {o['cgroup_memory_max_mib']:.0f} MiB "
    f"= {o['arena_capacity_mib']/max(o['cgroup_memory_max_mib'],1):.1f}x (arena_size_mib is configurable)")
row("-- WORKLOAD THROUGHPUT A/B --", "")
row("iterations, governor running", agg(a_it, nd=0))
row("iterations, no governor", agg(b_it, nd=0))

if len(a_it) >= 2 and len(b_it) >= 2:
```

run_overhead.sh (302 lines) (continued)

```
    ma, mb = statistics.median(a_it), statistics.median(b_it)
    diff = mb - ma
    noise = max(max(a_it) - min(a_it), max(b_it) - min(b_it))
    row("difference (no-gov - gov)", f"{diff:+.0f} iterations ({100*diff/mb:+.2f}% of no-governor median)")
    row("run-to-run noise band", f"+/-{noise:.0f} iterations (widest within-arm spread)")
    if abs(diff) <= noise:
        # NB: never put '|' in cell text -- it is the awk formatter's column separator.
        row("verdict", f"INDISTINGUISHABLE from noise: abs({diff:+.0f}) <= band {noise:.0f}. "
                       f"No throughput cost can be claimed from this data.")
    else:
        row("verdict", f"difference {diff:+.0f} exceeds the noise band {noise:.0f}; "
                       f"a real cost of about {100*diff/mb:.2f}% is indicated")
else:
    row("difference", "NOT MEASURED - needs >=2 runs per arm; a single-run difference is not a measurement")
print("\n".join(rows))
PYEOF

{
  echo "PRESERVE idle-overhead report -- $(date '+%Y-%m-%d %H:%M:%S')"
  echo
  { echo "Metric|Value"; cat "$WORK/rows.psv"; } | awk -F'|' '
    function pad(s, n) { while (length(s) < n) s = s " "; return s }
    { rows++; if (NF > nc) nc = NF
      for (i = 1; i <= NF; i++) { cell[rows, i] = $i; if (length($i) > w[i]) w[i] = length($i) } }
    END {
      for (r = 1; r <= rows; r++) {
        out = ""
        for (i = 1; i <= nc; i++) { out = out pad(cell[r, i], w[i]); if (i < nc) out = out " | " }
        sub(/ +$/, "", out); print out
        if (r == 1) { sep = ""
          for (i = 1; i <= nc; i++) { d = ""; for (k = 0; k < w[i]; k++) d = d "-"
            sep = sep d; if (i < nc) sep = sep "-+-" } print sep }
      } }'
  echo
  echo "Idle = the normal case: the workload runs, there is no antagonist, pressure never"
  echo "reaches the advisory threshold, and the governor samples and decides not to act."
  echo
  echo "The arena is a byte-budgeted DIRECTORY on ext4, not pinned memory: the code only"
  echo "creates it (mkdir), stats the files in it, and calls statvfs. It holds no RAM at any"
  echo "time, and an idle governor writes nothing into it (images written: 0 above). A"
  echo "checkpoint of this workload is ~307 MiB, so the 2048 MiB budget is a cap on"
  echo "concurrent image sets, not a reservation. The 4x ratio against the 512 MiB protected"
  echo "cgroup is a configuration choice (arena_size_mib): in a real deployment it would be"
  echo "sized to the protected workload, not left at 4x. The real standing cost of an idle"
  echo "governor is the CPU and RSS above, plus fs_reserve_mib of filesystem free space that"
  echo "the admission test keeps spare."
  echo
  echo "Per-run logs and JSON: evidence/overhead-$STAMP/"
} | tee "$OUT"
echo
echo "report saved to $OUT"
```

scripts/video_demo.sh (363 lines)

```
#!/usr/bin/env bash
# ======================================================================
# scripts/video_demo.sh -- run the whole PRESERVE test set with one command
# and finish with three summary tables (for recording the project video).
#
#   ./scripts/video_demo.sh
#   ./scripts/video_demo.sh --render evidence/video-<ts>   # re-print the tables of THAT
#                                                          # run from its own logs (no runs)
#
# Runs, one at a time (raw output goes to evidence/video-<ts>/, the screen
# shows one banner line with an elapsed timer per step):
#   1-6   Scenarios A, B, C: baseline and PRESERVE      (run_demo.sh)
#   7-12  Fault injections on scenario B                (run_failsafe.sh's run_one + classify,
#                                                        governor --inject, self-verifying)
#   13    PID whitelist safety gate                      (cgroup.py / capture.py --refusal-demo)
#
# Nothing here re-implements an experiment or a check: every run goes through
# the existing scripts, and every number in the tables is parsed from the logs
# this invocation wrote (matrix.py build_row, baseline_result.json, governor
# SUMMARY lines, the failsafe classifier). A value that is not in this run's
# logs is printed as "-".
#
# Exit: 0 all checks passed, 1 some check failed, 75 another governor holds
# the run lock (nothing started).
# ======================================================================
set -uo pipefail

V_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

# Reuse run_failsafe.sh as a library: run_one (runs one scenario-B governor with
# --inject and collects its evidence), classify, refuse_if_locked, reap helpers.
FAILSAFE_LIB=1 source "$V_ROOT/run_failsafe.sh"
rm -f "$ROWS"   # the library created an empty evidence/.failsafe-<stamp>.rows on load

V_PY="$(command -v python3)"
V_RENDER_ONLY=0
if [ "${1:-}" = --render ]; then
  [ -d "${2:-}" ] || { echo "usage: $0 --render evidence/video-<ts>"; exit 2; }
  V_RENDER_ONLY=1
  V_DIR="$(cd "$2" && pwd)"
  V_TS="${V_DIR##*/video-}"
else
  V_TS="$(date +%Y%m%d-%H%M%S)"
  V_DIR="$V_ROOT/evidence/video-$V_TS"
  mkdir -p "$V_DIR"
fi
STAMP="video-$V_TS"                 # run_one writes evidence/failsafe-video-<ts>-<fault>/
ROWS="$V_DIR/failsafe.rows"         # classify appends one row per fault here
V_SCEN_LIST="$V_DIR/scenarios.list"  # scen|mode|rc|evidence_dir|wrapper_log
V_START=$(date +%s)
V_TOTAL=13
V_STEP=0

if [ -t 1 ] && [ -z "${NO_COLOR:-}" ]; then
  C_G=$'\e[32m'; C_R=$'\e[31m'; C_B=$'\e[1m'; C_D=$'\e[2m'; C_N=$'\e[0m'
else
  C_G=""; C_R=""; C_B=""; C_D=""; C_N=""
fi

locked_stop() {  # explain exit 75 and stop
  echo
  echo "################################################################################"
  echo "##  STOPPED -- another PRESERVE governor holds the run lock (exit 75)."
  echo "##  Nothing further was started. Wait for it to finish, then re-run."
  echo "##  $("${LOCKCLI[@]}" --status 2>&1)"
  echo "################################################################################"
  echo "partial logs: $V_DIR"
  exit 75
}

if [ "$V_RENDER_ONLY" -eq 0 ]; then
: > "$ROWS"
: > "$V_SCEN_LIST"
# ---- 1. lock + sudo -------------------------------------------------------
"${LOCKCLI[@]}" --status >/dev/null 2>&1 || locked_stop
echo "PRESERVE needs root for CRIU. Enter your sudo password once; it is kept alive."
if [ -t 0 ]; then
  sudo -v || { echo "sudo failed; cannot run the PRESERVE runs."; exit 1; }
else
  sudo -n true 2>/dev/null || { echo "need cached sudo (run 'sudo -v' first)"; exit 1; }
fi
( while kill -0 "$$" 2>/dev/null; do sudo -n true 2>/dev/null; sleep 50; done ) &
V_KEEPALIVE=$!
trap 'kill "$V_KEEPALIVE" 2>/dev/null' EXIT
```

scripts/video_demo.sh (363 lines) (continued)

```
85
86 # ---- 2. header ---------------------------------------------------------------
87 V_CRIU="$( (command -v criu >/dev/null && criu --version || /usr/sbin/criu --version) 2>/dev/null \
88           | awk '/^Version/ {print $2; exit}')"
89 V_ARENA_MIB="$(cd "$V_ROOT/src" && "$V_PY" -c \
90   'from config import Config; print(Config().get_str("admission", "arena_size_mib"))' 2>/dev/null)"
91 # Reap finished images first (never one still checkpointed and not restored), as the
92 # matrix and failsafe runners do, and record the occupancy line it reports.
93 V_ARENA_LINE="$(sudo "$V_PY" "$V_ROOT/src/governor.py" --config "$V_ROOT/preserve.conf" --reap-arena 2>&1 \
94                 | tee "$V_DIR/arena-reap.log" | grep -o 'ARENA: .*' | tail -1)"
95 {
96   echo "PRESERVE -- checkpoint instead of kill under memory pressure"
97   echo "kernel : $(uname -r)"
98   echo "date   : $(date '+%Y-%m-%d %H:%M:%S %Z')"
99   echo "CRIU   : ${V_CRIU:--}"
100  echo "arena  : budget ${V_ARENA_MIB:--} MiB; ${V_ARENA_LINE:-occupancy -}"
101  echo "logs   : $V_DIR"
102 } | tee "$V_DIR/header.txt"
103 echo
104
105 # ---- step runner: hide output, show banner + elapsed timer ------------------------
106 V_SPIN='|/-\'
107 run_step() {  # label logfile cmd...  -> sets STEP_RC, STEP_SECS
108   local label=$1 log=$2; shift 2
109   V_STEP=$((V_STEP + 1))
110   local tag; tag="$(printf '[%2d/%d]' "$V_STEP" "$V_TOTAL")"
111   local t0; t0=$(date +%s)
112   "$@" > "$log" 2>&1 < /dev/null &
113   local pid=$! i=0
114   while kill -0 "$pid" 2>/dev/null; do
115     printf '\r%s %-36s %s %4ds ' "$tag" "$label" "${V_SPIN:i++%4:1}" "$(( $(date +%s) - t0 ))"
116     sleep 1
117   done
118   wait "$pid"; STEP_RC=$?
119   STEP_SECS=$(( $(date +%s) - t0 ))
120   printf '\r%s %-36s ... done (%d s)%s\n' "$tag" "$label" "$STEP_SECS" "$( [ "$STEP_RC" -eq 75 ] && echo '  LOCKED')
 + "
121   [ "$STEP_RC" -eq 75 ] && locked_stop
122 }
123
124 scenario_step() {  # scen mode(baseline|live)
125   local scen=$1 mode=$2 label log dir
126   label="Scenario $scen - $( [ "$mode" = live ] && echo PRESERVE || echo baseline)"
127   log="$V_DIR/scenario-$scen-$mode.log"
128   if [ "$mode" = baseline ]; then run_step "$label" "$log" "$V_ROOT/run_demo.sh" --scenario "$scen" --baseline
129   else run_step "$label" "$log" "$V_ROOT/run_demo.sh" --scenario "$scen"; fi
130   dir="$(grep -E '^console log : ' "$log" | tail -1 | sed 's/^console log : //')"
131   [ -n "$dir" ] && dir="$(dirname "$dir")"
132   echo "$scen|$mode|$STEP_RC|$dir|$log" >> "$V_SCEN_LIST"
133 }
134
135 fault_step() {  # fault
136   local f=$1
137   run_step "Fault - $f" "$V_DIR/fault-$f.log" run_one "$f" "$f" live
138   # keep the run's evidence with this video session
139   if [ -d "$V_ROOT/evidence/failsafe-$STAMP-$f" ]; then
140     mv "$V_ROOT/evidence/failsafe-$STAMP-$f" "$V_DIR/fault-$f"
141     sed -i "s#$V_ROOT/evidence/failsafe-$STAMP-$f#$V_DIR/fault-$f#g" "$ROWS"
142   fi
143 }
144
145 gate_check() {
146   cd "$V_ROOT/src" || return 1
147   local rc=0
148   echo "## cgroup.py --refusal-demo"; "$V_PY" cgroup.py --refusal-demo || rc=1
149   echo "## capture.py --refusal-demo"; "$V_PY" capture.py --refusal-demo || rc=1
150   return $rc
151 }
152
153 # ---- 3-5. the runs ----------------------------------------------------------------
154 for s in A B C; do
155   scenario_step "$s" baseline
156   scenario_step "$s" live
157 done
158 for f in "${FAULTS[@]}"; do
159   fault_step "$f"
160 done
161 run_step "PID whitelist safety gate" "$V_DIR/gate.log" gate_check
162 echo "$STEP_RC" > "$V_DIR/gate.rc"
163 echo "$(( $(date +%s) - V_START ))" > "$V_DIR/runtime.s"
164 fi  # end of the runs (skipped with --render)
165 V_GATE_RC="$(cat "$V_DIR/gate.rc" 2>/dev/null || echo 1)"
166 V_RUNTIME="$(cat "$V_DIR/runtime.s" 2>/dev/null || echo 0)"
167
```

scripts/video_demo.sh (363 lines) (continued)

```
# ---- 6. parse THIS run's logs into three pipe-separated tables ---------------------
"$V_PY" - "$V_ROOT" "$V_DIR" "$V_GATE_RC" <<'PYEOF' > "$V_DIR/tables.psv" 2> "$V_DIR/parse-errors.log"
import json, re, sys
from pathlib import Path

root, vdir, gate_rc = Path(sys.argv[1]), Path(sys.argv[2]), int(sys.argv[3])
sys.path.insert(0, str(root / "src"))
from matrix import build_row  # the existing per-run extractor (dump/restore times, preserved work)

D = "-"
def f2(x): return D if x is None else f"{float(x):.2f}"
def read(p):
    try: return Path(p).read_text(errors="replace")
    except OSError: return ""

# ---------------- TABLE 1: scenarios ----------------
print("#T1")
print("Scenario|Mode|Valuable job outcome|Work lost (s)|Work preserved (s)|Capture time (s)|Restore time (s)|Hash ch
+ ain (mismatches)|Check")
COMPUTE_V = re.compile(r"compute_job continuity: (.+?) -- (.*)")
GROUP_V = re.compile(r"pipeline group continuity: (.+?) -- ")
CHAIN_C = re.compile(r"^verified \d+ lines up to iteration \d+: (\d+) mismatches, (\d+) restart", re.M)
CHAIN_G = re.compile(r"member (pipeline\S*) pid=\d+: .*?verified \d+ lines up to iteration \d+: (\d+) mismatches, (\
+ d+) restart")
MEMBER_CKPT = re.compile(r"group member \S+ pid=(\d+) checkpointed in the same image; .*?work ([\d.]+)s")
MEMBER_EXIT = re.compile(r"\] (pipeline(?:-child)?) pid=(\d+) (?:exited|killed)[^(\n]*\(last reported iteration \d+,
+  work ([\d.]+)s\)")
for line in read(vdir / "scenarios.list").splitlines():
    scen, mode, rc, ev, wlog = line.split("|")
    rc = int(rc)
    evp = Path(ev) if ev else None
    row = build_row(scen, mode, rc, 0, evp)
    console = read(evp / "console.log") if evp else ""
    wrapper = read(wlog)
    group = "pipeline group continuity" in console or (evp is not None and (evp / "pipeline_job.parent.progress.log"
+ ).exists())
    outcome = lost = kept = cap = res = D
    # hash chain: run_demo.sh step 4 (compute_job) or the governor's per-member check (group)
    chain, chain_ok = D, None
    if group:
        per = {}
        for m in CHAIN_G.finditer(console):
            per.setdefault(m[1], (int(m[2]), int(m[3])))
        if per:
            mm, rs = sum(v[0] for v in per.values()), sum(v[1] for v in per.values())
            chain = f"{mm} ({len(per)} chains)" + (f" +{rs} restarts" if rs else "")
            chain_ok = mm == 0 and rs == 0
    else:
        m = CHAIN_C.search(wrapper) or CHAIN_C.search(console)
        if m:
            chain = m[1] + (f" +{m[2]} restarts" if int(m[2]) else "")
            chain_ok = int(m[1]) == 0 and int(m[2]) == 0
    verdict = None
    if mode == "baseline":
        try:
            br = json.loads(read(evp / "baseline_result.json")) if evp else None
        except ValueError:
            br = None
        if br:
            if group and br.get("group"):
                g = br["group"]
                outcome = (f"killed ({g['members_died']}/{g['members_total']} members)"
                           if g["members_died"] else "not touched")
                lost = f2(g["work_destroyed_s"])
            elif not group:
                outcome = "killed" if br.get("compute_killed") else "not touched"
                lost = f2(br.get("compute_work_lost_s"))
        ok = rc == 0 and br is not None and outcome != D
    else:
        if row.get("dump_s") is not None: cap = f2(row["dump_s"])
        if row.get("restore_s") is not None: res = f2(row["restore_s"])
        if group:
            m = GROUP_V.search(console); verdict = m[1] if m else None
            members = sorted({m[1] for m in MEMBER_CKPT.finditer(console)})
            if verdict == "RESUMED":
                work = {m[1]: float(m[2]) for m in MEMBER_CKPT.finditer(console)}
                outcome, lost, kept = f"preserved ({len(work)}/{len(work)} members)", "0.00", f2(sum(work.values()))
            elif verdict == "KILLED":
                work = {m[2]: float(m[3]) for m in MEMBER_EXIT.finditer(console)}
                outcome, lost, kept = f"killed ({len(work)} members)", f2(sum(work.values())), "0.00"
            elif verdict == "RAN CONTINUOUSLY":
                outcome, lost, kept = "not touched", "0.00", "0.00"
            elif verdict:
                outcome = verdict.lower()
```

```
        else:
            m = COMPUTE_V.search(console); verdict = m[1] if m else None
            if verdict == "RESUMED":
                outcome, lost, kept = "preserved", "0.00", f2(row.get("preserved_s"))
            elif verdict == "KILLED":
                w = re.search(r"([\d.]+)s of work lost", m[2])
                outcome, lost, kept = "killed", f2(w[1]) if w else D, "0.00"
            elif verdict == "RAN CONTINUOUSLY":
                outcome, lost, kept = "not touched", "0.00", "0.00"
            elif verdict:
                outcome = verdict.lower()
        ok = rc == 0 and verdict == "RESUMED"
    if chain_ok is False:
        ok = False
    label = "PRESERVE" if mode == "live" else "baseline"
    print("|".join([scen, label, outcome, lost, kept, cap, res, chain, "PASS" if ok else "FAIL"]))

# ---------------- TABLE 2: fault injection ----------------
print("#T2")
print("Fault|Expected safe behaviour|Observed|Verified")
EXPECT = {
    "arena-too-small": "M3 denies capture -> STAND DOWN, no dump",
    "dump-fail":       "real dump fails (ENOSPC) -> clean up, STAND DOWN",
    "dump-timeout":    "dump times out -> clean up, STAND DOWN",
    "restore-blocked": "restore denied -> image kept on disk, no restore",
    "stale-pid":       "PID-reuse guard refuses -> STAND DOWN, no dump",
    "governor-dies":   "governor crashes -> kernel in charge, nothing frozen",
}
rows = {}
for line in read(vdir / "failsafe.rows").splitlines():
    parts = line.split("|")
    if len(parts) == 4:
        rows[parts[0]] = parts
for fault, expect in EXPECT.items():
    r = rows.get(fault)
    console = read(vdir / f"fault-{fault}" / "console.log")
    wrapper = read(vdir / f"fault-{fault}.log")
    if r is None:
        print(f"{fault}|{expect}|-|NO"); continue
    bad = any(w in "|".join(r[1:]) for w in ("UNCLASSIFIED", "INJECTION FAILED", "REFUSED"))
    if fault == "governor-dies":
        m = re.search(r"^exit code: (\d+)", wrapper, re.M)
        verified = (not bad and m is not None and m[1] == "137"
                    and "simulating an abrupt governor crash before any capture" in console)
        state = re.sub(r" at that point \(pids: ([\d ]+)\)",
                       lambda p: f" ({len(p[1].split())} workload pid(s) left running)", r[2])
        observed = state + (f"; governor exit {m[1]}" if m else "")
    else:
        verified = not bad and re.search(rf"FAULT RESULT\s+inject={re.escape(fault)}\s+VERIFIED", console) is not No
ne
        observed = r[2]
    if bad:  # classifier lacked evidence: say so and point at the log, never a guessed state
        observed = f"{r[1]} (fault-{fault}/console.log)"
    print(f"{fault}|{expect}|{observed}|{'YES' if verified else 'NO'}")

# ---------------- TABLE 3: safety gate ----------------
print("#T3")
print("Target|Refusal class|Refused")
gate = read(vdir / "gate.log")
n = 0
for line in gate.splitlines():
    m = re.match(r"^\s+(.+?)\s+-> (refused|ALLOWED)(?::\s*(.*))?$", line)
    if m:
        why = m[3] or ""
        c = re.search(r"refusing to \w+ pid \S+: (.*)$", why)
        print(f"{m[1]}|{c[1] if c else why or '-'}|{'YES' if m[2] == 'refused' else 'NO'}"); n += 1
        continue
    m = re.match(r"^pid (\d+): (refused before CRIU ran|dump was NOT refused)(?::\s*(.*))?$", line)
    if m:
        c = re.search(r"refusing to \w+ pid \S+: (.*)$", m[3] or "")
        print(f"CRIU dump of pid {m[1]}|{c[1] if c else '-'}|{'YES' if m[2].startswith('refused') else 'NO'}"); n +=
 1
if n == 0 or gate_rc != 0:
    print(f"refusal demos|exit status {gate_rc}, {n} refusal lines parsed|{'YES' if gate_rc == 0 and n else 'NO'}")
PYEOF

# ---- render: awk-aligned tables (no column(1)), saved plain, colored on screen -----
render_table() {  # section-tag title
  echo "$2"
  awk -v tag="$1" -F'|' '
    function pad(s, n) { while (length(s) < n) s = s " "; return s }
    /^#T/ { on = ($0 == "#" tag); next }
    on { rows++; if (NF > nc) nc = NF
         for (i = 1; i <= NF; i++) { cell[rows, i] = $i; if (length($i) > w[i]) w[i] = length($i) } }
```

```
330     END {
331       for (r = 1; r <= rows; r++) {
332         out = ""
333         for (i = 1; i <= nc; i++) { out = out pad(cell[r, i], w[i]); if (i < nc) out = out " | " }
334         sub(/ +$/, "", out); print out
335         if (r == 1) { sep = ""
336           for (i = 1; i <= nc; i++) { d = ""; for (k = 0; k < w[i]; k++) d = d "-"
337             sep = sep d; if (i < nc) sep = sep "-+-" } print sep }
338       } }' "$V_DIR/tables.psv"
339   echo
340 }
341
342 V_PASS=$(awk -F'|' '/^#T/ {t=$0; next} t=="#T1" && $NF=="PASS" {p++} (t=="#T2"||t=="#T3") && $NF=="YES" {p++} END {p
  + rint p+0}' "$V_DIR/tables.psv")
343 V_CHECKS=$(awk -F'|' '/^#T/ {t=$0; hdr=1; next} hdr {hdr=0; next} t!="" && NF>1 {n++} END {print n+0}' "$V_DIR/table
  + s.psv")
344 {
345   cat "$V_DIR/header.txt"
346   echo "total runtime: $((V_RUNTIME / 60)) min $((V_RUNTIME % 60)) s"
347   echo
348   render_table T1 "TABLE 1 - SCENARIOS"
349   render_table T2 "TABLE 2 - FAULT INJECTION (scenario B)"
350   render_table T3 "TABLE 3 - SAFETY GATE"
351   echo "RESULT: $V_PASS/$V_CHECKS checks passed"
352 } > "$V_DIR/summary.txt"
353
354 [ -t 1 ] && printf '\033[2J\033[H'
355 # Color only the verdict words (trailing column) on screen; the saved file stays plain.
356 sed -E -e "s/\| PASS\$/| ${C_G}PASS${C_N}/" -e "s/\| FAIL\$/| ${C_R}FAIL${C_N}/" \
357        -e "s/\| YES\$/| ${C_G}YES${C_N}/"   -e "s/\| NO\$/| ${C_R}NO${C_N}/" \
358        -e "s/^(TABLE .*)\$/${C_B}\1${C_N}/" \
359        -e "s#^RESULT: (.*)\$#${C_B}RESULT: \1${C_N}#" "$V_DIR/summary.txt"
360 echo
361 echo "${C_D}summary saved to $V_DIR/summary.txt${C_N}"
362 [ -s "$V_DIR/parse-errors.log" ] && echo "${C_R}parser messages: $V_DIR/parse-errors.log${C_N}"
363 [ "$V_PASS" -eq "$V_CHECKS" ] && [ "$V_CHECKS" -gt 0 ] && exit 0 || exit 1
```

5. Dataset / Evidence

evidence/matrix-20261022-043246.txt (62 lines shown)

Experiment matrix (scenarios A/B, repeated runs)

```
PRESERVE experiment matrix -- 2026-10-22 05:26:21  (repeat=10)
Arena at batch start (reaped before each batch):
  batch 1: ARENA: before 0.0 MiB in 0 image(s); deleted 0 (0.0 MiB); kept 0 still-checkpointed; now 0.0 of 2048 MiB
  batch 2: ARENA: before 0.0 MiB in 0 image(s); deleted 0 (0.0 MiB); kept 0 still-checkpointed; now 0.0 of 2048 MiB
  batch 3: ARENA: before 0.0 MiB in 0 image(s); deleted 0 (0.0 MiB); kept 0 still-checkpointed; now 0.0 of 2048 MiB
  batch 4: ARENA: before 0.0 MiB in 0 image(s); deleted 0 (0.0 MiB); kept 0 still-checkpointed; now 0.0 of 2048 MiB

Scenario | Mode         | OOM kills | Victim          | Work destroyed         | Work preserved         | Dump                | Restore             | Benefit
---------+--------------+-----------+-----------------+------------------------+------------------------+---------------------+---------------------+-------------
B        | baseline x10 | 1 [1-1]   | compute (10/10) | 12.66s [11.22s-13.10s] | n/a                    | n/a                 | n/a                 | n/a
B        | PRESERVE x10 | 0 [0-0]   | none (10/10)    | 0.00s [0.00s-0.00s]    | 12.19s [11.46s-12.34s] | 0.30s [0.26s-0.41s] | 0.27s [0.25s-0.30s] | yes (+12.19s)
A        | baseline x10 | 1 [1-1]   | hog (10/10)     | 0.54s [0.47s-0.56s]    | n/a                    | n/a                 | n/a                 | n/a
A        | PRESERVE x10 | 0 [0-0]   | none (10/10)    | 0.00s [0.00s-0.00s]    | 8.13s [7.53s-8.34s]    | 0.17s [0.15s-0.19s] | 0.13s [0.11s-0.23s] | none

Headline (Scenario B): the kernel OOM killer destroyed 12.66s of compute_job's work; PRESERVE checkpointed 12.19s of work and resumed it with nothing lost (medians, n=10 / n=10).

Work destroyed = accumulated work of processes the kernel killed (cost-model R at death).
Work preserved = compute_job work held at the checkpoint and resumed after restore.
Benefit        = PRESERVE row vs the same scenario's baseline in this session: yes (+Xs) if the baseline OOM killer killed compute_job and PRESERVE kept it (X = work preserved); none if the baseline never killed compute_job, so the preserved work was never at risk; n/a for baseline and dry-run rows.
With repeats: median [min-max].

Runs and evidence:
   1. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043246-baseline-B
   2. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043357-baseline-B
   3. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043508-baseline-B
   4. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043619-baseline-B
   5. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043730-baseline-B
   6. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043839-baseline-B
   7. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-043949-baseline-B
   8. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044059-baseline-B
   9. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044208-baseline-B
  10. B               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044318-baseline-B
  11. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044428-live-B
  12. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044612-live-B
  13. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044759-live-B
  14. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-044943-live-B
  15. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-045129-live-B
  16. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-045315-live-B
  17. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-045500-live-B
  18. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-045644-live-B
  19. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-045828-live-B
  20. B               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050014-live-B
  21. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050159-baseline-A
  22. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050256-baseline-A
  23. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050352-baseline-A
  24. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050447-baseline-A
  25. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050544-baseline-A
  26. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050641-baseline-A
  27. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050736-baseline-A
  28. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050833-baseline-A
  29. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-050928-baseline-A
  30. A               baseline  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051024-baseline-A
  31. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051120-live-A
  32. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051250-live-A
  33. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051420-live-A
  34. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051549-live-A
  35. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051719-live-A
  36. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-051850-live-A
```

evidence/matrix-20261022-043246.txt (62 lines shown) (continued)

```
37. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-052021-live-A
38. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-052151-live-A
39. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-052320-live-A
40. A               PRESERVE  rc=0   ok          /home/whitman/preserve/evidence/demo-20261022-052450-live-A
```

evidence/failsafe-20261022-160303.txt (33 lines shown)

Fail-safe matrix (newest)

```
PRESERVE fail-safe matrix -- 2026-10-22 16:14:22  (scenario B)
Arena at session start (reaped first): ARENA: before 307.1 MiB in 1 image(s); deleted 0 (0.0 MiB); kept 1 still-checkpointed; now 307.1 of 2048 MiB

Fault             | Reached capture?        | Process state at exit                                                                                  | Same as baseline?
------------------+-------------------------+--------------------------------------------------------------------------------------------------------+-----------------------------------
baseline (clean)  | n/a                     | compute_job killed by kernel OOM                                                                       | reference
none (clean live) | yes (dumped+restored)   | restored, resumed                                                                                      | better (work preserved)
arena-too-small   | no (stood down)         | stood down; compute_job killed by kernel OOM                                                           | yes (compute killed == baseline)
dump-fail         | attempted (dump failed) | dump failed (ENOSPC); survived the failure, then killed by kernel OOM                                  | yes (== baseline)
dump-timeout      | attempted (dump failed) | dump failed (timeout); survived the failure, then killed by kernel OOM                                 | yes (== baseline)
restore-blocked   | yes (dumped)            | checkpointed, not restored; image present on disk                                                      | work recoverable, job incomplete
stale-pid         | no (stood down)         | stood down; compute_job killed by kernel OOM                                                           | yes (compute killed == baseline)
governor-dies     | no (crashed first)      | governor died before capture; compute_job still under kernel at that point (pids: 16617 16628 ) | yes (kernel in charge, == no governor)

Reached capture?  = did the governor get as far as attempting a CRIU dump.
Process state     = the valuable compute_job's fate when the run ended.
Same as baseline? = baseline kills compute_job and destroys its work; a fail-safe
                    outcome is no worse ('yes'). A cell may say 'better' ONLY if the
                    valuable process was in a better state AT EXIT than in baseline.
failed dumps      : a dump that fails after the process is frozen and partially written
                    does not cost the process -- PRESERVE does not destroy work the
                    kernel would have kept. The process resumes unfrozen and is then left
                    to the kernel, which kills it exactly as in baseline. That is a
                    fail-safe guarantee, not an improvement over baseline.
restore-blocked   : a permanently blocked restore leaves the work intact on disk but the
                    job unfinished ('work recoverable, job incomplete'). That is better
                    than baseline only if the restore can eventually proceed; within the
                    run itself the job does not complete, so it does not beat baseline on
                    completion.
UNCLASSIFIED      : the run's log lacked the evidence to decide this row; read the named
                    evidence directory. Never treated as a pass.

Per-run evidence: evidence/failsafe-20261022-160303-<label>/
```

evidence/overhead-20261022-053739.txt (49 lines shown)

Idle-overhead report (newest)

```
PRESERVE idle-overhead report -- 2026-10-22 05:49:44

Metric                          | Value
--------------------------------+-----------------------------------------------------------------------------------
scenario / window               | B / 120.0s at 0.5s interval
runs per arm                    | 3 requested; used A=3 B=3
excluded runs                   | none
arm B memory environment        | confined: MemoryMax=512M, MemorySwapMax=0 (same limits as arm A)
-- GOVERNOR SELF-COST --        |
sampling iterations             | 240 [240-240]
per-iteration mean              | 1.158 ms [1.070 ms-1.177 ms]
per-iteration p95               | 1.514 ms [1.469 ms-1.565 ms]
per-iteration max               | 3.924 ms [2.352 ms-15.128 ms]
governor CPU time               | 0.278 s [0.265 s-0.282 s]
governor CPU, % of one core     | 0.232 % [0.221 %-0.235 %]
governor RSS start              | 20.54 MiB [20.53 MiB-20.54 MiB]
governor RSS end                | 20.60 MiB [20.59 MiB-20.60 MiB]
governor RSS delta (leak check) | 0.06 MiB [0.06 MiB-0.07 MiB]
captures / restores             | 0 / 0  (idle: must be 0/0)
pressure levels seen            | {'normal': 240}  stayed below advisory: True
-- ARENA COST --                |
arena kind                      | disk budget (ext4 directory), not pinned RAM
arena budget (disk cap)         | 2048 MiB
arena peak on-disk use          | unknown - arena not readable by this uid (readable=False)
images written while idle       | 0 (captures=0 => the arena was not written to)
arena RAM pinned                | 0.0 MiB
free-space guard (fs_reserve)   | 1024 MiB kept free on the filesystem
budget vs protected cgroup      | 2048 MiB disk / 512 MiB = 4.0x (arena_size_mib is configurable)
-- WORKLOAD THROUGHPUT A/B --   |
iterations, governor running    | 1739 [1700-1740]
iterations, no governor         | 1692 [1691-1710]
difference (no-gov - gov)       | -47 iterations (-2.78% of no-governor median)
run-to-run noise band           | +/-40 iterations (widest within-arm spread)
verdict                         | difference -47 exceeds the noise band 40; a real cost of about -2.78% is indicated

Idle = the normal case: the workload runs, there is no antagonist, pressure never
reaches the advisory threshold, and the governor samples and decides not to act.

The arena is a byte-budgeted DIRECTORY on ext4, not pinned memory: the code only
creates it (mkdir), stats the files in it, and calls statvfs. It holds no RAM at any
time, and an idle governor writes nothing into it (images written: 0 above). A
checkpoint of this workload is ~307 MiB, so the 2048 MiB budget is a cap on
concurrent image sets, not a reservation. The 4x ratio against the 512 MiB protected
cgroup is a configuration choice (arena_size_mib): in a real deployment it would be
sized to the protected workload, not left at 4x. The real standing cost of an idle
governor is the CPU and RSS above, plus fs_reserve_mib of filesystem free space that
the admission test keeps spare.

Per-run logs and JSON: evidence/overhead-20261022-053739/
```

evidence/video-20261025-003739/summary.txt (43 lines shown)

Full test-set summary (newest video run)

```
PRESERVE -- checkpoint instead of kill under memory pressure
kernel : 6.18.33.2-microsoft-standard-WSL2
date   : 2026-10-25 00:37:43 IST
CRIU   : 4.2.1
arena  : budget 2048 MiB; ARENA: before 614.1 MiB in 2 image(s); deleted 0 (0.0 MiB); kept 2 still-checkpointed; now 614.1 of 2048 MiB
logs   : /home/whitman/preserve/evidence/video-20261025-003739
total runtime: 17 min 13 s

TABLE 1 - SCENARIOS
Scenario | Mode     | Valuable job outcome    | Work lost (s) | Work preserved (s) | Capture time (s) | Restore time (s) | Hash chain (mismatches) | Check
---------+----------+-------------------------+---------------+--------------------+------------------+------------------+-------------------------+------
A        | baseline | not touched             | 0.00          | -                  | -                | -                | 0                       | PASS
A        | PRESERVE | preserved               | 0.00          | 7.79               | 0.26             | 0.15             | 0                       | PASS
B        | baseline | killed                  | 12.46         | -                  | -                | -                | 0                       | PASS
B        | PRESERVE | preserved               | 0.00          | 11.25              | 0.31             | 0.32             | 0                       | PASS
C        | baseline | killed (2/2 members)    | 19.72         | -                  | -                | -                | 0 (2 chains)            | PASS
C        | PRESERVE | preserved (2/2 members) | 0.00          | 16.97              | 0.29             | 0.19             | 0 (2 chains)            | PASS

TABLE 2 - FAULT INJECTION (scenario B)
Fault           | Expected safe behaviour                            | Observed                                                                                                                | Verified
----------------+----------------------------------------------------+-------------------------------------------------------------------------------------------------------------------------+---------
arena-too-small | M3 denies capture -> STAND DOWN, no dump           | stood down; compute_job killed by kernel OOM                                                                            | YES
dump-fail       | real dump fails (ENOSPC) -> clean up, STAND DOWN   | dump failed (ENOSPC); survived the failure, then killed by kernel OOM                                                   | YES
dump-timeout    | dump times out -> clean up, STAND DOWN             | dump failed (timeout); survived the failure, then killed by kernel OOM                                                  | YES
restore-blocked | restore denied -> image kept on disk, no restore   | checkpointed, not restored; image present on disk                                                                       | YES
stale-pid       | PID-reuse guard refuses -> STAND DOWN, no dump     | stood down; compute_job killed by kernel OOM                                                                            | YES
governor-dies   | governor crashes -> kernel in charge, nothing frozen | governor died before capture; compute_job still under kernel (2 workload pid(s) left running); governor exit 137 | YES

TABLE 3 - SAFETY GATE
Target               | Refusal class                                  | Refused
---------------------+------------------------------------------------+--------
init                 | kernel/init PIDs are never touched             | YES
kernel               | kernel/init PIDs are never touched             | YES
this process         | that is the governor itself or its parent      | YES
parent shell         | that is the governor itself or its parent      | YES
non-integer          | pid is not an integer                          | YES
not launched by us   | not launched by this governor (not in whitelist) | YES
SIGHUP to self       | signal 1 is not in the allowed set             | YES
CRIU dump of pid 1   | kernel/init PIDs are never touched             | YES
CRIU dump of pid 3197 | that is the governor itself or its parent      | YES
CRIU dump of pid 3193 | that is the governor itself or its parent      | YES

RESULT: 22/22 checks passed
```

run/throughput.json (7 lines shown)

Persisted CRIU dump-throughput estimate

```
{
  "throughput_mib_s": 945.833,
  "last_observed_mib_s": 1196.304,
  "observations": 53,
  "ewma_alpha": 0.3,
  "updated": "2026-10-25 00:50:34"
}
```

evidence/telemetry.csv (50 lines shown)

Per-sample telemetry (PSI, memory, level, events) -- TRUNCATED: first 50 of 24388 lines shown

```
wall_time,elapsed_s,level,sys_some_avg10,cg_some_avg10,cg_full_avg10,mem_current_mib,mem_max_mib,swap_current_mib,usage_frac,oom_kill,running,preserved,event
1791915815.315,0.02,normal,0.0,0.0,0.0,0.7,512.0,0.0,0.001,0,1,0,launched compute pid=14182 into /user.slice/user-1000.slice/user@1000.service/preserve-test
1791915815.816,0.52,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915816.316,1.02,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915816.816,1.52,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915817.316,2.02,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915817.816,2.52,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915818.316,3.02,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915818.817,3.52,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915819.317,4.02,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915819.817,4.52,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915820.317,5.03,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915820.818,5.53,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915821.318,6.03,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915821.818,6.53,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915822.318,7.03,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915822.818,7.53,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915823.318,8.03,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915823.818,8.53,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915824.318,9.03,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915824.819,9.53,normal,0.0,0.0,0.0,136.7,512.0,0.0,0.267,0,1,0,
1791915825.319,10.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915825.819,10.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915826.319,11.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915826.819,11.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915827.320,12.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915827.820,12.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915828.320,13.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915828.820,13.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915829.320,14.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915829.821,14.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915830.321,15.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915830.821,15.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915831.321,16.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915831.821,16.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915832.321,17.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915832.822,17.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915833.322,18.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915833.822,18.53,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915834.322,19.03,normal,0.0,0.0,0.0,136.6,512.0,0.0,0.267,0,1,0,
1791915834.822,19.53,normal,0.0,0.0,0.0,136.8,512.0,0.0,0.267,0,1,0,
1791915835.346,20.05,normal,0.0,0.0,0.0,137.6,512.0,0.0,0.269,0,2,0,launched hog pid=14183 into /user.slice/user-1000.slice/user@1000.service/preserve-test
1791915835.823,20.53,normal,0.0,0.0,0.0,158.8,512.0,0.0,0.310,0,2,0,
1791915836.324,21.03,normal,0.0,0.0,0.0,174.8,512.0,0.0,0.341,0,2,0,
1791915836.824,21.53,normal,0.0,0.0,0.0,191.1,512.0,0.0,0.373,0,2,0,
1791915837.324,22.03,normal,0.0,0.0,0.0,207.1,512.0,0.0,0.404,0,2,0,
1791915837.824,22.53,normal,0.0,0.0,0.0,223.0,512.0,0.0,0.436,0,2,0,
1791915838.324,23.03,normal,0.0,0.0,0.0,239.0,512.0,0.0,0.467,0,2,0,
1791915838.824,23.53,normal,0.0,0.0,0.0,255.0,512.0,0.0,0.498,0,2,0,
1791915839.325,24.03,normal,0.0,0.0,0.0,271.0,512.0,0.0,0.529,0,2,0,
```